บทนำสู่อัลกอริทึมแบบขนาน
(ฉบับร่าง)

Introduction to Parallel Algorithms (DRAFT)

Guy E. Blelloch, Laxman Dhulipala และ Yihan Sun

1 กุมภาพันธ์ 2026

สารบัญ

1 บทนำ **3**

2 แบบจำลอง **4**

3 ความรู้พื้นฐาน **9**

4 ส่วนประกอบพื้นฐานบางประการ **12**

4.1 Scan . . . 12
4.2 Filter และ Flatten . . . 14
4.3 การค้นหา (Search) . . . 15
4.4 การผสาน (Merge) . . . 16
4.5 สมาชิกที่เล็กที่สุดลำดับที่ k (K-th Smallest) . . . 18
4.6 สรุป . . . 19

5 การเรียงลำดับ (Sorting) **20**

5.1 Mergesort . . . 20
5.2 Quicksort . . . 20
5.3 Sample Sort . . . 22
5.4 Counting Sort และ Radix Sort . . . 22
5.5 Semisort . . . 24

6 อัลกอริทึมกราฟ (Graph Algorithms) **25**

6.1 ฟังก์ชันพื้นฐานสำหรับกราฟ (Graph primitives) . . . 25
6.2 การค้นหาตามแนวกว้างแบบขนาน (Parallel breadth-first search) . . . 25
6.3 การแยกส่วนแบบเส้นผ่านศูนย์กลางต่ำ (Low-diameter decomposition) . . . 26
6.4 ความเชื่อมโยง (Connectivity) . . . 29
6.5 Spanners . . . 29

6.6 เซตอิสระสูงสุด (Maximal Independent Set) 30

7 ต้นไม้ทวิภาคแบบขนาน (Parallel Binary Trees) 34

7.1 ความรู้เบื้องต้น 34
7.2 อัลกอริทึม *join* สำหรับแต่ละรูปแบบการรักษาสมดุล 35
7.2.1 ต้นไม้ AVL 35
7.2.2 ต้นไม้แดง–ดำ 36
7.2.3 ต้นไม้สมดุลน้ำหนัก 38
7.2.4 Treaps 40
7.3 อัลกอริทึมที่ใช้ *join* 41
7.3.1 ฟังก์ชันช่วยสองตัว: *split* และ *join2* 41
7.4 ฟังก์ชันระหว่างเซตกับเซตที่ใช้ *join* 43
7.5 อัลกอริทึมต้นไม้อื่น ๆ ที่ใช้ *join* 44

8 แบบจำลองอื่น ๆ และการจำลอง (Other Models and Simulations) 48

8.1 PRAM 48
8.2 การจำลอง (Simulations) 48
8.3 ปัญหาการจัดตาราง (The Scheduling Problem) 48
8.4 การจัดตารางแบบละโมบ (Greedy Scheduling) 49
8.5 ตัวจัดตารางแบบขโมยงาน (Work Stealing Schedulers) 49

ภาคผนวก: รายการแก้คำผิดจากต้นฉบับ 52

1 บทนำ

เอกสารนี้มีจุดประสงค์เพื่อเป็นบทนำสู่อัลกอริทึมแบบขนาน (parallel algorithms) อัลกอริทึมและเทคนิคที่อธิบายไว้ในเอกสารนี้ครอบคลุมผลงานกว่า 40 ปีของนักวิจัยหลายร้อยคน งานด้านอัลกอริทึมแบบขนานชิ้นแรก ๆ ย้อนไปได้ถึงทศวรรษ 1970 ตัวอย่าง เช่น แนวคิดหลักของอัลกอริทึมการผสาน (merging) แบบขนานที่อธิบายในหัวข้อ 4.4 ปรากฏอยู่ในบทความปี 1975 ของ Leslie Valiant ผู้ได้รับรางวัลทัวริง (Turing Award) ผู้ได้รับรางวัลทัวริงอีกหลายคนก็มีส่วนร่วมในแนวคิดที่อยู่ในเอกสารนี้ รวมถึง Richard Karp, Robert Tarjan และ John Hopcroft

เอกสารนี้มุ่งเน้นแนวคิดสำคัญที่ผ่านการพิสูจน์ด้วยกาลเวลามาแล้ว หรือน่าจะยืนหยัดต่อไปได้ และน่าจะเป็นประโยชน์ต่อการออกแบบอัลกอริทึมแบบขนานทั้งในปัจจุบันและในอนาคตอันไกล

แม้ว่าเอกสารนี้จะเน้นทฤษฎีของอัลกอริทึมแบบขนาน แต่อัลกอริทึมและเทคนิคเชิงอัลกอริทึมในเอกสารนี้จำนวนมาก (หากไม่ใช่ส่วนใหญ่) ได้ถูกนำไปใช้งานจริงบนเครื่องมัลติคอร์ (multicore) สมัยใหม่แล้ว (เช่น แล็ปท็อป ไอโฟน หรือเซิร์ฟเวอร์ของคุณ) อัลกอริทึมเหล่านี้ส่วนใหญ่ (บางครั้งต้องปรับแต่งเล็กน้อย) ทำงานได้เร็วกว่าอัลกอริทึมแบบลำดับ (sequential) ที่ดีที่สุดบนเครื่องเดียวกันอย่างมาก แม้บนเครื่องที่มีจำนวนคอร์ไม่มากนัก

เอกสารนี้ยังอยู่ระหว่างการจัดทำ โปรดแจ้งเราหากพบข้อผิดพลาดใด ๆ และให้ข้อเสนอแนะเกี่ยวกับการนำเสนอ

Claude อธิบายเพิ่ม: วิธีอ่านเอกสารนี้

กล่องสีส้มแบบนี้เป็น**ส่วนที่ Claude เขียนเพิ่มเติม** เพื่อสรุป ยกตัวอย่าง และเชื่อมโยงเนื้อหา ไม่ใช่ส่วนของต้นฉบับ เนื้อหานอกกล่องทั้งหมดแปลจากต้นฉบับครบถ้วน ส่วนรูปทั้ง 32 รูปนำมาจากต้นฉบับโดยตรง (แปลเฉพาะคำบรรยายใต้รูป)

คำผิดในต้นฉบับ: จุดที่ต้นฉบับพิมพ์ผิดได้แก้ไว้แล้ว และมีเครื่องหมาย [แก้ n] กำกับ คลิกเพื่อดูว่าต้นฉบับเขียนว่าอะไร ฉบับที่ถูกคืออะไร และเพราะอะไร (รวมไว้ในภาคผนวกท้ายเล่ม) ถ้าจุดที่ผิดอยู่ในรูปซึ่งตัดมาจากต้นฉบับ ฉบับที่ถูกจะเขียนไว้ในคำบรรยายใต้รูป

เส้นทางการอ่านที่แนะนำ: บทที่ 2 (แบบจำลองและต้นทุน) → บทที่ 3 (เครื่องมือความน่าจะเป็น) → บทที่ 4 (ส่วนประกอบพื้นฐาน เช่น scan, filter) ซึ่งเป็น “อิฐ” ที่บทที่ 5–7 นำไปใช้ซ้ำ และบทที่ 8 อธิบายว่าทำไมการวิเคราะห์งาน–สแปนจึงใช้ทำนายเวลาจริงได้

2 แบบจำลอง

ในการวิเคราะห์ต้นทุนของอัลกอริทึม การมีแบบจำลอง (model) ที่ชัดเจนพร้อมนิยามของต้นทุนที่แน่นอนเป็นสิ่งสำคัญ ในกรณีแบบลำดับ เป็นที่ทราบกันดีว่าแบบจำลองเครื่องเข้าถึงหน่วยความจำแบบสุ่ม (Random Access Machine: RAM) ใช้งานได้ดีมาหลายปีแล้ว แบบจำลอง RAM มีจุดประสงค์เพื่อประมาณการทำงานของเครื่องแบบลำดับจริง ประกอบด้วยตัวประมวลผลเดียวที่มีรีจิสเตอร์จำนวนคงที่ ตัวนับคำสั่ง (instruction counter) และหน่วยความจำขนาดใหญ่ได้ไม่จำกัด คำสั่งประกอบด้วยคำสั่งระหว่างรีจิสเตอร์กับรีจิสเตอร์ (เช่น บวกค่าในรีจิสเตอร์สองตัวแล้วเก็บผลลัพธ์ไว้ในตัวที่สาม) คำสั่งควบคุม (เช่น การกระโดด) และความสามารถในการอ่านและเขียนตำแหน่งใดก็ได้ในหน่วยความจำ เพื่อวัตถุประสงค์ในการวิเคราะห์ต้นทุน แบบจำลอง RAM สมมติว่าทุกคำสั่งใช้เวลาหนึ่งหน่วย “เวลา” (ต้นทุน) ของการคำนวณจึงเป็นเพียงจำนวนคำสั่งทั้งหมดที่ทำตั้งแต่เริ่มต้นจนถึงคำสั่ง end สุดท้าย เพื่อให้สามารถเก็บพอยน์เตอร์ (pointer) ไปยังหน่วยความจำไว้ในรีจิสเตอร์หรือตำแหน่งหน่วยความจำได้ แต่ไม่อนุญาตให้ “เล่นตุกติก” ด้วยการเก็บค่าขนาดใหญ่เกินไป โดยส่วนใหญ่จะสมมติว่าสำหรับข้อมูลเข้าขนาด n แต่ละรีจิสเตอร์และตำแหน่งหน่วยความจำสามารถเก็บได้ $\Theta(\log n)$ บิต

RAM ไม่ได้มีเจตนาจะจำลองเวลาทำงานบนเครื่องจริงด้วยความแม่นยำระดับรอบสัญญาณนาฬิกาเลย ตัวอย่างเช่น มันไม่ได้จำลองว่าเครื่องสมัยใหม่มีลำดับชั้นของแคช (cache hierarchy) และดังนั้นการเข้าถึงหน่วยความจำแต่ละครั้งจึงมีต้นทุนไม่เท่ากัน การจำลองคุณลักษณะทั้งหมดของเครื่องสมัยใหม่จะนำไปสู่แบบจำลองที่ซับซ้อนมาก ซึ่งอาจใช้งานยากและยากที่จะสร้างความเข้าใจเชิงสัญชาตญาณ RAM จึงพยายามเรียบง่ายแทน ด้วยเหตุนี้ แม้ว่า RAM จะไม่ได้จำลองประสิทธิภาพของเครื่องจริงเครื่องใดอย่างแม่นยำ แต่ก็สามารถใช้ (และได้ถูกใช้มาแล้ว) อย่างได้ผลในการเปรียบเทียบอัลกอริทึมต่าง ๆ และทำความเข้าใจว่าประสิทธิภาพของอัลกอริทึมจะเปลี่ยนไปตามขนาดอย่างไร ด้วยเหตุผลเหล่านี้ แบบจำลอง RAM จึงควรใช้สำหรับการวิเคราะห์เชิงเส้นกำกับ (asymptotic) เท่านั้น (คือการวิเคราะห์แบบ big-O) RAM ยังเป็นเครื่องมือที่ช่วยให้เข้าใจเทคนิคเชิงอัลกอริทึมได้ดีขึ้น เช่น การแบ่งแยกและเอาชนะ (divide-and-conquer) และการโปรแกรมพลวัต (dynamic programming) เป็นต้น สุดท้ายและสำคัญคือ เราเขียนรหัสเทียม (pseudocode) หรือโค้ดจริงที่แปลงเป็นคำสั่งของ RAM ได้อย่างเป็นธรรมชาติ

ในบริบทของอัลกอริทึมแบบขนาน เราต้องการใช้แบบจำลองต้นทุนที่มีคุณสมบัติสำคัญเหมือนกัน กล่าวคือ เรียบง่าย ชี้นำผู้ใช้ไปสู่อัลกอริทึมที่มีประสิทธิภาพที่สุด ช่วยให้ผู้ใช้เข้าใจว่าอัลกอริทึมเปลี่ยนแปลงอย่างไรตามขนาดข้อมูลเข้า ทนทาน (robust) ต่อเครื่องที่หลากหลาย ช่วยให้ผู้ใช้เข้าใจเทคนิคเชิงอัลกอริทึม และสามารถแสดงออกได้อย่างเป็นธรรมชาติด้วยรหัสเทียมง่าย ๆ และโค้ดจริง งานยุคแรกเกี่ยวกับอัลกอริทึมแบบขนานส่วนใหญ่ใช้แบบจำลอง Parallel RAM (PRAM) [23] แบบจำลองนี้ประกอบด้วยตัวประมวลผล p ตัวที่ทำงานพร้อมเพรียงกันอย่างสมบูรณ์ (fully synchronous) และเข้าถึงหน่วยความจำร่วม (shared memory) ต้นทุนวัดในรูปของจำนวนตัวประมวลผลและจำนวนขั้นเวลา น่าเสียดายที่ PRAM ไม่ค่อยเหมาะกับรหัสเทียมแบบง่าย และส่งผลถึงโค้ดจริงด้วย เนื่องจากสมมติฐานว่ามีจำนวนตัวประมวลผลคงที่ ผู้ใช้จึงต้องกังวลเรื่องการจัดสรรงานให้ตัวประมวลผล

ในระยะหลัง นักวิจัยได้ใช้สิ่งที่เรียกว่าแบบจำลองงาน–สแปน (work-span) (หรือ work-depth) ซึ่งอัลกอริทึมยังคงสมมติว่ามีหน่วยความจำร่วมแบบเข้าถึงสุ่ม แต่อนุญาตให้สร้างงาน (task) (หรือโพรเซส) ขึ้นแบบพลวัต ต้นทุนวัดในรูปของจำนวนการดำเนินการทั้งหมด เรียกว่า*งาน* (work) และสายโซ่ของการพึ่งพา (dependence) ที่ยาวที่สุด เรียกว่า*ความลึก* (depth) หรือ*สแปน* (span) สิ่งนี้ทำให้การอธิบายและวิเคราะห์อัลกอริทึมง่ายขึ้น โดยไม่ต้องพูดถึงการจัดตารางงาน (scheduling) ให้กับตัวประมวลผล

ในเอกสารนี้เราใช้แบบจำลองงาน–สแปนที่ชื่อ MP-RAM ซึ่งเราคิดว่าน่าจะตอบโจทย์คุณสมบัติของแบบจำลองอัลกอริทึมแบบขนานที่ดีได้ดีที่สุด แบบจำลองนี้มีพื้นฐานมาจาก RAM แต่อนุญาตให้แตก (fork) โพรเซสใหม่ได้แบบพลวัต โชคดีที่ต้นทุนที่วิเคราะห์ได้ค่อนข้างคงเส้นคงวา (robust) ไม่ว่าจะวิเคราะห์ใน PRAM หรือ MP-RAM (หรือแบบจำลองอื่น ๆ) หากเรานิยามงานใน PRAM เป็นผลคูณของจำนวนตัวประมวลผลกับเวลา อัลกอริทึมเกือบทั้งหมดในเอกสารนี้จะมีงานเท่ากันทั้งใน PRAM และ MP-RAM หากเรานิยามสแปนใน PRAM เป็นเวลา อัลกอริทึมทั้งหมดในเอกสารนี้จะมีสแปนใน PRAM ที่ต่างจาก MP-RAM ไม่เกินตัวประกอบลอการิทึม

อาจไม่ชัดเจนนักว่าจะจับคู่โพรเซสแบบพลวัตเหล่านี้เข้ากับเครื่องจริงที่มีตัวประมวลผลจำนวนคงที่ได้อย่างไร เพื่อให้มั่นใจว่าทำได้ ในภายหลังเราจะแสดงวิธีออกแบบตัวจัดตาราง (scheduler) ที่จับคู่โพรเซสเข้ากับตัวประมวลผล และพิสูจน์ขอบเขตที่เชื่อม

โยงต้นทุนต่าง ๆ โดยเฉพาะอย่างยิ่ง เราจะแสดงรูปแบบต่าง ๆ ของความสัมพันธ์ระหว่างงาน–สแปนกับตัวประมวลผล–เวลาดังนี้

$$\max\left(\frac{W}{P}, D\right) \leq T \leq O\left(\frac{W}{P} + D\right) \tag{1}$$

เมื่อ W คืองาน D คือสแปน P คือจำนวนตัวประมวลผล และ T คือเวลาทำงานเมื่อใช้ตัวประมวลผล P ตัว รายละเอียดเพิ่มเติมเกี่ยวกับแบบจำลองอื่น ๆ และการจำลอง (simulation) ระหว่างแบบจำลองเหล่านั้นอยู่ในหัวข้อ 8

เมื่อประเมินการคำนวณแบบขนาน เราสนใจเป็นพิเศษในปริมาณสองอย่าง คืองานและสแปน เพราะแต่ละอย่างสะท้อนแง่มุมหนึ่งของการคำนวณแบบขนาน งานตามนิยามของมันเทียบเท่ากับเวลาทำงานของการคำนวณเดียวกันบนตัวประมวลผลตัวเดียว (หรือแบบลำดับ) บางคนอาจสงสัยว่าทำไมเราต้องสนใจความซับซ้อนเชิงเวลาแบบลำดับของอัลกอริทึมแบบขนาน ประการแรก งานทั้งหมดยังคงบ่งบอกว่าการคำนวณต้องใช้ทรัพยากรมากเพียงใด ซึ่งรวมถึงความสามารถในการคำนวณ พลังงาน ฯลฯ ที่สำคัญกว่านั้น ในมุมของเวลาทำงาน งานทั้งหมดส่งผลต่อเวลาในพจน์ W/P (ดูสมการ 1) ในเครื่องมัลติคอร์สมัยใหม่ จำนวนตัวประมวลผล P มักน้อยเมื่อเทียบกับงาน W เพราะงานมักเป็น $\Omega(n)$ สำหรับข้อมูลเข้าขนาด n ในขณะเดียวกัน บ่อยครั้ง D เล็กกว่า W อย่างมากในเชิงเส้นกำกับ (ดูรายละเอียดเพิ่มเติมด้านล่าง) ดังนั้นขอบเขตเวลาทำงานจึงถูกกำหนดโดยงานทั้งหมด W เป็นหลัก (dominated) เราเรียกอัลกอริทึมแบบขนานว่า*มีประสิทธิภาพเชิงงาน* (work-efficient) หากงานของมันเท่ากันในเชิงเส้นกำกับกับอัลกอริทึมแบบลำดับที่ดีที่สุดที่ทราบ เมื่อออกแบบอัลกอริทึมแบบขนาน เป้าหมายของเรามักเป็นการทำให้อัลกอริทึมมีประสิทธิภาพเชิงงาน หรืออย่างน้อยใกล้เคียง

สแปนของอัลกอริทึมแบบขนานสะท้อนอีกแง่มุมหนึ่งของการคำนวณ นั่นคือ เวลาทำงานจะเป็นเท่าใดเมื่อมีตัวประมวลผลจำนวนอนันต์ กล่าวอีกนัยหนึ่ง นี่คือ “ขอบเขตล่าง” ของเวลาทำงานไม่ว่าจะมีตัวประมวลผลกี่ตัวก็ตาม แม้ว่าดังที่กล่าวไว้ข้างต้น เวลาทำงานมักถูกจำกัดด้วยงาน W แต่สแปนบ่งบอกความสามารถของอัลกอริทึมในการได้ประสิทธิภาพที่ดีขึ้นเมื่อมีตัวประมวลผลมากขึ้น สิ่งนี้เรียกอีกอย่างว่า*ความสามารถในการขยายขนาด* (scalability) ของอัลกอริทึม[1] โดยสัญชาตญาณ เมื่อออกแบบอัลกอริทึมแบบขนาน เป้าหมายในแง่สแปนคือไม่ปล่อยให้สแปนกลายเป็นต้นทุนหลัก โดยทั่วไป เราต้องการจำกัดสแปนให้เป็นพหุลอการิทึม (polylogarithmic) ใน n

อีกปริมาณหนึ่งที่ได้จากงานและสแปนคือ*ความขนาน* (parallelism) ซึ่งนิยามง่าย ๆ ว่าเป็นงานหารด้วยสแปน มันบ่งบอกในเชิงเส้นกำกับว่าการคำนวณสามารถใช้ตัวประมวลผลได้อย่างมีประสิทธิภาพกี่ตัว หากงานเท่ากับสแปน ความขนานเป็น 1 และการคำนวณเป็นแบบลำดับ หากงานเป็น $O(n \log n)$ และสแปนเป็น $O(\log^2 n)$ ความขนานจะเป็น $O(n/\log n)$ ซึ่งถือว่าสูงมาก และไม่น่าจะเป็นคอขวดบนเครื่องส่วนใหญ่ในอีก 25 ปีข้างหน้า หากงานเป็น $O(n^2)$ และสแปนเป็น $O(n)$ ความขนานจะเป็น $O(n)$ ซึ่งสูงยิ่งกว่า แม้ว่าสแปนจะไม่เป็นพหุลอการิทึมก็ตาม

โดยสรุป จุดเน้นหลักควรอยู่ที่การออกแบบอัลกอริทึมที่มีประสิทธิภาพเชิงงานพร้อมความขนานที่ดี และในอุดมคติ (แต่ไม่จำเป็น) มีสแปนเป็นพหุลอการิทึม

แบบจำลองงาน–สแปนเป็นแบบจำลองต้นทุนที่เรียบง่าย ซึ่งระบุว่าควรประเมินต้นทุนรวมของอัลกอริทึมอย่างไร อย่างไรก็ตาม มันไม่ได้ระบุว่าการดำเนินการใดได้รับอนุญาตหรือไม่ได้รับอนุญาตในการคำนวณแบบขนาน เช่น รองรับการทำงานพร้อมกัน (concurrency) หรือการประสานจังหวะ (synchronization) แบบใด ในส่วนต่อไปนี้ เราจะแนะนำแบบจำลอง MP-RAM ซึ่งจะเป็นแบบจำลองหลักที่ใช้ในหนังสือเล่มนี้

MP-RAM เครื่อง Multi-Process Random-Access Machine (MP-RAM) ประกอบด้วยเซตของโพรเซสที่ใช้หน่วยความจำร่วมขนาดไม่จำกัด แต่ละโพรเซสรันคำสั่งของ RAM กล่าวคือ ทำงานตามโปรแกรมที่เก็บในหน่วยความจำ มีตัวนับโปรแกรม (program counter) ของตัวเอง มีรีจิสเตอร์ของตัวเองจำนวนคงที่ และรันคำสั่ง RAM มาตรฐาน MP-RAM ขยาย RAM ด้วยคำสั่ง `fork` ซึ่งรับจำนวนเต็มบวก k และแตกโพรเซสลูกใหม่ k โพรเซส แต่ละโพรเซสลูกได้รับจำนวนเต็มที่ไม่ซ้ำกันในช่วง $[1, \ldots, k]$ ในรีจิสเตอร์ตัวแรก และมีสถานะอื่นเหมือนกับโพรเซสแม่ (โพรเซสที่ทำการ fork) ทุกประการ ส่วนโพรเซสแม่จะมีรีจิสเตอร์นั้นเป็น 0 โพรเซสลูกทั้งหมดเริ่มต้นด้วยการรันคำสั่งถัดไป และโพรเซสแม่จะหยุดรอจนกว่าโพรเซสลูกทั้งหมดจะสิ้นสุด (ทำคำสั่ง `end`) คำสั่งแรกของโพรเซสแม่หลังจากโพรเซสลูกทั้งหมดสิ้นสุดเรียกว่าคำสั่ง `join` การคำนวณเริ่มต้นด้วยโพรเซสราก (root) เดียว และสิ้นสุดเมื่อ

[1]ในบริบทอื่น ๆ หลายแห่ง ความสามารถในการขยายขนาดหมายถึงความสามารถในการได้ประสิทธิภาพที่ดีเมื่อขนาดข้อมูลเข้าเพิ่มขึ้น ในอัลกอริทึมแบบขนาน โดยเฉพาะในหนังสือเล่มนี้ เราใช้คำนี้ในความหมายของความสามารถในการได้ประสิทธิภาพเพิ่มเมื่อจำนวนตัวประมวลผลเพิ่มขึ้น

โพรเซสรากนั้นจบ แบบจำลองนี้รองรับ*ความขนานแบบซ้อน* (nested parallelism) คือความสามารถในการ fork โพรเซสแบบซ้อนกัน หากโพรเซสรากไม่เคย fork เลย มันก็คือโปรแกรมแบบลำดับมาตรฐาน

การคำนวณใน MP-RAM นิยามอันดับบางส่วน (partial order) บนคำสั่ง โดยเฉพาะ (1) ทุกคำสั่งขึ้นอยู่กับคำสั่งก่อนหน้าในเธรดเดียวกัน (ถ้ามี) (2) คำสั่งแรกของทุกโพรเซสขึ้นอยู่กับคำสั่ง fork ของโพรเซสแม่ที่สร้างมันขึ้นมา และ (3) ทุกคำสั่ง join ขึ้นอยู่กับคำสั่ง end ของโพรเซสลูกทั้งหมดที่สร้างโดย fork ที่สอดคล้องกัน การพึ่งพาเหล่านี้นิยามอันดับบางส่วน *งาน*ของการคำนวณคือจำนวนคำสั่งทั้งหมด และ*สแปน*คือลำดับของคำสั่งที่พึ่งพากันที่ยาวที่สุด ตามปกติ อันดับบางส่วนสามารถมองเป็น DAG ได้ สำหรับการ fork เซตของโพรเซสลูกและการ join ที่สอดคล้องกัน สแปนของการคำนวณย่อยคือค่าสูงสุดของสแปนของโพรเซสลูก และงานคือผลรวม คุณสมบัตินี้มีประโยชน์ในการวิเคราะห์อัลกอริทึม และโดยเฉพาะในการเขียนความสัมพันธ์เวียนเกิด (recurrence) เพื่อหางานและสแปน

เราสมมติว่าผลลัพธ์ของการดำเนินการกับหน่วยความจำสอดคล้องกับอันดับทั้งหมด (total order) แบบใดแบบหนึ่งบนคำสั่ง (เรียกว่า linearization) ที่รักษาอันดับบางส่วนไว้ กล่าวคือ การอ่านจะคืนค่าของการเขียนครั้งก่อนหน้าไปยังตำแหน่งเดียวกันตามอันดับทั้งหมดนั้น การเลือกอันดับทั้งหมดอาจส่งผลต่อผลลัพธ์ของโปรแกรม เนื่องจากโพรเซสสามารถสื่อสารกันผ่านหน่วยความจำร่วม ดังนั้นโดยทั่วไปการคำนวณจึงอาจไม่เป็นเชิงกำหนด (nondeterministic) คำสั่งสองคำสั่งเรียกว่า*ทำงานพร้อมกัน* (concurrent) หากไม่มีอันดับระหว่างกัน และเรียกว่า*มีอันดับ* (ordered) ในกรณีอื่น คำสั่งสองคำสั่ง*ขัดแย้งกัน* (conflict) หากคำสั่งหนึ่งเขียนไปยังตำแหน่งหน่วยความจำที่อีกคำสั่งอ่านหรือเขียนตำแหน่งเดียวกัน เรากล่าวว่าคำสั่งสองคำสั่ง*แข่งขันกัน* (race) หากทำงานพร้อมกันและขัดแย้งกัน หากไม่มีการแข่งขันในการคำนวณ และไม่มีแหล่งอื่นของความไม่เป็นเชิงกำหนด ทุกอันดับเชิงเส้นจะให้ผลลัพธ์เดียวกัน เพราะทุกคู่ของคำสั่งที่ขัดแย้งกันจะถูกจัดอันดับโดยอันดับบางส่วน (มิฉะนั้นจะเป็นการแข่งขัน) และดังนั้นจะต้องปรากฏในลำดับสัมพัทธ์เดียวกันในทุกการทำให้เป็นเชิงเส้น อันดับเชิงเส้นหนึ่งที่เฉพาะเจาะจงคือการวนแบบลำดับจากโพรเซสลูกแรกถึงสุดท้ายในแต่ละ fork เราเรียกสิ่งนี้ว่า*อันดับแบบลำดับ* (sequential ordering)

รหัสเทียม (Pseudocode) รหัสเทียมของเราจะดูเหมือนโค้ดแบบลำดับมาตรฐาน ยกเว้นการเพิ่มโครงสร้างสองแบบสำหรับแสดงความขนาน โครงสร้างแรกคือลูปขนาน ซึ่งระบุด้วย **parFor** ตัวอย่างเช่น ลูปต่อไปนี้ใช้ฟังก์ชัน f กับแต่ละสมาชิกของอาร์เรย์ A และเขียนผลลัพธ์ลงในอาร์เรย์ B:

$$
\begin{array}{l}
\textbf{parfor } i \texttt{ in } [0 : |A|] \\
\quad B[i] \leftarrow f(A[i])
\end{array}
$$

ในรหัสเทียม $[s : e]$ หมายถึงลำดับของจำนวนเต็มตั้งแต่ s (รวม) ถึง e (ไม่รวม) และ $\leftarrow$ หมายถึงการกำหนดค่า อาร์เรย์ของเราเริ่มต้นที่ศูนย์ (zero based) ลูปขนานที่มี n รอบสามารถนำไปใช้ใน MP-RAM ได้อย่างง่ายดายโดย fork โพรเซสลูก n โพรเซส ให้แต่ละโพรเซสลูกทำงานในตัวลูป (loop body) แล้วจบโพรเซสลูกแต่ละตัว งานของลูปขนานคือผลรวมของงานของตัวลูป สแปนคือค่าสูงสุดของสแปนของตัวลูป

โครงสร้างที่สองคือ*parallel do* ซึ่งเพียงแค่รันคำสั่งจำนวนหนึ่งแบบขนาน ในรหัสเทียมเราใช้เครื่องหมายอัฒภาค (semicolon) เพื่อแสดงลำดับของคำสั่งแบบลำดับ และใช้แถบคู่ (||) เพื่อแสดงคำสั่งแบบขนาน ตัวอย่างเช่น โค้ดต่อไปนี้จะหาผลรวมของสมาชิกในอาร์เรย์

$$
\begin{array}{l}
\textsc{sum}(A) = \\
\quad \textbf{if } (|A| = 1) \textbf{ then return } A[0] \\
\quad \textbf{else} \\
\qquad l \leftarrow \textsc{sum}(A[0 : |A|/2]) \parallel \\
\qquad r \leftarrow \textsc{sum}(A[|A|/2 : |A|]); \\
\qquad \textbf{return } l + r
\end{array}
$$

โครงสร้าง || ในโค้ดบ่งบอกว่าคำสั่งทั้งสองที่มีการเรียก SUM แบบเวียนเกิดควรทำแบบขนาน อัฒภาคก่อน return บ่งบอกว่าการเรียกเวียนเกิดแบบขนานทั้งสองต้องเสร็จสิ้นก่อนนำผลลัพธ์มาบวกกัน ในรหัสเทียมของเรา เราใช้สัญกรณ์ $A[s : e]$ เพื่อระบุส่วน

ย่อย (slice) ของอาร์เรย์ระหว่างตำแหน่ง s (รวม) และ e (ไม่รวม) หาก s (หรือ e) ว่าง หมายความว่าส่วนย่อยเริ่ม (สิ้นสุด) ที่จุดเริ่มต้น (จุดสิ้นสุด) ของอาร์เรย์ การหาส่วนย่อยใช้งานและสแปน $O(1)$ เพราะเพียงแค่ต้องติดตามค่าออฟเซ็ต (offset)

โครงสร้าง $\|$ สอดคล้องโดยตรงกับ fork ใน MP-RAM โดยให้โพรเซสลูกตัวแรกและตัวที่สองรันคำสั่งทั้งสองตามลำดับ ในทำนองเดียวกับ parFor งานของ $\|$ คือผลรวมของงานของคำสั่ง และสแปนคือค่าสูงสุดของสแปนของคำสั่ง สำหรับตัวอย่าง $\textsc{sum}$ งานโดยรวมสามารถเขียนเป็นความสัมพันธ์เวียนเกิดได้ดังนี้

$$W(n) = W(n/2) + W(n/2) + O(1) = 2W(n/2) + O(1)$$

ซึ่งมีผลเฉลยเป็น $O(n)$ และสแปนเป็น

$$D(n) = \max(D(n/2), D(n/2)) + O(1) = D(n/2) + O(1)$$ [แก้ 1]

ซึ่งมีผลเฉลยเป็น $O(\log n)$

สิ่งสำคัญที่ควรทราบคือ ลูป parallel-for และคำสั่ง parallel-do ($\|$) สามารถซ้อนกันได้ตามต้องการ

การ fork แบบทวิภาคหรือแบบใดก็ได้ (Binary or Arbitrary Forking) อัลกอริทึมบางตัวของเราใช้เพียงการ fork แบบทวิภาค (binary forking) ในขณะที่บางตัวใช้การ fork แบบ n ทางใดก็ได้ สิ่งนี้สร้างความแตกต่างบ้างเมื่อเราพูดถึงการจัดตาราง MP-RAM ลงบนตัวประมวลผลจำนวนคงที่ และอาจสร้างความแตกต่างในขอบเขตของสแปน เราจะใช้คำว่า MP-RAM แบบ fork ทวิภาค (binary-forking MP-RAM) หรือเรียกสั้น ๆ ว่า*แบบจำลอง fork ทวิภาค* (binary-forking model) เพื่อระบุรุ่นที่อนุญาตเฉพาะการ fork แบบทวิภาค ในบางกรณีเราให้ขอบเขตแยกกันสำหรับ MP-RAM และแบบจำลอง fork ทวิภาค เราสามารถสร้าง fork แบบ n ทางจาก fork แบบทวิภาคได้เสมอ โดยสร้างต้นไม้ของ fork ทวิภาคที่มีสแปน $\log n$ ดังนั้นโดยทั่วไปวิธีนี้อาจเพิ่มสแปน แต่ก็ไม่เสมอไป ในกรณีเหล่านี้เราจะใช้ **parfor2** เพื่อระบุว่าเราใช้ต้นไม้ในการ fork การเรียกแบบขนาน n ครั้ง

การจัดตาราง (Scheduling) เห็นได้ชัดว่า MP-RAM เป็นแบบจำลองเชิงนามธรรม เพราะเราไม่สามารถสร้างตัวประมวลผลใหม่ได้เพียงแค่เรียกคำสั่ง fork ในความเป็นจริง โพรเซสของ MP-RAM จะต้องถูกจัดตารางลงบนตัวประมวลผลจริงของเครื่อง นี่คือหน้าที่ของ*ตัวจัดตาราง* (scheduler) ตัวจัดตารางต้องเก็บคิวงานบางรูปแบบแล้วกระจายงานไปยังตัวประมวลผล เมื่อมีการ fork งาน (โพรเซส) ใหม่จะถูกเพิ่มเข้าคิว และเมื่อตัวประมวลผลทำงานของมันเสร็จ มันจะไปที่คิวเพื่อเอางานเพิ่ม ในที่นี้เราใช้คำว่า “คิว” ในความหมายทั่วไป เพราะมันอาจถูกนำไปใช้แบบกระจาย และอาจเป็นแบบเข้าก่อนออกก่อน (first-in first-out) หรือเข้าหลังออกก่อน (last-in first-out)[แก้ 2] หรือผสมกันแบบใดก็ได้ การสร้างตัวจัดตารางจะเป็นหัวข้อในบทต่อ ๆ ไป และจะใช้เพื่อแสดงขอบเขตดังที่ให้ไว้ในสมการ 1

คำสั่งเพิ่มเติม (Additional Instructions) ในบริบทแบบขนาน การเพิ่มคำสั่งบางอย่างที่จัดการหน่วยความจำนั้นมีประโยชน์ คำสั่งที่ใช้กันทั่วไปได้แก่ test-and-set (TS), compare-and-swap (CAS), fetch-and-add (FA) และ priority-write (PW) คำสั่งพื้นฐานเหล่านี้เป็น*อะตอมิก* (atomic) หมายความว่าแบ่งแยกไม่ได้ แม้จะทำงานพร้อมกันกับคำสั่งอื่น คำสั่งพื้นฐานเหล่านี้จะทำงานเสร็จสมบูรณ์อย่างถูกต้อง หรือเสมือนว่าไม่ได้ทำงานเลย (ไม่ส่งผลต่อสถานะของหน่วยความจำร่วม) คำสั่งเหล่านี้ได้รับการรองรับโดยสถาปัตยกรรมสมัยใหม่ส่วนใหญ่ ไม่ว่าโดยตรงหรือโดยอ้อม เราจะพูดถึงแบบจำลองของเราที่เปิดใช้การดำเนินการเหล่านี้อย่างใดอย่างหนึ่งว่าเป็นรุ่น TS-, FA- และ PW- ของ MP-RAM

คำสั่ง $\texttt{testAndSet}(x)$ (TS) รับการอ้างอิงไปยังตำแหน่งหน่วยความจำ x ตรวจสอบว่าค่าของ x เป็นเท็จหรือไม่ ถ้าใช่จะตั้งค่าเป็นจริงแบบอะตอมิกแล้วคืนค่าจริง ถ้าเป็นจริงอยู่แล้วจะคืนค่าเท็จ

คำสั่ง $\texttt{compareAndSwap}(x, o, n)$ (CAS) รับการอ้างอิงไปยังตำแหน่งหน่วยความจำ x ตรวจสอบว่าค่าของ x เท่ากับ o หรือไม่ ถ้าใช่ คำสั่งจะเปลี่ยนค่าเป็น n และคืนค่าจริง ถ้าไม่ คำสั่งจะไม่ทำอะไรและเพียงคืนค่าเท็จ

คำสั่ง $\texttt{fetchAndAdd}(x, y)$ (FA) รับการอ้างอิงไปยังตำแหน่งหน่วยความจำ x และค่า y แล้วบวก y เข้ากับค่าของ x โดยคืนค่าเดิม ต่างจาก TS หรือ CAS คำสั่ง FA จะบวก y เข้ากับค่าที่เก็บใน x สำเร็จเสมอ

คำสั่ง $\texttt{priorityWrite}(x, y)$ (PW) รับการอ้างอิงไปยังตำแหน่งหน่วยความจำ x และตรวจสอบว่าค่า y น้อยกว่าค่าปัจจุบันใน x หรือไม่ ถ้าใช่ จะเปลี่ยนค่าที่เก็บใน x เป็น y และคืนค่าจริง ถ้าไม่ จะไม่ทำอะไรและคืนค่าเท็จ

สำหรับการดำเนินการข้างต้นที่คืนค่าบูลีน (TS, CAS และ PW) เรากล่าวว่ามัน*สำเร็จ* (succeeds) หากคืนค่าจริง และ*ล้มเหลว* (fails) ในกรณีอื่น

ควรสังเกตว่าคำสั่งบางคำสั่งเหล่านี้สามารถใช้สร้างคำสั่งอื่นได้ และจึงมีพลังมากกว่า ในขณะที่บางคำสั่งทำไม่ได้ โดยปกติ TS ถือว่าเป็นคำสั่งที่อ่อนที่สุด เพราะไม่สามารถใช้จำลองคำสั่งอื่นใดได้เลย ในทางกลับกัน อีกสามคำสั่งสามารถใช้จำลอง TS ได้ทั้งหมด

การจัดสรรหน่วยความจำ (Memory Allocation) เพื่อให้ประเด็นเรื่องการจัดสรรหน่วยความจำแบบขนานง่ายขึ้น เราสมมติว่ามีคำสั่ง `allocate` ซึ่งรับจำนวนเต็มบวก n และจัดสรรบล็อกต่อเนื่องของตำแหน่งหน่วยความจำ n ตำแหน่ง โดยคืนพอยน์เตอร์ไปยังบล็อกนั้น และคำสั่ง `free` ซึ่งเมื่อได้รับพอยน์เตอร์ไปยังบล็อกที่จัดสรรไว้ จะปล่อยคืน (free) บล็อกนั้น

Claude อธิบายเพิ่ม: สรุปบทที่ 2 และสิ่งที่ควรรู้

- **งาน (work)** W = จำนวนคำสั่งทั้งหมด = เวลาถ้ามีตัวประมวลผลตัวเดียว **สแปน (span)** D = สายโซ่การพึ่งพาที่ยาวที่สุด = เวลาถ้ามีตัวประมวลผลไม่จำกัด
- **กฎการประกอบ:** คำสั่งแบบลำดับ (;) $\Rightarrow$ งานบวกกัน สแปนบวกกัน คำสั่งแบบขนาน (||, parfor) $\Rightarrow$ งานบวกกัน แต่สแปนเอา*ค่าสูงสุด*
- **ตัวอย่างตัวเลขของสมการ (1):** สมมติ $W = 10^9$, $D = 10^3$, $P = 64$ จะได้ $W/P \approx 1.6 \times 10^7 \gg D$ ดังนั้น $T \approx W/P$ คือเร็วขึ้นเกือบ 64 เท่า ความขนาน $W/D = 10^6$ บอกว่าใช้ตัวประมวลผลได้ถึงราวหนึ่งล้านตัวก่อนที่สแปนจะเริ่มเป็นคอขวด
- **ทำไมต้องสนใจ work-efficient:** ถ้าอัลกอริทึมขนานมีงาน $O(n \log n)$ แต่อัลกอริทึมแบบลำดับใช้ $O(n)$ บนเครื่อง 8 คอร์ เมื่อ n ใหญ่ ตัวประกอบ $\log n \approx 30$ จะกินประโยชน์ของ 8 คอร์จนหมด
- **การแข่งขัน (race)** คือสองคำสั่งที่ทำงานพร้อมกันและแตะตำแหน่งเดียวกันโดยมีอย่างน้อยหนึ่งการเขียน ถ้าไม่มี race ผลลัพธ์จะเหมือนรันแบบลำดับเสมอ ซึ่งช่วยให้ดีบักง่าย

เชื่อมโยง: test-and-set ถูกใช้ใน BFS (หัวข้อ 6.2, รูปที่ 15) ส่วน priority-write/WRITEMIN ถูกใช้ใน LDD (รูปที่ 16) และสมการ (1) ถูกพิสูจน์ในหัวข้อ 8.4–8.5

3 ความรู้พื้นฐาน

เราจะใช้อัลกอริทึมเชิงสุ่ม (randomized algorithms) อย่างมากในเอกสารนี้ และสมมติว่าผู้อ่านมีความเข้าใจทฤษฎีความน่าจะเป็นในระดับที่เหมาะสม (เช่น ตัวแปรสุ่ม ค่าคาดหมาย ความเป็นอิสระ ความน่าจะเป็นแบบมีเงื่อนไข ขอบเขตยูเนียน (union bound) อสมการของมาร์คอฟ (Markov's inequality)) เรามักสนใจขอบเขตหาง (tail bounds) ของการแจกแจงความน่าจะเป็น และการแสดงว่าความน่าจะเป็นของหางบางส่วนมีค่าน้อยมาก กล่าวคือ ความน่าจะเป็นสะสมที่อยู่เหนือค่าขีดแบ่งบางค่าในฟังก์ชันความหนาแน่นของความน่าจะเป็นมีค่าน้อยมาก ดังนั้นความน่าจะเป็นสะสมที่ไม่อยู่ในหางนี้จึงสูงมาก เข้าใกล้ 1

ขอบเขตหางเหล่านี้มักเป็นเชิงเส้นกำกับในรูปฟังก์ชันของพารามิเตอร์ n บางตัว โดยเฉพาะเราจะใช้นิยามต่อไปนี้สำหรับ "ความน่าจะเป็นสูง" (high probability)

นิยาม 3.1 (w.h.p., แบบไม่เป็นทางการ). $g(n) \in O(f(n))$ ด้วยความน่าจะเป็นสูง (*w.h.p.*) หาก $g(n) \in O(cf(n))$ ด้วยความน่าจะเป็นอย่างน้อย $1 - \left(\frac{1}{n}\right)^c$ สำหรับค่าคงที่ c_0 บางค่าและทุก $c \geq c_0$

นิยาม 3.2 (w.h.p., แบบเป็นทางการ). $g(n) \in O(f(n))$ ด้วยความน่าจะเป็นสูง (*w.h.p.*) หากมีค่าคงที่ c_0, k, n_0 ซึ่งสำหรับ $n \geq n_0$ และ $c \geq c_0$ ความน่าจะเป็นที่ $g(n) \leq ckf(n)$ มีค่าอย่างน้อย $1 - \frac{1}{n^c}$

นิยามนี้ทำให้เราสามารถทำให้ความน่าจะเป็นเข้าใกล้ 1 เท่าใดก็ได้โดยการเพิ่ม c และแม้ใช้ค่าคงที่ c ที่ไม่มากนัก (เช่น 3) ความน่าจะเป็นก็ใกล้ 1 มากแล้ว

ขอบเขตแบบความน่าจะเป็นสูงหรือขอบเขตหางเช่นนี้มีความสำคัญเป็นพิเศษในอัลกอริทึมแบบขนาน เพราะบ่อยครั้งเราไม่สามารถใช้ค่าคาดหมายได้อย่างได้ผล ในอัลกอริทึมแบบลำดับ เราเพียงแค่บวกเวลาเข้าด้วยกัน เราจึงหาเวลารวมของส่วนประกอบต่าง ๆ ได้โดยบวกเวลาคาดหมายของแต่ละส่วน และอาศัยความเป็นเชิงเส้นของค่าคาดหมาย (linearity of expectations) เพื่อหาเวลาคาดหมายรวม ในแบบขนาน เรามักรันส่วนประกอบหลายส่วนพร้อมกันแล้วรอให้ส่วนสุดท้ายเสร็จ ดังนั้นสแปนโดยรวมจึงเป็นค่าสูงสุดของสแปนของส่วนประกอบ ไม่ใช่ผลรวม

น่าเสียดายที่ไม่มีสิ่งที่เทียบเท่ากับความเป็นเชิงเส้นของค่าคาดหมายสำหรับค่าสูงสุด แนวทางที่ใช้กันทั่วไปแทนคือการแสดงว่าความน่าจะเป็นที่แต่ละส่วนประกอบจะไม่เสร็จภายในสแปนที่กำหนดมีค่าน้อยมาก ขอบเขตยูเนียนบอกเราว่า หากเรารวมความน่าจะเป็นเหล่านี้ ผลรวมจะเป็นขอบเขตบนของความน่าจะเป็นที่ส่วนใดส่วนหนึ่งจะไม่เสร็จภายในขอบเขตนั้น เป้าหมายจึงเป็นการแสดงว่าผลรวมนี้ยังคงน้อยเพียงพอ พจน์พหุนาม $(1/n)^c$ ในขอบเขตความน่าจะเป็นสูงมีความสำคัญ เพราะเมื่อเราคูณความเป็นไปได้ที่จะไม่อยู่ในขอบเขตนี้ด้วยส่วนประกอบขนาน n ส่วน เราจะได้ว่าความน่าจะเป็นที่ทั้งหมดอยู่ในขอบเขตของเรามีค่าอย่างน้อย $1 - (1/n)^{c-1}$[แก้ 3] เท่านี้เพียงพอแล้ว เพราะเราสามารถใช้ค่าคงที่ c ใดก็ได้

มีหลายวิธีในการหาขอบเขตหาง หลายวิธีอาศัยขอบเขตของเชอร์นอฟ (Chernoff) หรือโฮฟดิง (Hoeffding) ในเอกสารนี้เราจะใช้อสมการที่ค่อนข้างง่ายต่อไปนี้สำหรับขอบเขตหางเป็นส่วนใหญ่

ทฤษฎีบท 3.1. *พิจารณาเซตของตัวแปรสุ่มชี้บอก (indicator random variables) $X_1, \ldots X_n$ ซึ่ง $p(X_i = 1) \leq \bar{p}_i$ ภายใต้เงื่อนไขของทุกเหตุการณ์ที่เป็นไปได้ $X_j \in \{0, 1\}, i \neq j$[แก้ 4] ให้ $X = \sum_{i=1}^{n} X_i$ และ $\bar{E}[X] = \sum_{i=1}^{n} \bar{p}_i$ แล้ว*

$$\mathbf{Pr}[X \geq k] \leq \left(\frac{e\bar{E}[X]}{k}\right)^k.$$

พิสูจน์. ก่อนอื่นพิจารณากรณีพิเศษที่ $\bar{p}_i$ ทั้งหมดเท่ากันและมีค่าเป็น p ถ้า $X \geq k$ เราจะได้ว่ามีตัวแปรสุ่มอย่างน้อย k ตัวที่มีค่าเป็น 1 ความน่าจะเป็นที่ตัวแปร k ตัวใด ๆ ที่เจาะจงจะมีค่าเป็น 1 ทั้งหมด และตัวอื่นเป็นค่าใดก็ได้ ถูกจำกัดบนด้วย p^k ตอนนี้เราใช้ขอบเขตยูเนียนบนทางเลือกที่เป็นไปได้ $\binom{n}{k}$ แบบของตัวแปร k ตัว จะได้

$$\mathbf{Pr}[X \geq k] \leq p^k \binom{n}{k} < p^k \left(\frac{ne}{k}\right)^k = \left(\frac{pne}{k}\right)^k = \left(\frac{e\bar{E}[X]}{k}\right)^k$$

ในที่นี้เราใช้ขอบเขตบนมาตรฐานของสัมประสิทธิ์ทวินาม: $\binom{n}{m} < \left(\frac{ne}{m}\right)^m$

ต่อไปพิจารณากรณีที่ $\bar{p}_i$ ไม่เท่ากัน เราอ้างว่าการทำให้ไม่เท่ากันมีแต่จะทำให้ขอบเขตรวมของความน่าจะเป็นลดลง ซึ่งเห็นได้โดยเริ่มจากความน่าจะเป็นที่เท่ากันทั้งหมด แล้วย้ายความน่าจะเป็นบางส่วนจากตัวแปรหนึ่งไปยังอีกตัวหนึ่ง เซตย่อยขนาด k บางเซตจะรวมทั้งสองตัวแปร และความน่าจะเป็นเหล่านี้จะลดลง เพราะหากเซตของค่าที่ไม่เป็นลบมีผลรวมคงที่ ผลคูณของมันจะมีค่าสูงสุดเมื่อทั้งหมดเท่ากัน และลดลงเมื่อสองค่าห่างกันมากขึ้น สำหรับทุกเซตย่อยที่รวมเฉพาะตัวแรก จะมีเซตย่อยที่เทียบเท่ากันซึ่งรวมเฉพาะตัวที่สอง (คือเหมือนกันทุกอย่างยกเว้นสลับตัวแรกกับตัวที่สอง) ผลรวมของผลคูณของแต่ละคู่เหล่านี้จะไม่เปลี่ยนแปลง เซตย่อยขนาด k ใดที่ไม่รวมทั้งสองตัวก็จะไม่เปลี่ยนแปลงอย่างชัดเจน ดังนั้นผลรวมของความน่าจะเป็นของทุกเซตย่อยขนาด k จะมีค่าสูงสุดเมื่อความน่าจะเป็นทั้งหมดเท่ากัน □

อาจดูเหมือนว่าแทนที่จะจำกัดความน่าจะเป็นแบบมีเงื่อนไข เราสามารถเพียงกำหนดให้ X_i เป็นอิสระต่อกัน สำหรับหลายกรณีเท่านี้ก็เพียงพอ และเราสามารถใช้ความน่าจะเป็นและค่าคาดหมายที่แท้จริงได้ อย่างไรก็ตาม สำหรับบางกรณีที่มีประโยชน์ ตัวแปรสุ่มจะไม่เป็นอิสระต่อกัน แต่เราจะสามารถแสดงขอบเขตบนของความน่าจะเป็นแบบมีเงื่อนไขได้ ในกรณีเหล่านี้ การขาดความเป็นอิสระอาจทำให้ความน่าจะเป็นจริงดีกว่าขอบเขตนี้ แต่ไม่แย่กว่า ทฤษฎีบทตามที่กล่าวไว้จึงมีพลังมากกว่าทฤษฎีบทที่ต้องการความเป็นอิสระ กล่าวคือ มันครอบคลุมกรณีที่เป็นอิสระได้อย่างสมบูรณ์ และยังครอบคลุมได้มากกว่านั้น

วิธีหนึ่งในการจำขอบเขตหางนี้คือสังเกตความคล้ายคลึงกับอสมการของมาร์คอฟ:

$$\mathbf{Pr}[X \geq k] \leq \left(\frac{E[X]}{k}\right)$$

ความแตกต่างเพียงอย่างเดียวคือการเพิ่มกำลัง k และค่าคงที่ e อสมการของมาร์คอฟไม่ต้องการความเป็นอิสระใด ๆ แต่เนื่องจากไม่มีกำลัง k จึงอาจเป็นขอบเขตที่อ่อนกว่ามาก ดังนั้นคุณอาจจำสิ่งนี้ว่าเป็น “พลัง” (power) ของความเป็นอิสระ

เรามักสนใจพฤติกรรมเชิงเส้นกำกับ และจะใช้บทแทรกต่อไปนี้บ่อยครั้ง:

บทแทรก 3.1. *พิจารณาเซตของตัวแปรสุ่มชี้บอก $X_1, \ldots X_n$ โดย n แปรผันได้ ซึ่ง $p(X_i = 1) \leq \bar{p}_i$ ภายใต้เงื่อนไขของทุกเหตุการณ์ที่เป็นไปได้ $X_j \in \{0, 1\}, i \neq j$[แก้ 4] ให้ $X(n) = \sum_{i=1}^{n} X_i$ และ $f(n) = \sum_{i=1}^{n} \bar{p}_i$ แล้ว $X(n) \in O(f(n) + \log n)$ w.h.p.*

พิสูจน์. จากทฤษฎีบท 3.1 เราได้ว่า

$$\mathbf{Pr}[X(n) \geq g(n)] \leq \left(\frac{ef(n)}{g(n)}\right)^{g(n)}$$

เลือก $g(n) = 2ec(f(n) + \lg n)$ เราจะได้ว่า

$$\begin{aligned}
\mathbf{Pr}[X(n) \geq g(n)] &\leq \left(\tfrac{f(n)}{2f(n)+\ln n}\right)^{g(n)} \\
&\leq \left(\tfrac{1}{2}\right)^{g(n)} \\
&\leq \left(\tfrac{1}{2}\right)^{2ec \lg n} \\
&= \left(\tfrac{1}{n}\right)^{2ec} \\
&\leq \left(\tfrac{1}{n}\right)^{c}
\end{aligned}$$

ตอนนี้ชัดเจนว่า $g(n) \in O(c(f(n) + \log n))$[แก้ 5] และความน่าจะเป็นที่ $X(n) \leq g(n)$ มีค่าอย่างน้อย $1 - (1/n)^c$ ซึ่งสอดคล้องกับนิยามของความน่าจะเป็นสูงของเรา □

ตัวอย่างหนึ่งที่เราจะได้เห็นอีกหลายครั้ง คือกรณีเซตของตัวแปรสุ่มชี้บอกที่เป็นอิสระต่อกัน $X_1, \ldots X_n$ โดยแต่ละตัวมี $p(X_i = 1) = 1/i$ ในที่นี้ $f(n) = \sum_{i=1}^{n} 1/i < \ln n + 1$ ดังนั้นบทแทรกบอกเราว่าผลรวมของตัวแปรสุ่มชี้บอกเป็น $X(n) \in O(\lg n)$ w.h.p.

บทแทรก 3.1 ไม่สามารถใช้ได้เสมอไปในที่ที่ทฤษฎีบท 3.1 ใช้ได้ ตัวอย่างเช่น พิจารณาการโยนลูกบอล n ลูกลงในถัง n ใบแบบสุ่ม และจำกัดขนาดของถังที่ใหญ่ที่สุด ในที่นี้เราวิเคราะห์ขนาดของถังใบใดใบหนึ่งก่อน และ X_i บ่งบอกว่าลูกบอลลูกที่ i ตกลงไปในถังใบนั้นหรือไม่ เรามี $p_i = 1/n$ และให้จำนวนรวมในถังนั้นเป็น X เรามี $E[X] = 1$ จะได้

$$\mathbf{Pr}[X \geq k] \leq \left(\frac{e}{k}\right)^k$$

กำหนด $k = (c+1)e \ln n / \ln \ln n$ จะได้:

$$\begin{aligned}
\mathbf{Pr}[X \geq k] &\leq \left(\tfrac{\ln \ln n}{(c+1)\ln n}\right)^{(c+1)e \ln n/\ln \ln n} \\
&\leq \left(\tfrac{\ln \ln n}{\ln n}\right)^{(c+1)e \ln n/\ln \ln n} \\
&= \left(e^{-\ln \ln n + \ln \ln \ln n}\right)^{(c+1)e \ln n/\ln \ln n} \\
&\leq \left(e^{-.5 \ln \ln n}\right)^{(c+1)e \ln n/\ln \ln n} \\
&= \left(e^{-.5}\right)^{(c+1)e \ln n} \\
&= n^{-.5e(c+1)} \\
&\leq \left(\tfrac{1}{n}\right)^{c+1}
\end{aligned}$$

สิ่งนี้บอกเราว่า $X(n) \in O(\ln n / \ln \ln n)$ w.h.p. ซึ่งแน่นกว่า (tighter) ขอบเขตที่ได้จากบทแทรก 3.1 อยู่ตัวประกอบ $\ln \ln n$ นี่เป็นเพียงการจำกัดขนาดของถังใบเดียว แต่เราสามารถใช้ขอบเขตยูเนียนบนถัง n ใบได้โดยคูณความน่าจะเป็นด้วย n ซึ่งจำกัดบนความน่าจะเป็นที่ถังใบใดใบหนึ่งจะใหญ่กว่า $k = (c+1)e \ln n / \ln \ln n$ ด้วย $n\left(\frac{1}{n}\right)^{c+1} = \left(\frac{1}{n}\right)^c$ ดังนั้นความน่าจะเป็นที่ไม่มีถังใบใดใหญ่กว่า $O(c \ln n / \ln \ln n)$ มีค่าอย่างน้อย $1 - \left(\frac{1}{n}\right)^c$ และถังที่ใหญ่ที่สุดมีขนาด $O(\ln n / \ln \ln n)$ w.h.p.

Claude อธิบายเพิ่ม: สรุปบทที่ 3: เมื่อไรใช้เครื่องมือไหน

- **ค่าคาดหมายพอสำหรับ*งาน*** เพราะงานคือผลรวม และค่าคาดหมายของผลรวมคือผลรวมของค่าคาดหมาย (linearity of expectation)
- ***สแปน*ต้องใช้ w.h.p.** เพราะสแปนคือ*ค่าสูงสุด*ของงานย่อยที่ขนานกัน ค่าคาดหมายของค่าสูงสุดหาตรง ๆ ไม่ได้ สูตรที่ใช้ซ้ำทั้งเล่มคือ “พิสูจน์ว่างานย่อยหนึ่งชิ้นเกินขอบเขตด้วยความน่าจะเป็น $\leq 1/n^c$ แล้วใช้ union bound กับ n ชิ้น ได้ $\leq 1/n^{c-1}$”
- **สูตรลัดของบทแทรก 3.1:** ถ้าผลรวมของขอบเขตความน่าจะเป็นคือ $f(n)$ ผลรวมของตัวแปรชี้บอกจะเป็น $O(f(n) + \log n)$ w.h.p. เช่น $\sum 1/i = H_n \approx \ln n$ ให้ $O(\log n)$ w.h.p. (ใช้กับความลึกของ quicksort ในหัวข้อ 5.2)
- **ตัวเลขช่วยเห็นภาพ:** สำหรับ $n = 10^6$ และ $c = 3$ ความน่าจะเป็นที่จะล้มเหลวคือ 10^{-18} น้อยกว่าโอกาสที่ฮาร์ดแวร์จะเสียระหว่างรันมาก

เชื่อมโยง: ทฤษฎีบท 3.1 และขอบเขตเชอร์นอฟถูกใช้ซ้ำใน quicksort (5.2), LDD (6.3), MIS (6.6) และ work stealing (8.5)

4 ส่วนประกอบพื้นฐานบางประการ

ปัญหาหลายอย่าง เช่น การคำนวณผลบวกสะสม (prefix-sum) การผสานลำดับที่เรียงแล้ว และการกรอง มักเกิดขึ้นเป็นปัญหาย่อยเมื่อออกแบบอัลกอริทึมแบบขนานอื่น ๆ ในที่นี้เราจะกล่าวถึงส่วนประกอบพื้นฐาน (building blocks) เหล่านี้บางส่วน ในบางกรณีเราจะเห็นว่ามีการแลกเปลี่ยน (tradeoff) ระหว่างงานและสแปน ในบางกรณีเราจะเห็นว่าการ fork แบบใดก็ได้ให้พลังเพิ่มเติมในการลดสแปน

4.1 Scan

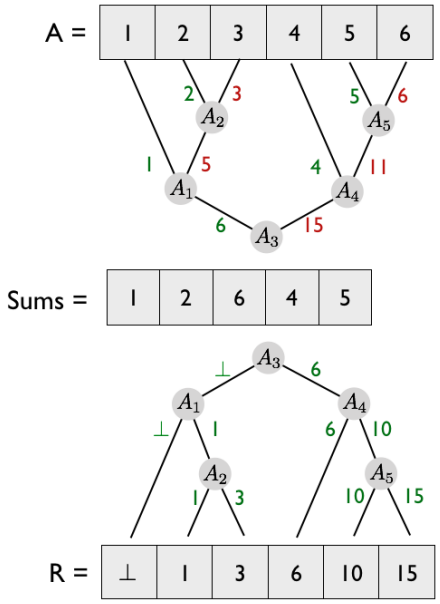
ฟังก์ชัน *scan* หรือ *prefix-sum* รับลำดับ A ฟังก์ชันที่มีสมบัติการเปลี่ยนหมู่ (associative) f และสมาชิกเอกลักษณ์ทางซ้าย (left identity) $\perp$ แล้วคำนวณค่า:

$$r_i = \begin{cases} \perp & i = 0 \\ f(r_{i-1}, A_{i-1}) & 0 < i \leq |A| \end{cases}$$

แต่ละ r_i คือ “ผลรวม” ของส่วนนำหน้า (prefix) $A[0 : i]$[แก้ 6] ของ A เทียบกับฟังก์ชัน f บ่อยครั้งผลลัพธ์ถูกคืนเป็นอาร์เรย์ความยาว $|A|$ โดยค่าที่ดัชนี i คือผลรวมของสมาชิกทั้งหมดที่อยู่ก่อนสมาชิกตัวที่ i พร้อมกับผลรวมทั้งหมด ดังนี้:

$$[r_0, \ldots, r_{|A|-1}], r_{|A|} \,.$$

ฟังก์ชัน scan มีประโยชน์เพราะช่วยให้เราคำนวณค่าสำหรับแต่ละสมาชิกในอาร์เรย์ที่ขึ้นอยู่กับค่าก่อนหน้าทั้งหมดได้ ซึ่งดูเหมือนจะเป็นแบบลำดับโดยธรรมชาติ แต่ที่จริงแล้วทำแบบขนานได้อย่างมีประสิทธิภาพ เรามักอ้างถึงการดำเนินการ PLUSSCAN ซึ่งเป็น scan ที่ f คือการบวก และ $\perp = 0$



รูปที่ 1: การสร้าง scan แบบเวียนเกิด *ขออภัย* SUMS *ในรูปตรงกับ* L *ในโค้ดและคำอธิบาย*

รหัสเทียมสำหรับการสร้าง scan แบบเวียนเกิดแสดงในรูปที่ 2 โค้ดนี้ทำงานกับฟังก์ชันที่มีสมบัติการเปลี่ยนหมู่ f ใดก็ได้ ในเชิงแนวคิด การสร้างนี้ท่อง (traverse) ต้นไม้ทวิภาคที่สมดุลซึ่งสร้างบนอาร์เรย์สองรอบ ทั้งสองรอบท่องต้นไม้แบบเวียนเกิดในลักษณะเดียวกันทุกประการ ต่างกันที่รอบแรกทำงานจากใบไปยังราก (คือใช้ f หลังการเรียกเวียนเกิด) ส่วนรอบที่สองทำงานจากรากไปยัง

ใบ (คือใช้ f ก่อนการเรียกเวียนเกิด) รูปที่ 2 แสดงการท่องทั้งสองให้เห็นภาพ แต่ละโหนดของต้นไม้สอดคล้องกับการเรียกเวียนเกิดหนึ่งครั้ง และโหนดภายในถูกกำกับด้วยสมาชิกใน A ที่สอดคล้องกับตำแหน่งจุดกึ่งกลาง m ของการเรียกนั้น

การท่องแรก scanUp คำนวณผลรวมบางส่วนของต้นไม้ย่อยทางซ้ายแล้วเก็บไว้ใน L โดยทำจากล่างขึ้นบน: แต่ละการเรียกแบ่ง A ที่จุดกึ่งกลาง m และแบ่ง L ออกเป็นสองช่วง ($L[0:m-1]$ และ $L[m:n-1]$) และสมาชิกหนึ่งตัวที่เหลืออยู่ตรงกลาง (คือ $L[m-1]$) สำหรับเขียนผลลัพธ์ มันเรียกเวียนเกิดบนส่วนที่แบ่งของ A และ L แต่ละการเรียกคืนผลรวมของช่วง A ของตนเทียบกับ f และแต่ละการเรียกภายในที่สอดคล้องกับโหนดภายในของต้นไม้ (คือไม่ใช่กรณีฐาน) จะเขียนผลลัพธ์ที่ได้รับจากการเรียกทางซ้ายลงใน $L[m-1]$ อาร์เรย์ L ของผลรวมบางส่วนมีขนาด $|A|-1$ เพราะต้นไม้ที่มีใบ n ใบมีโหนดภายใน $n-1$ โหนด

การท่องที่สอง scanDown ทำการท่องจากบนลงล่าง โดยรับค่า s จากโหนดแม่ (เอกลักษณ์ I สำหรับการเรียกระดับบนสุด) และเรียกเวียนเกิดสองครั้งแบบขนานบนลูกทางซ้ายและขวาอีกเช่นกัน แต่ละการเรียกส่ง s ที่ได้รับจากโหนดแม่ไปยังลูกทางซ้ายโดยตรง และส่ง $f(s, L[m-1])$ ไปยังลูกทางขวา จำไว้ว่า $L[m-1]$[แก้ 7] คือผลรวมของลูกทางซ้ายเทียบกับฟังก์ชันรวม f ใบของต้นไม้จะเขียนค่าที่โหนดแม่ส่งมาให้ลงในอาร์เรย์ผลลัพธ์ R

```
scanUp(A, L, f) =
  if (|A| = 1) then return A[0]
  else
    n ← |A|;
    m ← n/2;
    l ← scanUp(A[0 : m], L[0 : m − 1], f) ||
    r ← scanUp(A[m : n], L[m : n − 1], f);
    L[m − 1] ← l;
    return f(l, r)
```

```
scanDown(R, L, f, s) =
  if (|R| = 1) then R[0] = s; return;
  else
    n ← |A|;
    m ← |R|/2;
    scanDown(R[0 : m], L[0 : m − 1], s) ||
    scanDown(R[m : n], L[m : n − 1], f(s, L[m − 1]));
    return

scan(A, f, I) =
  L ← array[|A| − 1];
  R ← array[|A|];
  total ← scanUp(A, L, f);
  scanDown(R, L, f, I);
  return ⟨R, total⟩;
```

รูปที่ 2: ฟังก์ชัน scan *(โค้ด* scanDown *ในรูปมีจุดผิด ที่ถูกคือ: การเรียกเวียนเกิดครั้งแรกต้องเป็น* $\text{scanDown}(R[0:m], L[0:m-1], f, s)$*, บรรทัด* $n \leftarrow |A|$ *ต้องเป็น* $n \leftarrow |R|$ *และ* $R[0] = s$ *ต้องเป็น* $R[0] \leftarrow s$*)*[แก้ 8]

ต่อไปเราพิจารณาว่าทำไมอัลกอริทึมนี้ถูกต้อง ประการแรก สำหรับ scanUp ควรชัดเจนว่าค่าที่เขียนลงใน L เป็นผลรวมของต้นไม้ย่อยทางซ้ายจริง สำหรับ scanDown พิจารณาโหนด v ในต้นไม้และค่า s ที่ส่งมาให้ ค่าคงตัว (invariant) ที่อัลกอริทึมรักษาไว้คือค่า s เป็นผลรวมของค่าทั้งหมดที่อยู่ทางซ้ายของต้นไม้ย่อยที่มีรากที่ v เราอ้างว่า หากสิ่งนี้เป็นจริงที่โหนดแม่ มันจะเป็นจริงที่ลูกทั้งสอง มันเป็นจริงสำหรับรากเพราะไม่มีอะไรอยู่ทางซ้ายและเอกลักษณ์ถูกส่งเข้ามา มันเป็นจริงสำหรับลูกทางซ้ายของแต่ละโหนดเพราะได้รับ s โดยตรงและมีค่าทางซ้ายเหมือนกับโหนดแม่ มันยังเป็นจริงที่ลูกทางขวา เพราะค่าทางซ้ายของต้นไม้ย่อยทางขวาคือยูเนียนแบบไม่ซ้อนทับ (disjoint union) ของค่าทางซ้ายของต้นไม้ย่อยของ v ทั้งหมดกับค่าในต้นไม้ย่อยทางซ้ายของ v พอดี ดังนั้นการรวมผลรวมของสองส่วนนี้ด้วย f จะได้ผลรวมทั้งหมดสำหรับต้นไม้ย่อยทางขวา สุดท้าย เมื่ออัลกอริทึมไปถึงกรณีฐาน (ใบ) มันจะเขียนผลรวมของทุกอย่างทางซ้ายของมันลงในตำแหน่งที่เหมาะสมของ R พอดี

อัลกอริทึมของเราไม่ต้องการให้ f มีสมบัติการสลับที่ (commutative) เมื่อใดก็ตามที่มันรวมค่า มันจะรวมในลำดับที่ถูกต้องโดยค่าทางซ้ายใช้เป็นอาร์กิวเมนต์ซ้ายของ f และค่าทางขวาใช้เป็นอาร์กิวเมนต์ขวา

สมมติว่าฟังก์ชัน f ของเราใช้งาน $O(1)$ งานของ scanUp และ scanDown ได้จากความสัมพันธ์เวียนเกิดต่อไปนี้

$$W(n) = 2W(n/2) + O(1)$$

ซึ่งมีผลเฉลยเป็น $O(n)$ และสแปนเป็น

$$D(n) = D(n/2) + O(1)$$

ซึ่งมีผลเฉลยเป็น $O(\log n)$ สำหรับฟังก์ชัน f ที่มีต้นทุนอื่น เราต้องแทนต้นทุนของมันลงในความสัมพันธ์เวียนเกิดเพื่อหาต้นทุนรวม อัลกอริทึม SCAN ทำงานในแบบจำลอง fork ทวิภาคได้ เพราะเราเรียกเวียนเกิดเป็นคู่เท่านั้น

Claude อธิบายเพิ่ม: ตัวอย่าง scan แบบลงมือทำ

ให้ $A = [2, 1, 3, 4]$ และ $f = +, \perp = 0$ $\quad$ PLUSSCAN(A) ให้ $\langle [0, 2, 3, 6],\ 10\rangle$ (แต่ละช่องคือผลรวมของ*สมาชิกก่อนหน้า* ไม่รวมตัวเอง เรียกว่า exclusive scan)

- SCANUP: จากใบขึ้นราก คำนวณผลรวมของครึ่งซ้ายของทุกโหนดเก็บไว้ใน L: ที่ราก ครึ่งซ้ายคือ $2 + 1 = 3$ ที่โหนดซ้ายคือ 2 ที่โหนดขวาคือ 3
- SCANDOWN: จากรากลงใบ ส่ง s ไปทางซ้ายตามเดิม และส่ง $f(s, L[\cdot])$ ไปทางขวา ใบจะได้ $0, 2, 3, 6$ ตามลำดับ

ทำไมสำคัญ: scan เปลี่ยนงานที่ดูเป็นลำดับโดยธรรมชาติ ("ตำแหน่งของฉันขึ้นกับทุกตัวก่อนหน้า") ให้เป็นงานขนานที่มีสแปน $O(\log n)$ และเป็นแกนของ FILTER, FLATTEN, counting sort (รูปที่ 13) และ EDGEMAP (รูปที่ 14)

```
FILTER(A, p) =
  n ← |A|;
  F ← ARRAY[n];
  parfor i in [0 : n]
    F[i] ← p(A[i])
  ⟨X, m⟩ ← PLUSSCAN(F);
  R ← ARRAY[m];
  parfor i in [0 : n]
    if (F[i]) then R[X[i]] ← A[i]
  return R
```

รูปที่ 3: ฟังก์ชัน FILTER

```
FLATTEN(A) =
  sizes ← ARRAY(|A|);
  parfor i in [0 : |A|]
    sizes[i] ← |A[i]|
  ⟨X, m⟩ ← PLUSSCAN(sizes);
  R ← ARRAY(m);
  parfor i in [0 : |A|]
    o ← X[i];
    parfor j in [0 : |A[i]|]
      R[o + j] ← A[i][j]
  return R
```

รูปที่ 4: ฟังก์ชัน FLATTEN

ในกรณีพิเศษของ PLUSSCAN บนจำนวนเต็ม n ตัวในช่วง $[0 : n^c]$ สามารถปรับปรุงขอบเขตเป็นงาน $O(n)$ และสแปน $O(\log n / \log\log n)$ ได้ด้วย MP-RAM แบบหลายทาง [12] แนวทางนี้ค่อนข้างซับซ้อน โดยแบ่งเวิร์ด (word) ออกเป็นส่วน ๆ แล้ว scan แต่ละส่วนอย่างอิสระ และใช้การค้นตาราง (table lookup) เป็นส่วนประกอบพื้นฐาน

4.2 Filter และ Flatten

ฟังก์ชันพื้นฐาน *filter* รับข้อมูลเข้าเป็นลำดับ A และเพรดิเคต (predicate) p แล้วคืนอาร์เรย์ที่ประกอบด้วย $a \in A$ ซึ่ง $p(a)$ เป็นจริง ในลำดับเดียวกับใน A รหัสเทียมสำหรับฟังก์ชัน filter แสดงในรูปที่ 3 ขั้นแรกเราคำนวณอาร์เรย์ของแฟล็ก (flag) F โดยที่ $F[i] = p(A[i])$ กล่าวคือ $F[i] = 1$ ก็ต่อเมื่อ $A[i]$ เป็นสมาชิกที่ยังอยู่ (live) ซึ่งควรถูกคืนในอาร์เรย์ผลลัพธ์ ต่อมา เรา PLUSSCAN แฟล็กเพื่อจับคู่สมาชิกที่ยังอยู่แต่ละตัวเข้ากับดัชนีที่ไม่ซ้ำกันระหว่าง 0 ถึง m ซึ่งคือจำนวนสมาชิกที่ยังอยู่ทั้งหมด สุดท้าย เราจัดสรรอาร์เรย์ผลลัพธ์ R และ map บนแฟล็ก โดยเขียนสมาชิกที่ยังอยู่ที่ดัชนี i ลงใน $R[X[i]]$ เราทำขั้นตอนที่ map บนสมาชิก n ตัวเพียงจำนวนครั้งคงที่ ดังนั้นงานของ filter คือ $O(n)$ และสแปนคือ $O(\log n)$ เนื่องจาก PLUSSCAN

ฟังก์ชันพื้นฐาน *flatten* รับข้อมูลเข้าเป็นลำดับซ้อน (nested sequence) A (ลำดับของลำดับ) และคืนลำดับแบน R ที่ประกอบด้วยลำดับใน A ต่อกัน ตัวอย่างเช่น FLATTEN$([[3, 1, 2], [5, 1], [7, 8]])$ คืนลำดับ $[3, 1, 2, 5, 1, 7, 8]$

รหัสเทียมสำหรับฟังก์ชัน FLATTEN แสดงในรูปที่ 4 ขั้นแรกเราเขียนขนาดของแต่ละอาร์เรย์ใน A และใช้ PLUSSCAN เพื่อคำนวณขนาดของผลลัพธ์ ขั้นสุดท้ายคือ map บน $A[i]$ แบบขนาน และคัดลอกแต่ละลำดับไปยังตำแหน่งเฉพาะของมันในผลลัพธ์ โดยใช้ออฟเซ็ตที่ได้จาก PLUSSCAN

Claude อธิบายเพิ่ม: ตัวอย่าง filter

$A = [5, 8, 2, 9]$, $p(x) = (x > 4)$ $F = [1, 1, 0, 1]$ PLUSSCAN(F) ได้ $X = [0, 1, 2, 2]$ และ $m = 3$ จากนั้นทุก i ที่ $F[i] = 1$ เขียนพร้อมกันไปที่ $R[X[i]]$ ได้ $R = [5, 8, 9]$ โดยไม่มีการชนกัน เพราะแต่ละตัวได้ดัชนีของตัวเอง

หลักคิดที่ใช้บ่อย: "คำนวณแฟล็ก → scan หาตำแหน่ง → เขียนแบบขนาน" เรียกว่าเทคนิค *pack* ซึ่งใช้ใน quicksort (รูปที่ 12) และ Luby's MIS (รูปที่ 19)

4.3 การค้นหา (Search)

ปัญหาการค้นหาในลำดับที่เรียงแล้ว (sorted search) คือ กำหนดลำดับที่เรียงแล้ว A และคีย์ v ให้หาตำแหน่งของสมาชิกที่มากที่สุดใน A ที่น้อยกว่า v ปัญหานี้แก้ได้ด้วยการค้นหาแบบทวิภาค (binary search) ด้วยงานและสแปน $O(\log |A|)$ ในแบบขนาน เป็นไปได้ที่จะลดสแปนลงโดยแลกกับงานที่เพิ่มขึ้น แนวคิดคือใช้การค้นหาแบบ k ทาง แทนการค้นหาแบบทวิภาค

```
// finds which of k blocks contains v, returning block and offset
FINDBLOCK(A, v, k) =
  s ← |A|/k;
  r ← k;
  parfor i in [0 : k]
    if (A[i × s] < v and A[(i + 1) × s] > v)
    then r ← i
  return (A[r × s, (r + 1) × s], i × s)

SEARCH(A, v, k) =
  (B, o) = FINDBLOCK(A, v, min(|A|, k));
  if (|A| ≤ k) then return o
  else return o + SEARCH(B, v, k);
```

รูปที่ 5: ฟังก์ชัน SEARCH *(บรรทัด* **return** *ของ* FINDBLOCK *ในรูปที่ถูกคือ* $(A[r \times s : (r + 1) \times s],\ r \times s)$*)*[แก้ 9]

วิธีนี้ทำให้เราหาตำแหน่งได้ใน $O(\log_k |A|)$ รอบ โดยแต่ละรอบต้องเปรียบเทียบ k ครั้ง รูปที่ 5 แสดงรหัสเทียม แต่ละรอบซึ่งทำโดย FINDBLOCK ทำงานด้วยสแปนคงที่ โดยการเลือก $k = n_0^\alpha$ เมื่อ n_0 คือขนาดข้อมูลเข้าตั้งต้น และสำหรับ $0 < \alpha \leq 1$ อัลกอริทึมจะเรียก $O(1/\alpha)$ ครั้งจนกว่าขนาดจะเป็น 1 หรือน้อยกว่า งานในแต่ละการเรียกเวียนเกิดคือ $O(k) = O(n_0^\alpha)$ ดังนั้นงานรวมคือ $W(n) = O((1/\alpha)n^\alpha)$ บน MP-RAM สแปนในแต่ละระดับคือ $O(1)$ ดังนั้นสแปนโดยรวมคือ $O(1/\alpha)$ ตัวอย่างเช่น หากเราเลือก $\alpha = 1/2$ เราจะได้อัลกอริทึมที่มีสแปนคงที่ด้วยงาน $O(n^{1/2})$ หากเราเลือก $\alpha = 1/\log(n)$ เราจะได้ $W(n) = D(n) = O(\log n)$[แก้ 10] และกลับไปเป็นการค้นหาแบบทวิภาคมาตรฐาน อัลกอริทึมแบบแตกกิ่งหลายทางต้องใช้การ fork แบบ k ทาง และไม่ดีไปกว่าการค้นหาแบบทวิภาคสำหรับแบบจำลอง fork ทวิภาค

อีกปัญหาที่เกี่ยวข้องคือ กำหนดลำดับที่เรียงแล้วสองลำดับ A และ B และจำนวนเต็ม k ให้หาสมาชิกที่เล็กที่สุด k ตัว กล่าวให้เจาะจงขึ้น KTH(A, B, k) คืนตำแหน่ง (l_a, l_b) ใน A และ B ซึ่ง $l_a + l_b = k$ และสมาชิกทั้งหมดใน $A[0 : l_a] \cup B[0 : l_b]$ น้อยกว่าสมาชิกทั้งหมดใน $A[l_a : |A|] \cup B[l_b : |B|]$ ปัญหานี้แก้ได้ด้วยการค้นหาแบบทวิภาคคู่ (dual binary search) ดังแสดงในรูปที่ 6 แต่ละการเรียกเวียนเกิดลดขนาดของ A หรือขนาดของ B ลงครึ่งหนึ่ง และดังนั้นจึงทำงานด้วยงานและสแปน $O(\log |A| + \log |B|)$

การค้นหาแบบทวิภาคคู่ในรูปที่ 6 ไม่เป็นแบบขนาน แต่เช่นเดียวกับปัญหาการค้นหาในลำดับที่เรียงแล้ว เป็นไปได้ที่จะแลกงา

$$
\begin{array}{l}
\textsc{kthHelp}(A, a_o, B, b_o, k) \;=\\
\quad \textbf{if } (|A| + |B| = 0) \textbf{ then return } (a_o, b_o);\\
\quad \textbf{else if } (|A| = 0) \textbf{ then return } (a_o, b_o + k);\\
\quad \textbf{else if } (|B| = 0) \textbf{ then return } (a_o + k, b_o);\\
\quad \textbf{else}\\
\qquad m_a \leftarrow |A|/2; m_b \leftarrow |B|/2;\\
\qquad \textbf{case } (A[m_a] < B[m_b], k > m_a + m_b) \textbf{ of}\\
\qquad\quad (T,T) \Rightarrow \textbf{return } \textsc{kthHelp}(A[m_a + 1 : |A|], a_o + m_a + 1, B, b_o, k - m_a - 1)\\
\qquad\quad (T,F) \Rightarrow \textbf{return } \textsc{kthHelp}(A, a_o, B[0 : m_b], b_o, k)\\
\qquad\quad (F,T) \Rightarrow \textbf{return } \textsc{kthHelp}(A, a_o, B[m_b + 1 : |B|], b_o + m_b + 1, k - m_b - 1)\\
\qquad\quad (F,F) \Rightarrow \textbf{return } \textsc{kthHelp}(A[0 : m_a], a_o, B, b_o, k)\\
\\
\textsc{kth}(A, B, k) \;=\; \textbf{return } \textsc{kthHelp}(A, 0, B, 0, k)
\end{array}
$$

รูปที่ 6: ฟังก์ชัน KTH

นกับสแปน แนวคิดอีกครั้งคือการค้นหาแบบ k ทาง โดยการเลือกตำแหน่ง k ตำแหน่งที่ห่างเท่า ๆ กันในอาร์เรย์หนึ่ง เป็นไปได้ที่จะหาตำแหน่งเหล่านั้นในอีกอาร์เรย์หนึ่งโดยใช้ปัญหาการค้นหาในลำดับที่เรียงแล้ว ซึ่งใช้หาบล็อกย่อยของ A และ B ที่มีตำแหน่ง (l_a, l_b) ได้ โดยทำแบบนี้อีกครั้งจากอีกอาร์เรย์หนึ่ง บล็อกย่อยทั้งสองจะลดขนาดลงด้วยตัวประกอบ k ทำซ้ำเช่นนี้เป็นจำนวน $\log_k |A| + \log_k |B|$ ระดับ โดยการเลือก $k = n_0^\alpha$ จะได้อัลกอริทึมที่ใช้งาน $O(n^{2\alpha})$ และสแปน $O(1/\alpha^2)$ เช่นเดียวกับปัญหาการค้นหาในอาร์เรย์ที่เรียงแล้วแบบสแปนคงที่ วิธีนี้ใช้ไม่ได้กับแบบจำลอง fork ทวิภาค

4.4 การผสาน (Merge)

ปัญหาการผสาน (merging) คือการรับลำดับที่เรียงแล้วสองลำดับ A และ B แล้วสร้างผลลัพธ์เป็นลำดับ R ที่มีสมาชิกทั้งหมดของ A และ B ในลำดับที่เรียงแล้วและเสถียร (stable) ในแบบลำดับ ปกติจะทำโดยเริ่มที่ต้นของแต่ละลำดับแล้วดึงสมาชิกที่น้อยกว่าจากสองลำดับที่เหลือออกมาซ้ำ ๆ อัลกอริทึมนี้ไม่เป็นแบบขนาน แต่ในที่นี้เราอธิบายอัลกอริทึมแบบขนานหลายแบบสำหรับปัญหานี้ ซึ่งเป็นอัลกอริทึมแบบลำดับที่ดีด้วย (คือมีประสิทธิภาพเชิงงาน)

โดยใช้ฟังก์ชัน KTH การผสานสามารถทำได้ด้วยการแบ่งแยกและเอาชนะ ดังแสดงในรูปที่ 7 การเรียก KTH แบ่งขนาดผลลัพธ์ออกครึ่งหนึ่ง (คลาดเคลื่อนไม่เกินหนึ่ง) จากนั้น merge เรียกเวียนเกิดบนส่วนล่างของ A และ B และบนส่วนบนแบบขนาน การเขียนค่าลงในผลลัพธ์ R เกิดขึ้นในกรณีฐานของการเวียนเกิด ดังนั้น merge จึงไม่คืนค่าอะไร ให้ $m = |A| + |B|$ และใช้การค้นหาแบบทวิภาคคู่สำหรับ KTH ความสัมพันธ์เวียนเกิดของต้นทุนสำหรับ merge คือ:

$$
\begin{aligned}
W(m) &= 2W(m/2) + O(\log m)\\
D(m) &= D(m/2) + O(\log m)
\end{aligned}
$$

ซึ่งมีผลเฉลยเป็น $W(m) = O(m)$ และ $D(m) = O(\log^2 m)$ วิธีนี้ทำงานในแบบจำลอง fork ทวิภาคได้ โดยการใช้ KTH รุ่นขนานด้วย $\alpha = 1/4$ ความสัมพันธ์เวียนเกิดคือ:

$$
\begin{aligned}
W(m) &= 2W(m/2) + O(m^{1/2}) \quad \text{[แก้ 11]}\\
D(m) &= D(m/2) + O(1)
\end{aligned}
$$

ซึ่งมีผลเฉลยเป็น $W(m) = O(m)$ และ $D(m) = O(\log m)$ วิธีนี้ใช้ไม่ได้ในแบบจำลอง fork ทวิภาค

สแปนของการผสานแบบขนานสามารถปรับปรุงได้โดยใช้การแบ่งแยกและเอาชนะแบบหลายทางแทนแบบสองทาง ดังแสดงในรูปที่ 8 โค้ดเรียกเวียนเกิด $f(n)$ ครั้ง แต่ละครั้งรับผิดชอบส่วนของผลลัพธ์ขนาด l หากเราใช้ $f(n) = \sqrt{n}$ และใช้การค้นหาแบบทวิภาคคู่สำหรับ KTH ความสัมพันธ์เวียนเกิดของต้นทุนคือ:

```
MERGE(A, B, R) =
  case (|A|, |B|) of
    (0, _) ⇒ copy B to R; return;
    (_, 0) ⇒ copy A to R; return;
    otherwise ⇒
      m ← |R|/2;
      (m_a, m_b) = KTH(A, B, m);
      MERGE(A[0 : m_a], B[0 : m_b], R[0 : m]) ||
      MERGE(A[m_a : |A|], B[m_b : |B|], R[m : |R|]);
      return
```

รูปที่ 7: การผสานแบบแบ่งแยกและเอาชนะ 2 ทาง (MERGE)

```
MERGEFWAY(A, B, R, f) =
  // Same base cases
  otherwise ⇒
    l ← (|R| − 1)/f(|R|) + 1;
    parfor i in [0 : f(|R|)]
      s ← min(i × l, |R|);
      e ← min((i + 1) × l, |R|);
      (s_a, s_b) ← KTH(A, B, s);
      (e_a, e_b) ← KTH(A, B, e);
      MERGEFWAY(A[s_a : e_a], B[s_b : e_b], R[s : e], f)
    return
```

รูปที่ 8: การผสานแบบแบ่งแยกและเอาชนะ $f(n)$ ทาง

$$W(m) = \sqrt{m}\,W(\sqrt{m}) + O(\sqrt{m}\log m)$$
$$D(m) = D(\sqrt{m}) + O(\log m)$$

ซึ่งมีผลเฉลยเป็น $W(m) = O(m)$ และ $D(m) = O(\log m)$[แก้ 11] รุ่นนี้ทำงานในแบบจำลอง fork ทวิภาคได้ เพราะ parFor สามารถทำด้วยการ fork แบบทวิภาคภายในสแปนที่ต้องการ โดยการใช้ $f(n) = \sqrt{n}$ และ KTH รุ่นขนานด้วย $\alpha = 1/8$ ความสัมพันธ์เวียนเกิดของต้นทุนคือ:

$$W(m) = \sqrt{m}\,W(\sqrt{m}) + O(m^{3/4})$$
$$D(m) = D(\sqrt{m}) + O(1)$$

ซึ่งมีผลเฉลยเป็น $W(m) = O(m)$ และ $D(m) = O(\log\log m)$[แก้ 11]

Claude อธิบายเพิ่ม: สูตรช่วยแก้ความสัมพันธ์เวียนเกิดที่ใช้ในบทนี้

$W(n) = 2W(n/2) + O(1)$	$\Rightarrow O(n)$
$W(n) = 2W(n/2) + O(\log n)$	$\Rightarrow O(n)$ (ต้นทุนส่วนใหญ่อยู่ที่ระดับใบ)
$W(n) = 2W(n/2) + O(n)$	$\Rightarrow O(n \log n)$
$D(n) = D(n/2) + O(1)$	$\Rightarrow O(\log n)$
$D(n) = D(n/2) + O(\log n)$	$\Rightarrow O(\log^2 n)$
$D(n) = D(\sqrt{n}) + O(1)$	$\Rightarrow O(\log\log n)$ (ถอดราก $\log\log n$ ครั้ง n ก็เหลือค่าคงที่)
$D(n) = D(\sqrt{n}) + O(\log n)$	$\Rightarrow O(\log n)$ (อนุกรมเรขาคณิต $\log n + \frac{1}{2}\log n + \cdots$)

แนวคิดหลักของการผสานแบบขนาน: แทนที่จะเดินทีละตัว เราใช้ KTH หา "จุดตัด" ที่แบ่งผลลัพธ์เป็นชิ้นขนาดเท่ากันได้ทันที แล้วผสานแต่ละชิ้นอย่างอิสระ เทคนิค "หาจุดแบ่งก่อน แล้วทำแต่ละชิ้นขนานกัน" นี้กลับมาอีกใน sample sort (5.3) และ *split* ของต้นไม้ (7.3)

ขอบเขต 4.1. *การผสานสามารถแก้ได้ด้วยงาน $O(n)$ และสแปน $O(\log n)$ ในแบบจำลอง fork ทวิภาค และด้วยงาน $O(n)$ และสแปน $O(\log\log n)$ บน MP-RAM*

เราสังเกตว่าการใช้ $f(n) = n/\log(n)$ และใช้การผสานแบบลำดับในการเรียกเวียนเกิด ให้อีกรูปแบบหนึ่งซึ่งทำงานด้วยงาน $O(n)$ และสแปน $O(\log n)$ บนแบบจำลอง fork ทวิภาค เมื่อใช้กับค่าคงที่ขนาดเล็ก เช่น $f(n) = .1 \times n/\log n$ รุ่นนี้ทำงานได้ดีในทางปฏิบัติ

4.5 สมาชิกที่เล็กที่สุดลำดับที่ k (K-th Smallest)

ปัญหาสมาชิกที่เล็กที่สุดลำดับที่ k คือการหาสมาชิกที่เล็กเป็นลำดับที่ k ในลำดับ รูปที่ 9 ให้อัลกอริทึมสำหรับปัญหานี้ ประสิทธิภาพขึ้นอยู่กับวิธีเลือกตัวแบ่ง (pivot) หากเลือกอย่างสุ่มสม่ำเสมอ (uniformly at random) จากสมาชิกของ A อัลกอริทึมจะเรียกเวียนเกิด $O(\log|A| + \log(1/\epsilon))$ ครั้งด้วยความน่าจะเป็น $1-\epsilon$ วิธีหนึ่งในการวิเคราะห์คือสังเกตว่าด้วยความน่าจะเป็น $1/2$ ตัวแบ่งจะถูกเลือกจากครึ่งกลาง (ระหว่าง $1/4$ และ $3/4$) และในกรณีนั้นขนาดของอาร์เรย์ที่ส่งให้การเรียกเวียนเกิดจะไม่เกิน $3/4|A|$ เราเรียกการเรียกเช่นนี้ว่าดี (good) หลังจากการเรียกที่ดีไม่เกิน $\log_{4/3}|A|$ ครั้ง ขนาดจะเป็น 1 และอัลกอริทึมจะเสร็จสิ้น การวิเคราะห์จำนวนการเรียกเวียนเกิดเหมือนกับการถามว่าต้องโยนเหรียญที่ไม่ลำเอียงและเป็นอิสระต่อกันกี่ครั้งจึงจะได้หัว $\log_{4/3}|A|$ ครั้ง ซึ่งถูกจำกัดดังที่กล่าวข้างต้น

```
kthSmallest(A, k) =
  p ← selectPivot A;
  L ← filter(A, λx.(x < p));
  G ← filter(A, λx.(x > p));
  if (k < |L|) then
    return kthSmallest(L, k)
  else if (k > |A| − |G|) then
    return kthSmallest(G, k − (|A| − |G|))
  else return p
```

รูปที่ 9: kthSmallest *ในรูป ที่ถูกคือ:* $p \leftarrow$ selectPivot(A) *และเงื่อนไข* $k \geq |A| - |G|$ *แทน* $k > |A| - |G|$[แก้ 12]

```
selectPivotR(A) = A[rand(n)];

selectPivotD(A, l) =
  l ← f(|A|);
  m ← (|A| − 1)/l + 1;
  B ← array[m];
  parfor i in [0 : m]
    s ← i × l;
    B[i] ← kthSmallest(A[s : s + l], l/2);
  return kthSmallest(B, m/2);
```

รูปที่ 10: การเลือกตัวแบ่งแบบสุ่มและแบบเชิงกำหนด *ในรูป* rand(n) *ที่ถูกคือ* rand$(|A|)$[แก้ 12]

โดยทั่วไปเรากล่าวว่าอัลกอริทึมมีสมบัติบางอย่าง*ด้วยความน่าจะเป็นสูง* (w.h.p.) หากสำหรับข้อมูลเข้าขนาด n และค่าคงที่ k ใด ๆ ความน่าจะเป็นมีค่าอย่างน้อย $1 - 1/n^k$ ดังนั้น kthSmallest รุ่นสุ่มจึงเรียกเวียนเกิด $O(\log|A|)$ ครั้ง w.h.p. (โดยเลือก $\epsilon = 1/|A|^k$) เนื่องจาก filter มีสแปน $O(\log n)$ สำหรับอาร์เรย์ขนาด n สแปนโดยรวมจึงเป็น $O(\log^2|A|)$[แก้ 13] w.h.p. งานเป็น $O(|A|)$ ในค่าคาดหมาย อัลกอริทึมนี้ทำงานบนแบบจำลอง fork ทวิภาคได้

เป็นไปได้เช่นกันที่จะสร้าง kthSmallest รุ่นเชิงกำหนด (deterministic) โดยเลือกตัวแบ่งอย่างระมัดระวังมากขึ้น โดยเฉพาะเราสามารถใช้วิธีมัธยฐานของมัธยฐาน (median of median) ที่แสดงในรูปที่ 10 มันแบ่งอาร์เรย์ออกเป็นบล็อกขนาด $f(|A|)$ หามัธยฐานของแต่ละบล็อก แล้วหามัธยฐานของผลลัพธ์ มัธยฐานที่ได้จะต้องอยู่ในครึ่งกลางของค่าของ A การกำหนด $f(n) = 5$ จะได้รุ่นขนานของอัลกอริทึมแบบลำดับเชิงกำหนดมาตรฐานสำหรับ kthSmallest เนื่องจากบล็อกมีขนาดคงที่ เราไม่ต้องเรียกเวียนเกิดสำหรับแต่ละบล็อก แต่สามารถคำนวณมัธยฐานของห้าตัวแต่ละชุดด้วยการเรียงลำดับแทน ในกรณีนี้การเรียกเวียนเกิดจะไม่ใหญ่กว่า $7/10|A|$ ด้วย รุ่นขนานจึงสอดคล้องกับความสัมพันธ์เวียนเกิดของต้นทุน:

$$W(n) = W(7/10n) + W(1/5n) + O(n)$$
$$D(n) = D(7/10n) + D(1/5n) + O(1) \quad \text{[แก้ 14]}$$

ซึ่งมีผลเฉลยเป็น $W(n) = O(n)$ และ $D(n) = O(n^\alpha)$ เมื่อ $\alpha \approx .84$ สอดคล้องกับสมการ $\left(\frac{7}{10}\right)^\alpha + \left(\frac{1}{5}\right)^\alpha = 1$

สแปนสามารถปรับปรุงได้โดยกำหนด $f(n) = \log n$ ใช้มัธยฐานแบบลำดับสำหรับแต่ละบล็อก และใช้การเรียงลำดับเพื่อหามัธยฐานของมัธยฐาน สมมติว่าการเรียงลำดับใช้งาน $O(n\log n)$ และมีสแปน $D_{sort}(n)$ จะได้ความสัมพันธ์เวียนเกิด:

$$W(n) = W(3/4n) + O((n/\log n)\log(n/\log n)) + O(n)$$
$$D(n) = D(3/4n) + O(\log n) + D_{sort}(n) \quad \text{[แก้ 14]}$$

ซึ่งมีผลเฉลยเป็น $W(n) = O(n)$ และ $D(n) = O(D_{sort}(n) \log n)$ การหยุดการเวียนเกิดของ KTHSMALLEST เมื่อข้อมูลเข้ามีขนาด $n/\log n$ (หลังจากการเรียกเวียนเกิด $O(\log \log n)$ ครั้ง) และใช้การเรียงลำดับกับสมาชิกที่เหลือ จะปรับปรุงสแปนเป็น $D(n) = O(D_{sort}(n) \log \log n)$

	MP-RAM		Binary-Forking	
ปัญหา	งาน (Work)	สแปน (Span)	งาน (Work)	สแปน (Span)
SCAN	$O(n)$	$O(\log n)$	—	—
FILTER	$O(n)$	$O(\log n)$	—	—
FLATTEN	$O(m+n)$	$O(\log n)$	—	$O(\log m)$
SEARCH	$O(n^\alpha/\alpha)$	$O(1/\alpha)$	$O(\log n)$	$O(\log n)$
MERGE	$O(n)$	$O(\log \log n)$	—	$O(\log n)$
KTHSMALLEST	$O(n)$	$O(D_s(n) \log \log n)$	—	$O(D_s(n) \log \log n)$

ตารางที่ 1: สรุปต้นทุน เครื่องหมาย — หมายความว่าต้นทุนเท่ากับบน MP-RAM สำหรับ FLATTEN m คือขนาดของผลลัพธ์ สำหรับ SEARCH $0 < \alpha \leq 1$ สำหรับ KTHSMALLEST $D_s(n)$ คือสแปนของการเรียงลำดับลำดับความยาว n บนแบบจำลองที่กำหนด

4.6 สรุป

ตารางที่ 1 สรุปต้นทุนของฟังก์ชันพื้นฐานที่เราได้กล่าวถึงมาจนถึงตอนนี้

Claude อธิบายเพิ่ม: สรุปบทที่ 4 และการเชื่อมโยง

- **scan** คือ primitive ที่สำคัญที่สุดของบท: FILTER และ FLATTEN สร้างจาก scan โดยตรง
- **การแลกงานกับสแปน:** SEARCH แบบ k ทาง ลดสแปนเหลือ $O(1/\alpha)$ แต่งานเพิ่มเป็น $O(n^\alpha/\alpha)$ ซึ่งใน MP-RAM ทำได้เพราะ fork ได้ k ทางในขั้นเดียว แต่ในแบบจำลอง fork ทวิภาค การ fork k ทางเองก็ใช้สแปน $\log k$ แล้ว ข้อได้เปรียบจึงหายไป
- **kthSmallest แบบสุ่ม** ใช้แนวคิดเดียวกับ quicksort แต่เรียกเวียนเกิดฝั่งเดียว จึงได้งาน $O(n)$ แทน $O(n \log n)$

นำไปใช้ต่อที่: mergesort ใช้ MERGE (5.1), quicksort ใช้ FILTER และ FLATTEN (5.2), counting sort ใช้ PLUSSCAN (5.4) และ EDGEMAP ใช้ FLATTEN (6.1)

5 การเรียงลำดับ (Sorting)

มีงานวิจัยจำนวนมากเกี่ยวกับการเรียงลำดับแบบขนานภายใต้แบบจำลองการคำนวณแบบขนานต่าง ๆ ในหัวข้อนี้ เรานำเสนออัลกอริทึมการเรียงลำดับแบบขนานคลาสสิกหลายตัว เช่น mergesort, quicksort, samplesort และ radix-sort เรายังอภิปรายปัญหาที่เกี่ยวข้อง เช่น การเรียงกึ่งลำดับ (semisorting) และการเรียงลำดับจำนวนเต็มแบบขนาน

5.1 Mergesort

Mergesort แบบขนานเป็นอัลกอริทึมแบ่งแยกและเอาชนะแบบขนานคลาสสิก รหัสเทียมสำหรับ mergesort แบบแบ่งแยกและเอาชนะแบบขนานแสดงในรูปที่ 11 อัลกอริทึมรับอาร์เรย์ข้อมูลเข้า A เรียงลำดับ $A[0:m]$ และ $A[m:|A|]$ แบบเวียนเกิด แล้วผสานลำดับที่เรียงแล้วทั้งสองเข้าด้วยกันเป็นลำดับผลลัพธ์ที่เรียงแล้ว R เนื่องจากทั้งขั้นการแบ่งและขั้นการผสานมีความเสถียร ผลลัพธ์จึงเรียงแบบเสถียร เราคำนวณการเรียกเวียนเกิดทั้งสองแบบขนาน และใช้การผสานแบบขนานที่อธิบายในหัวข้อ 4.4 เพื่อผสานผลลัพธ์ของการเรียกเวียนเกิดทั้งสอง ซึ่งให้ความสัมพันธ์เวียนเกิดสำหรับงานและสแปนดังนี้:

$$W(n) = 2W(n/2) + W_{merge}(n)$$
$$D(n) = D(n/2) + D_{merge}(n)\,. \quad \text{[แก้ 15]}$$

```
MERGESORT(A)
  if (|A| = 1) then return A;
  else
    m ← |A|/2
    l ← MERGESORT(A[0 : m]) ||
    r ← MERGESORT(A[m : |A|])
    return MERGE(l, r)
```

รูปที่ 11: Mergesort แบบขนาน

```
QUICKSORT(S) =
  if (|S| = 0) then return S;
  else
    p ← SELECTPIVOT(S);
    e ← FILTER(S, λx.(x = p));
    l ← QUICKSORT(FILTER(S, λx.(x < p))) ||
    r ← QUICKSORT(FILTER(S, λx.(x > p)))
    return FLATTEN(⟨l, e, r⟩)
```

รูปที่ 12: Quicksort แบบขนาน

จากผลลัพธ์ของการผสานในหัวข้อ 4.4 ความสัมพันธ์เหล่านี้มีผลเฉลยเป็นงาน $O(n \log n)$ และสแปน $O(\log n \log\log n)$ บน MP-RAM และได้งานที่เหมาะสมที่สุด (optimal) เท่ากันแต่มีสแปน $O(\log^2 n)$ บนแบบจำลอง fork ทวิภาค

5.2 Quicksort

รหัสเทียมสำหรับ quicksort แบบแบ่งแยกและเอาชนะแบบขนานแสดงในรูปที่ 12 เป็นที่ทราบกันดีว่าสำหรับการเลือกตัวแบ่งแบบสุ่ม เวลาคาดหมายของ quicksort เชิงสุ่มคือ $O(n \log n)$ เนื่องจาก quicksort รุ่นขนานทำการเรียกเหมือนกันทุกประการ งานรวมของอัลกอริทึมนี้จึงเป็น $O(n \log n)$ ในค่าคาดหมายเช่นกัน ในที่นี้เราวิเคราะห์งานและสแปนของอัลกอริทึมโดยย่อ เราสมมติว่าตัวแบ่งถูกเลือกแบบสุ่มด้วยความน่าจะเป็นเท่ากัน การวิเคราะห์งานเหมือนกับการวิเคราะห์อัลกอริทึมแบบลำดับ ในการวิเคราะห์สแปน เราสังเกตว่าสแปนของแต่ละการเรียก quicksort (ไม่รวมการเรียกเวียนเกิด) คือ $O(\log n)$ ดังนั้นหากเราจำกัดความลึกของการเวียนเกิดได้ เราก็จำกัดสแปนได้

สำหรับลำดับข้อมูลเข้าความยาว n ให้ A_{ij} เป็นตัวแปรสุ่มที่บ่งบอกว่า เมื่อสมาชิกที่มีอันดับ (rank) i ถูกเลือกเป็นตัวแบ่ง สมาชิกที่มีอันดับ j อยู่ในลำดับ S สำหรับการเรียก quicksort ครั้งนั้น สังเกตว่าตามที่โค้ดเขียนไว้ สมาชิกทุกตัวจะถูกเลือกเป็นตัวแบ่งในที่สุด ลองพิจารณาความน่าจะเป็นของเหตุการณ์ คือ $p(A_{ij} = 1)$ เหตุการณ์นี้เกิดขึ้นพอดีเมื่อสมาชิกอันดับ i ถูกเลือกเป็นตัวแรกในบรรดาสมาชิกตั้งแต่อันดับ i ถึงอันดับ j (รวมทั้งสอง) หากสมาชิกอื่นใดระหว่างสมาชิกอันดับ i และ j ถูกเลือกก่อน มันจะแยกทั้งสองออกจากกัน หากสมาชิกอันดับ j ถูกเลือกก่อน เห็นได้ชัดว่ามันจะไม่อยู่ใน S เมื่อ i ถูกเลือก หากสมาชิก i ถูกเลือก

ก่อน j จะอยู่ใน S เพราะไม่มีสมาชิกอื่นแยกทั้งสองออกจากกัน มีสมาชิก $1 + |j - i|$ ตัวตั้งแต่ i ถึง j รวมทั้ง i และ j ดังนั้นความน่าจะเป็นของเหตุการณ์คือ $1/(1 + |j - i|)$

ก่อนอื่นให้ B เป็นตัวแปรสุ่มที่ระบุว่าการเรียงลำดับทำการเปรียบเทียบกี่ครั้ง เรามี:

$$B = \sum_{i=1}^{n} \sum_{j=1, j\neq i}^{n} A_{ij} \quad \text{[แก้ 16]}$$

โดยความเป็นเชิงเส้นของค่าคาดหมาย เราจะได้

$$\begin{aligned}
E[B] &= \sum_{i=1}^{n} \sum_{j=1, j\neq i}^{n} E[A_{ij}] \\
&= \sum_{i=1}^{n} \left(\sum_{k=1}^{i-1} \frac{1}{k+1} + \sum_{k=1}^{n-i} \frac{1}{k+1} \right) \quad \text{[แก้ 16]} \\
&= \sum_{i=1}^{n} \big((H_i - 1) + (H_{n-i+1} - 1) \big) \\
&\leq \sum_{i=1}^{n} \big(\ln i + \ln(n - i + 1) \big) \\
&\leq \sum_{i=1}^{n} 2 \ln n \\
&= 2n \ln n
\end{aligned}$$

ในการวิเคราะห์สแปน เราต้องหาความลึกของการเวียนเกิด ความลึกของสมาชิกอันดับ i จะเท่ากับจำนวนตัวแบ่งที่ถูกเลือกเหนือมัน ให้ D_i เป็นจำนวนตัวแบ่งดังกล่าว เราจะได้ว่า $D_i = \sum_{j=1, j\neq i}^{n} A_{ji}$ [แก้ 17] และ

$$E[D_i] = \sum_{j=1, j\neq i}^{n} \frac{1}{1 + |i - j|} \leq 2 \ln n$$

สิ่งนี้บอกความลึกคาดหมายของสมาชิกแต่ละตัว แต่เราสนใจความลึกสูงสุดบนสมาชิกทั้งหมด น่าเสียดายที่ไม่มีสิ่งที่เทียบเท่ากับความเป็นเชิงเส้นของค่าคาดหมายสำหรับค่าสูงสุด เราจะใช้ขอบเขตหางแทน เพื่อจำกัดความน่าจะเป็นที่สมาชิกใดจะมีความลึกสูงกว่า $O(\log n)$ โดยเฉพาะเราจะใช้ขอบเขตเชอร์นอฟต่อไปนี้

ทฤษฎีบท 5.1 (ขอบเขตเชอร์นอฟ (Chernoff Bounds)). *ให้ $X_1, \ldots, X_n$ เป็นตัวแปรสุ่มชี้บอกที่เป็นอิสระต่อกัน แล้ว:*

$$P[X \geq c] \leq \left(\frac{eE[X]}{c} \right)^{c}$$

ข้อสำคัญคือสังเกตว่า A_{ij} ของเราเป็นอิสระต่อกันข้าม i ที่ต่างกันสำหรับ j ที่คงที่ใด ๆ เพราะ A_{ij} บอกเราหลายอย่างเกี่ยวกับ i (มันถูกเลือกเป็นตัวแรกในช่วง) แต่ไม่บอกอะไรเกี่ยวกับ j นอกจากว่ามันถูกเลือกหลัง i โดยเฉพาะ หลังจาก i ถูกเลือกสมาชิกใด ๆ ที่มีอันดับ $i + 1$ ถึง j ยังคงมีโอกาสเท่ากันที่จะถูกเลือกเป็นตัวถัดไปในหมู่พวกมัน ดังนั้นเราจึงใช้ขอบเขตเชอร์นอฟได้โดยกำหนด $c = 2a \ln n$ สำหรับค่าคงที่ a บางค่า จะได้

$$P[D_i \geq 2a \ln n] \leq \left(\frac{eE[X]}{2a \ln n} \right)^{2a \ln n}$$

$$
\begin{aligned}
&\leq \left(\frac{e}{a}\right)^{2a\ln n} \\
&= n^{2a\ln(e/a)} \\
&= n^{-2a\ln(a/e)}
\end{aligned}
$$

ตัวอย่างเช่น สำหรับ $a = e^2$ เราจะได้ความน่าจะเป็นน้อยกว่า n^{-2e^2} ซึ่งน้อยกว่า n^{-14} แม้ว่าเราจะใช้ขอบเขตยูเนียนบนสมาชิกทั้ง n ตัว เราได้ว่าความน่าจะเป็นที่สมาชิกใดลึกกว่า $2e^2 \ln n$ มีค่าไม่เกิน n^{-13}

ดังนั้นเราจึงกล่าวว่า quicksort เชิงสุ่มมีความลึกของการเวียนเกิด $O(\log n)$ ด้วยความน่าจะเป็นสูง และดังนั้นจึงมีสแปน $O(\log^2 n)$ ด้วยความน่าจะเป็นสูง

5.3 Sample Sort

อยู่ระหว่างการเขียน

ในทางปฏิบัติ quicksort มีความแปรปรวนสูงในเวลาทำงาน หากการเลือกตัวแบ่งทำให้การเรียกย่อยมีปริมาณงานที่เบ้ (skewed) อย่างมาก เวลาทำงานโดยรวมของอัลกอริทึมอาจแย่ลงเนื่องจากความไม่สมดุลของงาน อัลกอริทึมเชิงปฏิบัติที่เรียกว่า *samplesort* จัดการกับความเบ้โดยสุ่มเลือกตัวแบ่งจำนวนมาก เรียกว่า *splitters* (การใช้ splitter $c \cdot p$ หรือ $\sqrt{n}$ ตัวเป็นทางเลือกที่พบบ่อย) และแบ่งลำดับข้อมูลเข้าออกเป็นถัง (bucket) ตาม splitter สมมติว่าเราเลือก splitter มากกว่าจำนวนตัวประมวลผล เราน่าจะกำหนดงานในปริมาณใกล้เคียงกันให้แต่ละตัวประมวลผล ขั้นตอนย่อยสำคัญอย่างหนึ่งของ samplesort คือการสับ (shuffle) สมาชิกในลำดับย่อยข้อมูลเข้าลงในถัง จะใช้ samplesort หรือ radix-sort ที่เราอธิบายในหัวข้อถัดไปทำขั้นตอนนี้ก็ได้ และทำได้อย่างมีประสิทธิภาพเชิงงาน (คือด้วยงาน $O(n)$)

5.4 Counting Sort และ Radix Sort

Counting sort และ radix sort เป็นการเรียงลำดับบนพื้นฐานจำนวนเต็มที่ไม่ได้อาศัยการเปรียบเทียบ และดังนั้นจึงไม่ถูกจำกัดด้วยขอบเขตล่าง $\Omega(n \log n)$[แก้ 18] ที่การเรียงลำดับแบบเปรียบเทียบมี อันที่จริง ในแบบลำดับ สมมติว่าจำนวนเต็มถูกจำกัดด้วยพหุนามในจำนวนคีย์ n radix sort ที่อาศัย counting sort ใช้เวลา $O(n)$

สำหรับคีย์ในช่วงคงที่ $[0 : m]$ counting sort ทำงานในสามขั้นตอน: ขั้นแรกนับจำนวนคีย์ที่มีแต่ละค่าที่เป็นไปได้ จากนั้นคำนวณออฟเซ็ตที่แต่ละคีย์ปรากฏในผลลัพธ์ และสุดท้ายผ่านคีย์ครั้งที่สองเพื่อวางไว้ในตำแหน่งที่ถูกต้อง ขั้นตอนเหล่านี้ตรงกับบรรทัดที่ 13, 14 และ 16 ในรูปที่ 13 สำหรับคีย์จำนวนเต็ม n ตัวในช่วง $[0 : m]$ counting sort ต้องใช้งาน (เวลา) $O(n + m)$ จึงไม่มีประสิทธิภาพเมื่อ $m \gg n$ สมบัติสำคัญของ counting sort คือมีความเสถียร: คีย์ที่เท่ากันคงลำดับเดิมไว้

เมื่อ $m > n$ เราสามารถใช้ radix sort ซึ่งเพียงเรียก counting sort ซ้ำ ๆ สำหรับฐาน (radix) ขนาด n[แก้ 19] รอบที่ i เรียงลำดับโดยอาศัยการดึงคีย์ประจำรอบ (round key) $(k/n^i) \bmod n$ จากแต่ละคีย์ k เนื่องจากคีย์ประจำรอบถูกจำกัดด้วย n เราจึงใช้ counting sort สำหรับรอบนั้นในเวลาเชิงเส้นได้ เนื่องจาก counting sort มีความเสถียร แต่ละรอบจึงรักษาลำดับเทียบกับรอบก่อนหน้าไว้ หากจำนวนเต็มข้อมูลเข้าถูกจำกัดด้วย n^c radix sort จะใช้ c รอบ และดังนั้นใช้เวลา $O(cn)$ สมมติว่า c เป็นค่าคงที่ จะได้เวลา $O(n)$

ที่น่าสนใจคือ ยังเป็นปัญหาเปิด (open problem) ว่าเป็นไปได้หรือไม่ที่จะเรียงลำดับจำนวนเต็มในช่วง $[0 : n^c], c > 1$ ด้วยงาน $O(n)$ และสแปนพหุลอการิทึม สิ่งที่เราจะแสดงต่อไปนี้คือ การทำ radix sort แบบขนานด้วยงาน $O\left(\frac{n}{\alpha}\right)$ และสแปน $O\left(\frac{n^\alpha}{\alpha}\right)$ สำหรับ α ใด ๆ ที่ $0 < \alpha \leq 1$ ทำได้ค่อนข้างง่าย สแปนนี้ไม่เป็นพหุลอการิทึม

เราเริ่มด้วยการพิจารณา counting sort แบบขนานที่เรียงลำดับจำนวนเต็ม n ตัวในช่วง $[0 : m]$ การเรียงลำดับนี้คือ ParallelCountSort ในรูปที่ 13 มันคล้ายกับรุ่นแบบลำดับ แต่ทำงานโดยแบ่งข้อมูลเข้าออกเป็น $b = n/m$ บล็อก บล็อกละ m ตัว[แก้ 20] แต่ละบล็อกนับคีย์ของตัวเอง จากนั้นเราต้องคำนวณออฟเซ็ตในผลลัพธ์สำหรับแต่ละบล็อกและแต่ละคีย์ว่าผลลัพธ์ควรวางไว้ที่ใด เช่นเดียวกับกรณีแบบลำดับ ขั้นนี้ใช้ plusScan แต่ทำข้ามบล็อกและถังทั้งหมด สุดท้ายเมื่อได้ออฟเซ็ตแล้ว เราทำงานข้ามบล็อกแบบขนานอีกครั้งเพื่อวางแต่ละค่าในตำแหน่งที่ถูกต้อง

ในการวิเคราะห์ต้นทุน เราสังเกตว่าฟังก์ชัน COUNTS และ PLACE เป็นแบบลำดับอย่างสมบูรณ์ เมื่อเรียกด้วยคีย์ m ตัว ทั้งสองใช้งานและสแปน $O(m)$ ข้ามการเรียกแบบขนาน $|K|/m$ ครั้ง รวมเป็นงาน $O(|K|)$ และสแปน $O(m)$ ต้นทุนของ PLUSSCAN การ flatten และการสลับเปลี่ยน (transpose) คืองาน $O(|K|)$ และสแปน $O(\log |K|)$ รวมทั้งหมดเป็นงาน $O(|K|)$ และสแปน $O(m + \log |K|)$ สำหรับจำนวนเต็มในช่วง $[0 : \log |K|]$ ผลลัพธ์นี้ดีมาก เพราะได้งานเชิงเส้นและสแปนลอการิทึม อย่างไรก็ตาม สำหรับช่วง $[0 : n]$ มันเป็นแบบลำดับอย่างสมบูรณ์ด้วยงานและสแปน $O(n)$

```
1  COUNTS(K, m)
2    for j ∈ [0 : m] : C_j ← 0
3    for j ∈ [0 : m]
4      k ← K_j
5      C_k ← C_k + 1
6    return C

7  PLACE(R, O, K, V, m)
8    for j ∈ [0 : m]
9      k ← K_j
10     R[O_k] ← V_j
11     O_k ← O_k + 1

12 SEQUENTIALCOUNTSORT(K, V, m)
13   C ← COUNTS(K, m)
14   O ← PLUSSCAN(C)
15   R ← an array of size |K|
16   PLACE(R, O, K, V, m)
17   return R

18 PARALLELCOUNTSORT(K, V, m)
19   b ← |K|/m
20   parfor i ∈ [0 : b]
21     C_i = COUNTS(K[mi : m(i + 1)], m)
22   O ← PLUSSCAN(FLATTEN(TRANSPOSE(C)))
23   O' ← TRANSPOSE(partition O into blocks of size b)
24   R ← an array of size |K|
25   parfor i ∈ [0 : b]
26     PLACE(R, O'_i, K[mi : m(i + 1)], V[mi : m(i + 1)], m)
27   return R
```

รูปที่ 13: Counting sort แบบขนาน K คือลำดับของคีย์ในช่วง $[0 : m]$ และ V คือลำดับของค่าที่มีความยาวเท่ากับ K ผลลัพธ์คือค่าจาก V ที่เรียงตามคีย์ที่สอดคล้องกันใน K

ในการใช้ counting sort แบบขนานนี้ใน radix sort บนจำนวนเต็ม n ตัว เราใช้คีย์ย่อย (subkey) ในช่วง $[0 : n^\alpha], 0 < \alpha \leq 1$ ซึ่งนำไปสู่งาน $O(n)$ และสแปน $O(n^\alpha)$ ต่อการเรียก counting sort หนึ่งครั้ง สำหรับจำนวนเต็มในช่วง $[0 : n^c]$ จะมีการเรียก counting sort c/α ครั้ง รวมเป็นงาน $O(n/\alpha)$ และสแปน $O((n^\alpha + \log n)/\alpha) \subset O(n^\alpha/\alpha)$ สมมติว่า c เป็นค่าคงที่

สำหรับจำนวนเต็ม $[0 : n \log^k n]$[แก้ 21] อัลกอริทึมการเรียงลำดับจำนวนเต็มที่มีประสิทธิภาพเชิงงานที่ดีที่สุดที่มีอยู่ สามารถเรียงลำดับจำนวนเต็มแบบไม่เสถียรได้ด้วยงาน $O(kn)$ ในค่าคาดหมาย และสแปน $O(k \log n)$ ด้วยความน่าจะเป็นสูง [21] อย่างไรก็ตาม เนื่องจากไม่เสถียร จึงไม่สามารถใช้ใน radix sort ได้

5.5 Semisort

กำหนดอาร์เรย์ของคีย์และระเบียน (record) ที่เกี่ยวข้อง ปัญหาการเรียงกึ่งลำดับ (semisorting) คือการคำนวณอาร์เรย์ที่จัดลำดับใหม่ ซึ่งระเบียนที่มีคีย์เหมือนกันอยู่ติดกัน ต่างจากผลลัพธ์ของอัลกอริทึมการเรียงลำดับ ระเบียนที่มีคีย์ต่างกันไม่จำเป็นต้องอยู่ในลำดับที่เรียงแล้ว semisort เป็นฟังก์ชันพื้นฐานแบบขนานที่มีประโยชน์อย่างกว้างขวาง และสามารถใช้สร้างขั้นตอน shuffle ใน MapReduce คำนวณการ join เชิงสัมพันธ์ (relational joins) และสร้างอัลกอริทึมกราฟแบบขนานที่เก็บ frontier แบบพลวัตไว้ในถังได้อย่างมีประสิทธิภาพ เป็นต้น

Gu, Shun, Sun และ Blelloch [14] เสนออัลกอริทึมล่าสุดสำหรับ semisort แบบขนานจากบนลงล่าง รูปแบบเฉพาะของ semisort เป็นดังนี้: กำหนดอาร์เรย์ของระเบียน n รายการ แต่ละรายการมีคีย์จากเอกภพ (universe) U และวงศ์ของฟังก์ชันแฮช $h : U \to [1, \ldots, n^k]$ สำหรับค่าคงที่ k บางค่า และฟังก์ชันความเท่ากันบนคีย์ $f : U \times U \to \texttt{bool}$ ให้คืนอาร์เรย์ของระเบียนเดียวกัน โดยระเบียนทั้งหมดที่อยู่ระหว่างระเบียนที่เท่ากันสองรายการเป็นระเบียนที่เท่ากันอื่น ๆ อัลกอริทึมของพวกเขาทำงานด้วยงานและพื้นที่ $O(n)$ ในค่าคาดหมาย และสแปน $O(\log n)$ w.h.p. บน TS-MP-RAM

Claude อธิบายเพิ่ม: สรุปบทที่ 5: เปรียบเทียบอัลกอริทึมการเรียงลำดับ

อัลกอริทึม	งาน	สแปน
Mergesort (MP-RAM)	$O(n \log n)$	$O(\log n \log\log n)$
Mergesort (binary-forking)	$O(n \log n)$	$O(\log^2 n)$
Quicksort แบบสุ่ม	$O(n \log n)$ คาดหมาย	$O(\log^2 n)$ w.h.p.
Radix sort (คีย์ใน $[0 : n^c]$)	$O(n/\alpha)$	$O(n^\alpha/\alpha)$
Semisort	$O(n)$ คาดหมาย	$O(\log n)$ w.h.p.

- **ทำไมความลึกของ quicksort เป็น $O(\log n)$:** สมาชิกหนึ่งตัวถูก "ตัดผ่าน" ด้วยตัวแบ่งตัวที่ j ด้วยความน่าจะเป็น $1/(1 + |i - j|)$ ผลรวมเป็นจำนวนฮาร์มอนิก $\approx 2 \ln n$ แล้วขอบเขตเชอร์นอฟ (บทที่ 3) ทำให้ได้ w.h.p. ส่วนสแปนต้องคูณด้วย $O(\log n)$ อีกครั้ง เพราะ FILTER แต่ละระดับมีสแปน $O(\log n)$
- **ข้อสังเกตเชิงปฏิบัติ:** ไลบรารีขนานจริง (เช่น ParlayLib ของกลุ่มผู้เขียน) นิยมใช้ sample sort เพราะแบ่งงานสม่ำเสมอและใช้แคชได้ดี
- **radix sort ต้องการความเสถียร (stable)** ของการเรียงในแต่ละรอบ ถ้าไม่เสถียร รอบหลังจะทำลายลำดับที่รอบก่อนจัดไว้

6 อัลกอริทึมกราฟ (Graph Algorithms)

ในหัวข้อนี้ เรานำเสนออัลกอริทึมกราฟแบบขนานสำหรับการค้นหาตามแนวกว้าง (breadth-first search) การแยกส่วนแบบเส้นผ่านศูนย์กลางต่ำ (low-diameter decomposition) ความเชื่อมโยง (connectivity) เซตอิสระสูงสุด (maximal independent set) และต้นไม้แผ่ทอดน้อยที่สุด (minimum spanning tree) ซึ่งแสดงให้เห็นเทคนิคที่มีประโยชน์ในอัลกอริทึมแบบขนาน เช่น การสุ่ม (randomization) การกระโดดพอยน์เตอร์ (pointer-jumping) และการหดกราฟ (contraction) หากไม่ได้ระบุไว้เป็นอย่างอื่น กราฟทั้งหมดถือว่าเป็นกราฟมีทิศทางและไม่มีน้ำหนัก เราใช้ $deg^-(u)$ และ $deg^+(u)$ แทนดีกรีเข้าและดีกรีออกของจุดยอด u สำหรับกราฟมีทิศทาง และ $deg(u)$ แทนดีกรีสำหรับกราฟไม่มีทิศทาง

$$
\begin{aligned}
&\textsc{edgeMap}(G, U, f_u) = \\
&\quad \textbf{parfor } i \in [0 : |U|] \\
&\quad\quad N[i] \leftarrow \{v \in N^+(G, v) \mid f_u(u, v)\} \\
&\quad \textbf{return } \textsc{flatten}(N)
\end{aligned}
$$

รูปที่ 14: EDGEMAP *ในรูป* $N^+(G, v)$ *ที่ถูกคือ* $N^+(G, u)$ *เมื่อ* $u = U[i]$[แก้ 22]

6.1 ฟังก์ชันพื้นฐานสำหรับกราฟ (Graph primitives)

อัลกอริทึมหลายตัวของเราไล่ทำงาน (map) บนเส้นเชื่อมที่ติดกับเซตย่อยของจุดยอด และคืนจุดยอดข้างเคียงที่สอดคล้องกับเพรดิเคตบางอย่าง แทนที่จะเขียนโค้ดที่ทำการดำเนินการนี้ซ้ำ ๆ เราเขียนแทนด้วยการดำเนินการที่เรียกว่า EDGEMAP ในแบบของ Ligra [24]

EDGEMAP รับข้อมูลเข้าเป็น U ซึ่งเป็นเซตย่อยของจุดยอด และ UPDATE ซึ่งเป็นฟังก์ชันปรับปรุง แล้วคืนอาร์เรย์ที่มีจุดยอดทั้งหมด $v \in V$ ซึ่ง $(u, v) \in E, u \in U$ และ $\textsc{update}(u, v) = \texttt{true}$ โดยปกติเราจะทำให้แน่ใจว่าผลลัพธ์ของ EDGEMAP เป็นเซต โดยทำให้แน่ใจว่าจุดยอด $v \in N(U)$ ถูกครอบครองแบบอะตอมิกโดยจุดยอดใน U เพียงจุดเดียว เราให้ตัวอย่างการสร้าง EDGEMAP อย่างง่ายบนพื้นฐานของ FLATTEN ในรูปที่ 14 โค้ดประมวลผล $u \in U$ ทั้งหมดแบบขนาน สำหรับแต่ละ u เรากรองจุดยอดข้างเคียงขาออกของมันและเก็บจุดยอดข้างเคียง v ซึ่ง $\textsc{update}(u, v) = \texttt{true}$ ไว้ในลำดับของลำดับ `nghs` เราคืนอาร์เรย์แบนโดยเรียก FLATTEN บน `nghs` ตรวจสอบได้ง่ายว่างานของการสร้างนี้คือ $O(|U| + \sum_{u \in U} deg^+(u))$ และความลึกคือ $O(\log n)$

เราสังเกตว่าการสร้างบนพื้นฐาน FLATTEN ที่ให้ไว้ที่นี่อาจไม่ค่อยใช้งานได้จริงนัก บทความหลายชิ้น [7, 24] อภิปรายการสร้าง EDGEMAP ที่มีประสิทธิภาพทั้งในทางทฤษฎีและในทางปฏิบัติ

6.2 การค้นหาตามแนวกว้างแบบขนาน (Parallel breadth-first search)

อัลกอริทึมการค้นหาในกราฟคลาสสิกอย่างหนึ่งคือการค้นหาตามแนวกว้าง (BFS) กำหนดกราฟ $G(V, E)$ และจุดยอด $v \in V$ ปัญหา BFS คือการกำหนดจุดยอดแม่ (parent) ให้แต่ละจุดยอดที่เข้าถึงได้จาก v โดยต้นไม้ที่เกิดจากเส้นเชื่อม $(u, \texttt{parent}[u])$ ทั้งหมดเป็นต้นไม้ BFS ที่ถูกต้อง (กล่าวคือ เส้นเชื่อมที่ไม่อยู่ในต้นไม้ $(u, v) \in E$ ใด ๆ อยู่ภายในระดับเดียวกันของต้นไม้หรืออยู่ระหว่างระดับที่ติดกัน) BFS สามารถคำนวณแบบลำดับได้ด้วยงาน $O(m)$ [13]

เราให้รหัสเทียมของอัลกอริทึมแบบขนานสำหรับ BFS ซึ่งทำงานด้วยงาน $O(m)$ และความลึก $O(\text{diam}(G) \log n)$ บน TS-MP-RAM ในรูปที่ 15 อัลกอริทึมสร้าง frontier เริ่มต้นซึ่งประกอบด้วย v เพียงจุดเดียวก่อน กำหนดค่าเริ่มต้นของอาร์เรย์ visited ให้เป็น 0 ทั้งหมด และให้แต่ละจุดยอดเป็นจุดยอดแม่ของตัวเอง ($P_v \leftarrow v$)[แก้ 23] แล้วทำเครื่องหมายว่า v ถูกเยี่ยมแล้ว เราทำ BFS โดยวนลูปขณะที่ frontier ไม่ว่าง และใช้ EDGEMAP ในแต่ละรอบเพื่อคำนวณ frontier ถัดไป ฟังก์ชันปรับปรุงที่ส่งให้ EDGEMAP ตรวจสอบว่าจุดยอดข้างเคียง v ยังไม่ถูกเยี่ยมหรือไม่ และหากยัง จะใช้ test-and-set หาก test-and-set สำเร็จ เราจะรู้ว่า u เป็นจุดยอดเดียวใน frontier ปัจจุบันที่ครอบครอง v ได้ ดังนั้นเราจึงกำหนดให้ u เป็นจุดยอดแม่ของ v และคืนค่าจริง ในกรณีอื่นคืนค่าเท็จ

```
BFS(G(V,E), v) =
  F ← {v};
  parfor v ∈ V :
    X_v ← false
    P_v ← v
  X_v ← true
  while (|F| > 0)
    F ← EDGEMAP(G, F, λ(u,v).
          if (TESTANDSET(X_v))
            P_v ← u;
            return true
          return false
  return P
```

รูปที่ 15: การค้นหาตามแนวกว้างแบบขนาน

6.3 การแยกส่วนแบบเส้นผ่านศูนย์กลางต่ำ (Low-diameter decomposition)

ปัญหาที่มีประโยชน์หลายอย่าง เช่น ความเชื่อมโยงและป่าแผ่ทอด (spanning forest) สามารถแก้แบบลำดับได้ด้วยการค้นหาตามแนวกว้าง น่าเสียดายที่ปัจจุบันยังไม่ทราบวิธีสร้างต้นไม้การค้นหาตามแนวกว้างที่มีรากที่จุดยอดหนึ่งด้วยความลึก $\text{polylog}(n)$ บนกราฟทั่วไปอย่างมีประสิทธิภาพ แทนที่จะค้นหากราฟจากจุดยอดเดียวอย่าง BFS การแยกส่วนแบบเส้นผ่านศูนย์กลางต่ำ (LDD) แบ่งกราฟออกเป็นกลุ่ม (cluster) ที่เชื่อมโยงกันจำนวนหนึ่ง โดยมีเส้นเชื่อมถูกตัดน้อย และเส้นผ่านศูนย์กลางภายในของแต่ละกลุ่มถูกจำกัด (แต่ละกลุ่มสามารถสำรวจแบบขนานได้อย่างมีประสิทธิภาพ) ต่างจาก BFS การแยกส่วนแบบเส้นผ่านศูนย์กลางต่ำสามารถคำนวณแบบขนานได้อย่างมีประสิทธิภาพ และนำไปสู่อัลกอริทึมที่เรียบง่ายสำหรับปัญหากราฟอื่นอีกหลายปัญหา เช่น ความเชื่อมโยง spanner, hop-set และต้นไม้แผ่ทอดที่มีการยืดต่ำ (low stretch spanning trees)

การแยกส่วนแบบ (β, d) แบ่ง V ออกเป็นกลุ่ม $V_1, \ldots, V_k$ โดยวิถีที่สั้นที่สุดระหว่างจุดยอดสองจุดใน V_i ที่ใช้เฉพาะจุดยอดใน V_i มีความยาวไม่เกิน d (เส้นผ่านศูนย์กลางแบบเข้ม (strong diameter)) และจำนวนเส้นเชื่อม (u, v) ที่ $u \in V_i, v \in V_j, j \neq i$ มีไม่เกิน βm การแยกส่วนแบบเส้นผ่านศูนย์กลางต่ำ (LDD) ถูกนำเสนอครั้งแรกในบริบทของการคำนวณแบบกระจาย [5] และต่อมาถูกใช้ในการฝังเมตริก (metric embedding) ตัวแก้ระบบเชิงเส้น และอัลกอริทึมแบบขนาน

ในแบบลำดับ LDD สามารถหาได้ด้วยเทคนิคการขยายลูกบอล (ball-growing) แบบลำดับอย่างง่าย [5] อัลกอริทึมเลือกจุดยอด v ใดก็ได้ที่ยังไม่ถูกครอบคลุมซ้ำ ๆ และขยายลูกบอลรอบมันด้วย BFS จนกว่าจำนวนเส้นเชื่อมที่ติดกับ frontier ปัจจุบันจะไม่เกินสัดส่วน β ของจำนวนเส้นเชื่อมภายใน แล้วจึงนำลูกบอลออก เนื่องจากเงื่อนไขการหยุด เส้นเชื่อมไม่เกินสัดส่วน β จะออกจากลูกบอล และเนื่องจากแต่ละระดับของ BFS จะเพิ่มจำนวนเส้นเชื่อมด้วยตัวประกอบ $1 + \beta$ จำนวนระดับของ BFS จึงถูกจำกัดด้วย $O(\log_{(1+\beta)} m) = O(\log n/\beta)$ ดังนั้นอัลกอริทึมให้การแยกส่วนแบบ $(\beta, O(\log n/\beta))$[แก้ 24] เนื่องจากแต่ละเส้นเชื่อมถูกตรวจสอบครั้งเดียว อัลกอริทึมจึงใช้งาน $O(n + m)$ แต่เนื่องจากลูกบอลถูกเยี่ยมทีละลูก ในกรณีแย่ที่สุดอัลกอริทึมจึงเป็นแบบลำดับอย่างสมบูรณ์

ต่อไปเราจะอภิปรายอัลกอริทึมแบบขนานที่มีประสิทธิภาพเชิงงาน [18] เช่นเดียวกับอัลกอริทึมแบบลำดับ อัลกอริทึมนี้อาศัยการขยายลูกบอลรอบจุดยอด แต่ในกรณีนี้ทำแบบขนาน การเริ่มขยายทุกลูกพร้อมกันใช้ไม่ได้ เพราะจะทำให้ทุกจุดยอดเป็นกลุ่มของตัวเอง อัลกอริทึมจึงขยายลูกบอลจากจุดยอดตามเวลาเริ่มต้นที่ถูกเลื่อนแบบสุ่มตามการแจกแจงเลขชี้กำลัง (exponential distribution) แทน ลูกบอลขยายด้วยอัตราหนึ่งระดับ (เส้นเชื่อม) ต่อหนึ่งหน่วยเวลา และแต่ละจุดยอดถูกกำหนดให้อยู่ในลูกบอลแรกที่มาถึงมัน เช่นเดียวกับกรณีแบบลำดับ เราสามารถขยายลูกบอลด้วย BFS

จำไว้ว่าตัวแปรสุ่มต่อเนื่องที่ไม่เป็นลบ X มีการแจกแจงเลขชี้กำลังด้วยพารามิเตอร์อัตรา $\beta > 0$ หากฟังก์ชันความหนาแน่นของความน่าจะเป็น (probability density function) ของมันคือ

$$f_X(x) = \beta e^{-\beta x}. \quad \text{[แก้ 25]}$$

โดยการอินทิเกรต ความน่าจะเป็นสะสมคือ:

$$\mathbf{Pr}[X \leq x] = 1 - e^{-\beta x}.$$

สมบัติสำคัญของการแจกแจงเลขชี้กำลังคือมันไม่มีความจำ (memoryless) กล่าวคือ

$$\mathbf{Pr}[X > a + b \mid X > a] = \mathbf{Pr}[X > b]$$

กล่าวเป็นคำพูดคือ หากเราเอาหางของการแจกแจงที่เลยจุด a ไป และปรับสเกลให้ความน่าจะเป็นที่เหลือทั้งหมดเป็น 1 (กล่าวคือภายใต้เงื่อนไข $X > a$) การแจกแจงที่เหลือจะเหมือนกับการแจกแจงเดิม (มันลืมไปแล้วว่ามีอะไรเกิดขึ้น)

ในการเลือกเวลาเริ่มต้น แต่ละจุดยอด v เลือก δ_v จากการแจกแจงเลขชี้กำลังด้วยพารามิเตอร์ β แล้วแต่ละจุดใช้เวลาเริ่มต้น $T_v = \delta_{\max} - \delta_v$ เมื่อ $\delta_{\max} = \max_{v \in V} \delta_v$ สังเกตว่าการแจกแจงเลขชี้กำลังถูกนำมาลบ ดังนั้นเวลาเริ่มต้นจึงมีการแจกแจงเลขชี้กำลังแบบย้อนกลับ ซึ่งเพิ่มขึ้นแทนที่จะลดลงตามเวลา โดยเฉพาะ จะมีจุดยอดน้อยมากที่เริ่มในหน่วยเวลาแรก (มักเพียงจุดเดียว) และจะมีจำนวนที่เพิ่มขึ้นแบบเลขชี้กำลังเริ่มในแต่ละหน่วยเวลาถัดไป จากเวลาเริ่มต้นเหล่านี้ ลูกบอลแรกที่มาถึงจุดยอด u และดังนั้นจะเป็นกลุ่มที่ u ถูกกำหนดให้ คือ

$$C_u = \arg\min_{v \in V}(T_v + d(u, v))$$

อัลกอริทึมนี้สามารถสร้างได้อย่างมีประสิทธิภาพโดยใช้การค้นหาตามแนวกว้างแบบขนานหลายตัวพร้อมกัน BFS แรกเริ่มที่จุดยอดที่สุ่มได้ δ_v มากที่สุด ($\delta_v = \delta_{\max}$) ซึ่งมีเวลาเริ่มต้นน้อยที่สุด $T_v = 0$[แก้ 26] แต่ละ $v \in V$ "ตื่นขึ้น" และเริ่ม BFS ของตน หากผ่านไปแล้ว $\lfloor T_v \rfloor$ ขั้น และยังไม่ถูกครอบคลุมโดย BFS ของจุดยอดอื่น รูปที่ 16 ให้รหัสเทียมของอัลกอริทึม การกำหนดกลุ่มสำหรับแต่ละจุดยอด C_v ถูกตั้งค่าเริ่มต้นเป็น unvisited และเมื่อถูกตั้งค่าโดยลูกบอลแรกที่มาถึงแล้ว จะไม่เปลี่ยนแปลง ตัวแปร γ_v เป็นที่ที่ลูกบอลแข่งกันเพื่อดูว่าลูกใดมี T_v เร็วที่สุด เราต้องการเพียงส่วนทศนิยมของ T_v เพราะเฉพาะลูกบอลที่มาถึงในรอบเดียวกันเท่านั้นที่แข่งขันกัน การสร้างที่ง่ายกว่าคือการเลือกลูกบอลใดก็ได้ที่มาถึงในรอบแรกที่มีลูกบอลมาถึง ปรากฏว่าวิธีนี้ยังคงรักษาสมบัติที่ต้องการไว้ภายในตัวประกอบคงที่ [25]

ในการวิเคราะห์งานและสแปนของอัลกอริทึม เราสังเกตว่าจำนวนรอบของอัลกอริทึมถูกจำกัดด้วย $\lceil \delta_{\max} \rceil$ เพราะทุกจุดยอดจะถูกประมวลผลแล้วเมื่อถึงตอนนั้น บรรทัดที่ 10 สามารถสร้างได้อย่างมีประสิทธิภาพโดยเรียงลำดับจุดยอดล่วงหน้าตามรอบที่มันเริ่มแล้วเพียงดึงจากถังที่เหมาะสมในแต่ละรอบ สามารถใช้ counting sort ซึ่งให้งานเชิงเส้นและสแปน $O(\log |V| + \delta_{\max})$ ทุกจุดยอดอยู่ใน frontier ไม่เกินหนึ่งครั้ง และดังนั้นทุกเส้นเชื่อมถูกเยี่ยมไม่เกินหนึ่งครั้ง งานรวมจึงเป็น $O(|V| + |E|)$ แต่ละรอบของอัลกอริทึมมีสแปนไม่เกิน $O(\log |V|)$ ดังนั้นสแปนรวมคือ $O(\delta_{\max} \log |V|)$

ต่อไปเราวิเคราะห์รัศมีของกลุ่มและจำนวนเส้นเชื่อมระหว่างกลุ่ม ก่อนอื่นเราแสดงว่ารัศมีสูงสุดของแต่ละลูกบอลคือ $O(\log n/\beta)$ w.h.p. เราได้กล่าวไปแล้วว่าจำนวนรอบของอัลกอริทึม LDD ถูกจำกัดด้วย $\delta_{\max}$ ซึ่งจำกัดรัศมีของ BFS ใด ๆ ด้วย และดังนั้นเส้นผ่านศูนย์กลางถูกจำกัดด้วย $2\delta_{\max}$ ในการจำกัด $\delta_{\max}$ พิจารณาความน่าจะเป็นที่จุดยอดหนึ่งจะเลือกการเลื่อน (shift) ที่ใหญ่กว่า $\frac{c \log n}{\beta}$:

$$\mathbf{Pr}\left[\delta_v > \frac{c \log n}{\beta}\right] = 1 - \mathbf{Pr}\left[\delta_v \leq \frac{c \log n}{\beta}\right] = 1 - (1 - e^{-c \log n}) = \frac{1}{n^c}$$

ต่อไป ใช้ขอบเขตยูเนียนบนจุดยอดทั้ง n จุด เราได้ว่าความน่าจะเป็นที่จุดยอดใดจะเลือกการเลื่อนที่ใหญ่กว่า $\frac{c \log n}{\beta}$ คือ:

$$\mathbf{Pr}\left[\delta_{\max} > \frac{c \log n}{\beta}\right] \leq \frac{1}{n^{c-1}}$$

ซึ่งบ่งบอกว่า $\delta_{\max} \in O(\log n/\beta)$ w.h.p.

ต่อไปเราแสดงว่ามีเส้นเชื่อมไม่เกิน βm เส้นถูกตัดในค่าคาดหมาย ขั้นแรกเราจะแสดงว่าไม่น่าจะเป็นไปได้ที่ลูกบอล k ลูกแรกที่ครอบคลุมจุดยอดหรือจุดกึ่งกลางของเส้นเชื่อมจะมาถึงทั้งหมดภายในช่วงเวลาสั้น ๆ จุดกึ่งกลางของเส้นเชื่อมในที่นี้หมายถึงเมื่อลูกบอลครอบคลุมเส้นเชื่อมครึ่งหนึ่งจากด้านหนึ่ง ลองนึกภาพจุดยอดที่อยู่กลางเส้นเชื่อมซึ่งมีระยะ .5 จากปลายแต่ละด้าน

```
1   parfor v ∈ V : δ_v ← Exp(β);
2   δ_max = max_{v∈V} δ_v;
3   parfor v ∈ V :
4       T_v ← δ_max − δ_v;
5       C_v ← unvisited;
6       γ_v ← ∞
7   l ← 0;
8   r ← 1;
9   while (l < |V|)
10    F ← F ∪ {v ∈ V | (T_v < r) ∧ (C_v = unvisited)};
11    l ← l + |F|;
12    EDGEMAP(G, F, λ(u, v).
13      if (C_v = unvisited)
14        c ← C_u;
15        WRITEMIN(γ_v, T_c − ⌊T_c⌋))
16    F ← EDGEMAP(G, F, λ(u, v).
17        c ← C_u;
18        if (γ_v = T_c − ⌊T_c⌋)
19          C_v ← c;
20          return true
21        return false)
22    r ← r + 1
23  return C
```

รูปที่ 16: การแยกส่วนแบบเส้นผ่านศูนย์กลางต่ำ

บทตั้ง 6.1. *สำหรับจุดยอดหรือจุดกึ่งกลางของเส้นเชื่อม u ใด ๆ ความน่าจะเป็นที่ค่าที่เล็กที่สุดและค่าที่เล็กเป็นลำดับที่ k จาก $\{T_v + d(v,u) : v \in V\}$ จะต่างกันน้อยกว่า a มีค่าไม่เกิน $\left(1 - e^{-\beta a}\right)^{k-1}$*

พิสูจน์. เราพิจารณาการขยายลูกบอลที่ทำงานย้อนกลับในเวลา ดังนั้นลูกบอลจะหดลงและเลิกครอบคลุม (uncover) จุดต่าง ๆ ไปตามเวลา จำไว้ว่าเนื่องจากเราลบการแจกแจงเลขชี้กำลัง δ_v เพื่อหาเวลาเริ่มต้น มันจึงเป็นการแจกแจงเลขชี้กำลังที่แท้จริงเมื่อย้อนกลับในเวลา พิจารณาเวลา t เมื่อลูกบอลที่ช้าที่สุดลำดับที่ k (ในเวลาย้อนกลับ) เลิกครอบคลุม u ณ จุดนี้มีลูกบอล $k-1$ ลูกที่ยังครอบคลุม u อยู่ เนื่องจากสมบัติไม่มีความจำของเลขชี้กำลัง แต่ละลูกมีการแจกแจงเลขชี้กำลังเริ่มที่ t ความน่าจะเป็นที่ลูกใดลูกหนึ่งจะถูกนำออกในช่วงเวลา a ถัดไป (ย้อนกลับอีกเช่นกัน) ได้จากการแจกแจงความน่าจะเป็นสะสม $1 - e^{-\beta a}$

ดังนั้นเราได้ว่าความน่าจะเป็นที่ลูกบอลที่เหลือลูกใดจะถูกนำออกในเวลา a น้อยกว่า $a\beta$ ทีนี้ ลูกบอลที่เหลือแต่ละลูกจาก $k-1$ ลูกเลือกเวลาเริ่มต้นอย่างเป็นอิสระ ดังนั้นความน่าจะเป็นที่ทั้งหมดจะถูกนำออกในเวลา a คือผลคูณของความน่าจะเป็นเหล่านี้ ซึ่งให้ผลลัพธ์ตามที่อ้าง □

เราสามารถจำกัด $\left(1 - e^{-\beta a}\right)^{k-1} < (\beta a)^{k-1}$ โดยใช้อสมการ $1 - e^{-\alpha} < \alpha$ สำหรับ $\alpha > 0$ ซึ่งได้จากการนำสองพจน์แรกของอนุกรมเทย์เลอร์ของ e^{-x} และสังเกตว่าพจน์ที่เหลือรวมกันได้ค่าบวก

สุดท้าย เราแสดงว่าเส้นเชื่อมจะเชื่อมระหว่างกลุ่มก็ต่อเมื่อลูกบอลสองลูกแรก (คือ $k = 2$) มาถึงจุดกึ่งกลางของมันภายในหนึ่งหน่วยเวลา (คือ $a = 1$) โดยเฉพาะ หากลูกบอลแรกมาถึงที่เวลา t (ตอนนี้เราพูดถึงเวลาไปข้างหน้า) และลูกถัดไปมาถึงที่เวลามากกว่า $t + 1$ ก็จะไม่มีลูกบอลอื่นไปถึงปลายของเส้นเชื่อมได้จนกว่าจะเลยเวลา $t + 1/2$ ณ จุดนี้ ลูกบอลแรกได้ครอบครองปลายทั้งสองแล้ว ดังนั้นทั้งสองจะอยู่ในกลุ่มเดียวกัน ดังนั้นโดยบทตั้ง 6.1 ความน่าจะเป็นที่เส้นเชื่อมจะเชื่อมระหว่างกลุ่มถูกจำกัดบนด้วย $(1\beta)^{2-1} = \beta$ สังเกตว่าเราไม่ได้ใช้บทตั้งในรูปทั่วไปเต็มที่ แต่จะใช้เมื่ออภิปรายเรื่อง spanner ของกราฟ เมื่อรวมทั้งหมด เราจึงได้ทฤษฎีบทต่อไปนี้

$\textsc{Connectivity}(G(V,E),\beta) =$
  $L \leftarrow \mathrm{LDD}(G,\beta)$;
  $G'(V',E') = \textsc{Contract}(G,L)$;
  if $(|E'|=0)$
    return L
  $L' \leftarrow \textsc{Connectivity}(G',\beta)$
  parfor $v \in V$:
    $u \leftarrow L_v$;
    $L''_v \leftarrow L'_u$
  return L''

รูปที่ 17: ความเชื่อมโยงแบบขนาน

ทฤษฎีบท 6.1. *สำหรับกราฟไม่มีทิศทาง $G=(V,E)$ และสำหรับ $\beta > 0$ ใด ๆ มีอัลกอริทึมแบบขนานเชิงสุ่มสำหรับหาการแยกส่วนแบบ $\left(\beta, O\left(\frac{\log|V|}{\beta}\right)\right)$ ของกราฟ ด้วยงาน $O(|V|+|E|)$ และสแปน $O\left(\frac{\log^2|V|}{\beta}\right)$*

พิสูจน์. เราเพิ่งแสดงว่าความน่าจะเป็นที่เส้นเชื่อมจะอยู่ระหว่างกลุ่มมีค่าไม่เกิน β โดยความเป็นเชิงเส้นของค่าคาดหมายข้ามเส้นเชื่อม หมายความว่าจำนวนคาดหมายของเส้นเชื่อมระหว่างส่วนประกอบมีไม่เกิน $\beta|E|$ เราได้แสดงข้างต้นว่า $\delta_{\max}$ และดังนั้นเส้นผ่านศูนย์กลางของส่วนประกอบถูกจำกัดด้วย $O\left(\frac{\log|V|}{\beta}\right)$ w.h.p. การแทน $\delta_{\max}$ ลงในการวิเคราะห์ต้นทุนของอัลกอริทึม LDD ให้ขอบเขตของงานและสแปน □

6.4 ความเชื่อมโยง (Connectivity)

6.5 Spanners

กราฟย่อย H ของ $G=(V,E)$ เป็น *k-spanner* ของ G หากสำหรับทุกคู่ของจุดยอด $u,v \in V$, $d_H(u,v) \le k d_G(u,v)$ โดยเฉพาะ spanner รักษาระยะทางไว้ภายในตัวประกอบ k มีขอบเขตล่างที่เป็นข้อคาดการณ์ (conjectured lower-bound) ซึ่งระบุว่าโดยทั่วไป spanner H ต้องมีเส้นเชื่อม $\Omega(n^{1+1/\lceil k/2\rceil})$ เส้น [20] ขอบเขตล่างนี้อาศัยข้อคาดการณ์เส้นรอบวงของแอร์ดิช (Erdos Girth Conjecture) เส้นรอบวง (girth) ของกราฟคือขนาดของวัฏจักรที่เล็กที่สุดของมัน และข้อคาดการณ์ระบุว่าจำนวนเส้นเชื่อมสูงสุดที่เป็นไปได้ในกราฟที่มีเส้นรอบวง k คือ $\Omega(n^{1+1/\lceil k/2\rceil})$ กราฟไม่มีน้ำหนักที่มีวัฏจักรเล็กที่สุด $> k+1$ ไม่สามารถมี k-spanner ได้ เพราะการตัดเส้นเชื่อมใด ๆ จะบิดเบือนน้ำหนักบนวัฏจักรของมันด้วยตัวประกอบที่มากกว่า k ดังนั้นข้อคาดการณ์เส้นรอบวง หากเป็นจริง จะบ่งบอกถึงขอบเขตล่างนี้

มีอัลกอริทึมแบบลำดับและแบบขนานมากมายที่ได้ $O(k)$-spanner ที่มีเส้นเชื่อม $O(n^{1+1/k})$ เส้น ด้วยการแลกเปลี่ยนต่าง ๆ กัน [4] ในที่นี้เราอธิบายอัลกอริทึมแบบขนานอย่างง่ายที่ใช้การแยกส่วนแบบเส้นผ่านศูนย์กลางต่ำ (LDD) [17] เราอธิบายเพียงรูปแบบสำหรับกราฟไม่มีน้ำหนัก แต่แนวคิดสามารถขยายไปยังกราฟมีน้ำหนักได้ [17]

แนวคิดคือเพียงรันการแยกส่วนแบบเส้นผ่านศูนย์กลางต่ำด้วยพารามิเตอร์ $\beta = \ln n/2k$ สำหรับแต่ละกลุ่มที่สร้างโดย LDD เราเพิ่มเส้นเชื่อมของต้นไม้ BFS ของกลุ่มนั้นลงใน spanner ของเรา เรายังเพิ่มเส้นเชื่อมหนึ่งเส้นสำหรับทุกคู่ของกลุ่มที่มีเส้นเชื่อมระหว่างกันอย่างน้อยหนึ่งเส้น สังเกตว่าสิ่งสำคัญคือ หากมีเส้นเชื่อมหลายเส้นระหว่างคู่ของกลุ่ม เราเก็บไว้เพียงเส้นเดียว ไม่สำคัญว่าเส้นใด เราอ้างว่า ด้วยความน่าจะเป็นสูง spanner ที่สร้างด้วยวิธีนี้จะรักษาระยะทางของวิถีที่สั้นที่สุดไว้ภายใน $O(k)$ ในขณะที่มีเส้นเชื่อมไม่เกิน $O(n^{1+1/k})$ เส้นในค่าคาดหมาย

ก่อนอื่นเราแสดงว่ากราฟรักษาระยะทางไว้ โดยเฉพาะเราแสดงว่าทุกเส้นเชื่อมมีการยืด (stretch) ไม่เกิน $O(k)$ ด้วยความน่าจะเป็นสูง (กล่าวคือ หากเรานำมันออก จะมีวิถีอื่นที่มีความยาว $O(k)$ ระหว่างปลายทั้งสองของมัน) ประการแรก เราสังเกตว่าจากการวิเคราะห์ LDD แต่ละกลุ่มมีเส้นผ่านศูนย์กลาง $O(\ln n/\beta)$ ด้วยความน่าจะเป็นสูง แทนค่า β ของเรา จะได้เส้นผ่านศูนย์กลางเป็น $O(k)$ ด้วยความน่าจะเป็นสูง ทีนี้สังเกตว่าเส้นเชื่อมอยู่ภายในกลุ่มหรือไม่ก็อยู่ระหว่างกลุ่ม เนื่องจากวิถีที่ยาวที่สุดที่ใช้ต้นไม้ BFS ถูกจำกัดด้วยสองเท่าของเส้นผ่านศูนย์กลาง เส้นเชื่อมใด ๆ ภายในกลุ่มสามารถใช้ต้นไม้ BFS และดังนั้นมีการยืดที่ถูกจำกัดด้วย

$O(k)$ ทีนี้พิจารณาเส้นเชื่อมระหว่างกลุ่ม และโดยเฉพาะเส้นที่เราไม่ได้เก็บไว้ เนื่องจากมีเส้นเชื่อมอีกเส้นระหว่างกลุ่ม เราสามารถใช้วิถีภายในกลุ่มที่ปลายด้านหนึ่งไปยังเส้นเชื่อมนั้น จากนั้นข้ามเส้นเชื่อม แล้วไปภายในอีกกลุ่มหนึ่งถึงปลายอีกด้าน วิถีนี้ก็มีความยาว $O(k)$ เช่นกัน

ต่อไปเราแสดงว่าจำนวนเส้นเชื่อมถูกจำกัดด้วย $O(n^{1+1/k})$ ในค่าคาดหมาย โดยเฉพาะ พิจารณาจุดยอดบนขอบของกลุ่ม มันอาจมีเส้นเชื่อมไปยังกลุ่มข้างเคียงจำนวนหนึ่ง สังเกตว่า การจะมีเส้นเชื่อมไปยังกลุ่มข้างเคียง l กลุ่ม ค่าที่เล็กที่สุดและค่าที่เล็กเป็นลำดับที่ l ที่มาถึงกลุ่มใดในนั้นต้องต่างกันไม่เกินสอง บทตั้ง 6.1 แสดงว่าความน่าจะเป็นของสิ่งนี้มีค่าไม่เกิน $\left(1-e^{-\beta a}\right)^{l-1} = \left(1-e^{-2\ln n/2k}\right)^{l-1} = \left(1-n^{-1/k}\right)^{l-1}$[แก้ 27] พิจารณาตัวแปรสุ่ม L ซึ่งคือจำนวนกลุ่มข้างเคียงของจุดยอด ในการคำนวณค่าคาดหมายของ L เราสามารถรวมข้ามค่าที่เป็นไปได้ของ l ได้:

$$E[L] = \sum_{l=1}^{\infty} P(L \geq l) \leq \sum_{l=1}^{\infty} \left(1-n^{-1/k}\right)^{l-1}$$

ใช้เอกลักษณ์ $\sum_{i=0}^{\infty} a^i = 1/(1-a)$ สำหรับ $a<1$ เราได้ว่า $E[L] \leq n^{1/k}$ หากเราคูณข้ามจุดยอดทั้งหมด เราได้ว่าขอบเขตคาดหมายของจำนวนเส้นเชื่อมข้ามกลุ่มทั้งหมดคือ $O(n^{1+1/k})$ ตามที่ต้องการ

6.6 เซตอิสระสูงสุด (Maximal Independent Set)

ในกราฟ $G=(V,E)$ เรากล่าวว่าเซตของจุดยอด $U \subset V$ เป็น*เซตอิสระ* (independent set) หากไม่มีจุดยอดคู่ใดในเซตเป็นจุดข้างเคียงกันใน G (กล่าวคือ $(U \times U) \cap E = \emptyset$) เซตของจุดยอด $U \subset V$ เป็น*เซตอิสระสูงสุด* (maximal independent set: MIS) หากเป็นเซตอิสระและไม่มีจุดยอดใดใน $V \setminus U$ ที่สามารถเพิ่มเข้าไปใน U ได้โดยที่ผลลัพธ์ยังเป็นเซตอิสระ ปัญหา MIS คือการหา MIS ในกราฟ ในแบบลำดับนั้นค่อนข้างง่าย เพียงเรียงจุดยอดในลำดับใดก็ได้ แล้วเพิ่มทีละจุด โดยเมื่อใดที่เพิ่มจุดยอด เราจะนำจุดข้างเคียงทั้งหมดของมันออกจากการพิจารณา ซึ่งดูเป็นกระบวนการแบบลำดับโดยธรรมชาติ

$N(G,v), N^-(G,v), N^+(G,v)$	neighbors of v in G $(+ = \text{out}, - = \text{in})$
$N(G,V), N^-(G,V), N^+(G,V)$	$N(G,V) = \bigcup_{v \in V} N(G,v)$ (similarly for N^- and N^+)

รูปที่ 18: สัญกรณ์บางส่วนที่เราใช้สำหรับกราฟ

$\textsc{LubyMIS}(G=(V,E)) =$
  if $(|V|=0)$ **return** $\{\}$
  parfor $v \in V$: $\rho_v \leftarrow$ `random priority`
  $I \leftarrow \{v \in V \mid \rho(v) < \min_{u \in N(G,v)} \rho(u)\}$
  $V' \leftarrow V \setminus (I \cup N(G,I))$
  $G' \leftarrow (V', \{(u,v) \in E \mid (u \in V' \wedge v \in V')\})$
  $R \leftarrow \textsc{LubyMIS}(G')$
  return $I \cup R$

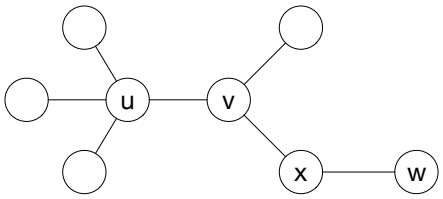
รูปที่ 19: อัลกอริทึมของ Luby สำหรับ MIS *(บรรทัดที่หา I ในรูปเลือกจุดยอดที่มี ρ น้อยกว่าจุดข้างเคียงทั้งหมด ซึ่งขัดกับคำอธิบายและบทพิสูจน์ที่ใช้ค่าสูงสุดเฉพาะที่ ที่ถูกคือ $I \leftarrow \{v \in V \mid \rho(v) > \max_{u \in N(G,v)} \rho(u)\}$)*[แก้ 28]

ในที่นี้เราอธิบายอัลกอริทึมของ Luby สำหรับหา MIS แบบขนานอย่างมีประสิทธิภาพ รหัสเทียมแสดงในรูปที่ 19[แก้ 29] อัลกอริทึมทำงานเป็นลำดับของการเรียกเวียนเกิด ในแต่ละการเรียก มันกำหนดลำดับความสำคัญ (priority) แบบสุ่มให้จุดยอดที่เหลือแบบขนาน จากนั้นระบุเซตของจุดยอด I ที่เป็นค่าสูงสุดเฉพาะที่ (local maximum) เทียบกับลำดับความสำคัญ (กล่าวคือ จุดข้างเคียงทั้งหมดมีลำดับความสำคัญต่ำกว่า) เซต I เป็นเซตอิสระเพราะจุดข้างเคียงสองจุดไม่สามารถเป็นค่าสูงสุดเฉพาะที่ทั้งคู่ได้ ดังนั้นอัลกอริทึมจะเพิ่มมันลงในผลลัพธ์ (บรรทัดสุดท้าย) อย่างไรก็ตาม มันไม่จำเป็นต้องเป็นเซตอิสระสูงสุด เพื่อระบุจุดยอดเพิ่มเติมที่จะ

เพิ่มลงใน MIS เรานำจุดยอด I และจุดข้างเคียงของมันออกจากกราฟ พร้อมกับเส้นเชื่อมที่ติดกัน เรารู้ว่าจุดข้างเคียง $N(G, I)$ ไม่สามารถอยู่ในเซตอิสระเดียวกับ I ได้ ตอนนี้อัลกอริทึมเรียกเวียนเกิดบนกราฟที่เหลือ

เราสามารถแสดงว่าอัลกอริทึมสร้าง MIS ได้โดยอุปนัย สมมติโดยอุปนัยว่ามันเป็นจริงสำหรับการเรียกเวียนเกิดที่เล็กกว่า กรณีฐานเป็นจริงเพราะกราฟว่างไม่มีจุดยอดที่จะใส่ใน MIS สำหรับการเรียกอื่นใด เรารู้ว่า I เป็นอิสระ และเรารู้ว่าเมื่อ I อยู่ใน MIS แล้ว จุดยอดใน $N(G, I)$ จะอยู่ไม่ได้ จุดยอดที่เหลือใน V' ใด ๆ เป็นอิสระจาก I เนื่องจากโดยอุปนัย การเรียกเวียนเกิดคืน MIS จึงไม่มีจุดยอดเพิ่มเติมที่เราสามารถเพิ่มลงใน R ได้โดยที่เซตยังคงเป็นอิสระ และก็ไม่มีจาก $N(G, I)$ ด้วย ซึ่งแสดงว่า $I \cup R$ เป็นอิสระและสูงสุด

การพิสูจน์ขอบเขตต้นทุนโดยรวมยุ่งยากกว่าเล็กน้อย สิ่งที่เราจะแสดงคือเราคาดว่าจะนำเส้นเชื่อมออกอย่างน้อยครึ่งหนึ่งในแต่ละรอบ (แต่ละการเรียกเวียนเกิด) การพิสูจน์นี้น่าสนใจเพราะเมื่อเพิ่มจุดยอดลงใน MIS เราจะไม่นับการนำเส้นเชื่อมของมันออก แต่จะนับการนำเส้นเชื่อมบางเส้นของจุดข้างเคียงออกแทน จุดประสงค์ของแนวทางนี้คือหลีกเลี่ยงการนับซ้ำ ปรากฏว่าสัดส่วนคาดหมายของจุดยอดที่ถูกเก็บไว้ในรอบหนึ่งอาจมากกว่า $1/2$ จำนวนจริงขึ้นอยู่กับกราฟเฉพาะ



รูปที่ 20: ตัวอย่างสำหรับการพิสูจน์ขอบเขตของอัลกอริทึมของ Luby

ทฤษฎีบท 6.2. *อัลกอริทึมของ Luby หนึ่งรอบนำเส้นเชื่อมออกอย่างน้อยครึ่งหนึ่งในค่าคาดหมาย*

พิสูจน์. พิจารณาเส้นเชื่อม (u, v) ให้ $A_{u,v}$ เป็นเหตุการณ์ที่ u มีลำดับความสำคัญสูงที่สุดในบรรดา u, v และจุดข้างเคียงทั้งหมดของทั้งสอง เนื่องจากลำดับความสำคัญถูกเลือกแบบสุ่มสม่ำเสมอ ในการหาความน่าจะเป็นของเหตุการณ์นี้ เราเพียงต้องนับจำนวนจุดยอดดังกล่าว ซึ่งให้:

$$\mathbf{Pr}[A_{u,v}] \geq \frac{1}{d(u) + d(v)}$$

มันเป็นอสมการเพราะ u และ v อาจมีจุดข้างเคียงร่วมกัน รูปที่ 20 แสดงตัวอย่างที่ $\mathbf{Pr}[A_{u,v}] = 1/(d(u) + d(v)) = 1/(4 + 3) = 1/7$ เมื่อเหตุการณ์ $A_{u,v}$ เกิดขึ้น u จะถูกเลือกให้อยู่ใน MIS อย่างแน่นอน และเส้นเชื่อมทั้งหมดที่ติดกับ v จะถูกนำออก เพราะ v เป็นจุดข้างเคียงของ u และตัวมันเองจะถูกนำออก ดังนั้นเราคาดว่าจะนำจุดข้างเคียง $\mathbf{Pr}[A_{u,v}]d(v)$ จุดของ v ออกเนื่องจากเหตุการณ์ $A_{u,v}$ สังเกตว่าเส้นเชื่อมที่ติดกับ u ก็จะถูกนำออกด้วย แต่เราจะไม่นับเส้นเหล่านี้ ยกเว้น (u, v) เพราะเป็นจุดข้างเคียงของ v

ทีนี้สังเกตว่าเมื่อเรามีเหตุการณ์ $A_{u,v}$ เราไม่สามารถมีเหตุการณ์อื่น $A_{w,v}$ ในรอบเดียวกันได้ เพราะจะหมายความว่าทั้ง $\rho(u) > \rho(w)$ และ $\rho(w) > \rho(u)$ ซึ่งเป็นไปไม่ได้ ดังนั้นหากเรานับเส้นเชื่อมที่ติดกับ v ที่ถูกนำออกโดย $A_{u,v}$ เราจะไม่นับซ้ำเนื่องจากเหตุการณ์ $A_{w,v}$ อื่นที่เกิดพร้อมกัน อย่างไรก็ตาม เนื่องจากเส้นเชื่อมมีปลายสองด้าน เส้นเชื่อมยังคงอาจถูกนับสองครั้ง ครั้งละหนึ่งจากปลายแต่ละด้าน ตัวอย่างเช่นในรูปที่ 20 เราอาจมีเหตุการณ์ $A_{u,v}$ และ $A_{w,x}$ พร้อมกัน ซึ่งแต่ละเหตุการณ์จะนับเส้นเชื่อม v, x ในเส้นเชื่อมที่มันนำออก

ให้ Y เป็นตัวแปรสุ่มที่ให้จำนวนเส้นเชื่อมที่เรานำออกในค่าคาดหมาย เราอ้างว่านิพจน์ต่อไปนี้เป็นค่าคาดหมายของ Y

$$E[Y] \geq \frac{1}{2} \sum_{(u,v) \in E} (\mathbf{Pr}[A_{u,v}]d(v) + \mathbf{Pr}[A_{v,u}]d(u))$$

เราได้สิ่งนี้โดยรวมผลของเส้นเชื่อมที่ถูกนำออกสำหรับทั้งสองทิศทางของเส้นเชื่อม (u, v) และคำนึงถึงการนับซ้ำเมื่อนำเส้นเชื่อมออกจากปลายแต่ละด้าน (คือตัวประกอบ $1/2$) แต่ละพจน์ของผลรวมมีค่าอย่างน้อย 1 รวมทั้งหมด $E[Y] \geq |E|/2$ ซึ่งเป็นสิ่งที่เราต้องการพิสูจน์ □

ต่อไปเราพิจารณาต้นทุนโดยรวม การกำหนดลำดับความสำคัญ การหาค่าสูงสุดของจุดข้างเคียงเพื่อหา I[แก้ 28] และการสร้างกราฟใหม่ ล้วนทำได้ด้วยงาน $O(|V| + |E|)$ และสแปน $O(\log |V|)$ ดังนั้นการเรียก LubyMIS หนึ่งครั้งโดยไม่รวมการเรียกเวียนเกิด ใช้งาน $O(|V| + |E|)$ และสแปน $O(\log |V|)$ โดยสัญชาตญาณ เนื่องจากจำนวนเส้นเชื่อมลดลงอย่างน้อย $1/2$ ในค่าคาดหมาย และจุดยอดจะถูกนำออกหากไม่มีเส้นเชื่อมที่ติดกัน ขนาดของกราฟจะลดลงแบบเรขาคณิต ซึ่งจะให้งาน $O(|V| + |E|)$ จำนวนรอบ $O(\log |E|)$ และดังนั้นสแปน $O(\log^2 |V|)$ อย่างไรก็ตาม เนื่องจากเป็นกระบวนการสุ่ม เราจึงต้องระมัดระวังมากขึ้น

ให้ F_i เป็นตัวแปรสุ่มสำหรับสัดส่วนของเส้นเชื่อมที่*เหลืออยู่*หลังขั้นที่ i[แก้ 30] และให้ M_i เป็นจำนวนเส้นเชื่อมที่เหลือในขั้นที่ i กล่าวคือ $M_i = m \prod_{j=0}^{i-1} F_j$ สมมติว่าเรามี $E[F_i] \leq \beta$ (สำหรับอัลกอริทึมของ Luby $\beta = 1/2$) นอกจากนี้ เนื่องจากเราเลือกลำดับความสำคัญแบบสุ่มในแต่ละรอบ อสมการนี้จึงเป็นจริงแม้ภายใต้เงื่อนไขของผลลัพธ์ทั้งหมดของการเรียกเวียนเกิดก่อนหน้าทั้งหมด ให้ $\bar{E}[F_i] = \beta$ เป็นขอบเขตบนของค่าคาดหมายแบบมีเงื่อนไข เนื่องจากขอบเขตบนเหล่านี้เป็นอิสระจาก F_i ก่อนหน้า เราสามารถนำผลคูณของค่าคาดหมายที่ถูกจำกัดเหล่านี้มาจำกัดค่าคาดหมายของผลคูณของ F_i ได้:

$$E[M_i] \leq m \prod_{j=0}^{i-1} \bar{E}[F_j] = m\beta^i$$

หากงานในรอบที่ i หนึ่งรอบคือ cM_i (สมมติว่า $|E_i| \geq |V_i|$) แล้วโดยความเป็นเชิงเส้นของค่าคาดหมาย เราจะได้:

$$E[W(m)] \leq \sum_{i=0}^{\infty} cE[M_i] \leq cm \sum_{i=0}^{\infty} \beta^i = O(m)$$

งานบนจุดยอดถูกจำกัดด้วยงานบนเส้นเชื่อม

ในการจำกัดสแปน เราใช้อสมการของมาร์คอฟ:

$$\mathbf{Pr}[X \geq a] \leq \frac{E[X]}{a}$$

และพิจารณาความน่าจะเป็น $\mathbf{Pr}[M_i \geq 1]$ สังเกตว่าหาก $M_i < 1$ อัลกอริทึมจะเสร็จสิ้นเพราะไม่มีเส้นเชื่อมเหลือ ดังนั้นเราเพียงต้องการแสดงว่าสำหรับ i ที่ใหญ่เพียงพอ ความน่าจะเป็นมีค่าน้อย เรามี:

$$\mathbf{Pr}[M_i \geq 1] \leq \frac{E[M_i]}{1} \leq m\beta^i$$

กำหนด $i = k \lg m$ สำหรับ k บางค่า เราได้:

$$\mathbf{Pr}[M_i \geq 1] \leq m\beta^{k \lg m} = mm^{k \lg \beta} = m^{1+k \lg \beta}$$

ในกรณีของเรา $\beta = 1/2$ ดังนั้น $\lg \beta = -1$ ได้:

$$\mathbf{Pr}[\text{rounds} < k \lg m] \geq 1 - m^{1-k}$$

ซึ่งบ่งบอกว่าจำนวนรอบคือ $O(\log m)$ (และดังนั้นก็เป็น $O(\log n)$ ด้วย) w.h.p.

Claude อธิบายเพิ่ม: สรุปบทที่ 6 และภาพรวมเทคนิค

- EDGEMAP คือ "ขยาย frontier หนึ่งก้าว" แบบขนาน BFS คือการเรียก EDGEMAP ซ้ำจน frontier ว่าง ดังนั้นสแปนจึงเป็น (จำนวนระดับ) $\times$ $O(\log n)$ = $O(\mathbf{diam}(G) \log n)$ ซึ่งแย่เมื่อกราฟมีเส้นผ่านศูนย์กลางใหญ่ เช่น กราฟที่เป็นเส้นตรงยาว
- **LDD แก้ปัญหานี้** โดยปล่อย BFS หลายลูกพร้อมกันตามเวลาเริ่มแบบสุ่มเลขชี้กำลัง ทำให้ทุกกลุ่มมีรัศมี $O(\log n/\beta)$ และมีเส้นเชื่อมถูกตัดเพียง βm ในค่าคาดหมาย
- **เทคนิคการหดกราฟ (contraction)** ในรูปที่ 17: ยุบแต่ละกลุ่มเป็นจุดยอดเดียวแล้วทำซ้ำ เพราะเหลือเส้นเชื่อมเพียงสัดส่วน β ในแต่ละรอบ งานรวมจึงเป็นอนุกรมเรขาคณิต $O(m)$ (ต้นฉบับหัวข้อ 6.4 ยังไม่มีเนื้อหา มีเพียงรูปที่ 17)
- **Luby's MIS:** การสุ่มลำดับความสำคัญทำให้ "ค่าสูงสุดเฉพาะที่" เลือกพร้อมกันได้โดยไม่ขัดแย้งกัน และแต่ละรอบลบเส้นเชื่อมได้ครึ่งหนึ่งในค่าคาดหมาย จึงจบใน $O(\log m)$ รอบ w.h.p.

เทคนิคที่วนกลับมา: การสุ่ม (บทที่ 3) + การหดขนาดแบบเรขาคณิต ซึ่งเป็นรูปแบบเดียวกับ kthSmallest (4.5) และ quicksort (5.2)

7 ต้นไม้ทวิภาคแบบขนาน (Parallel Binary Trees)

ในหัวข้อนี้ เรานำเสนออัลกอริทึมแบบขนานบางตัวสำหรับต้นไม้ทวิภาคที่สมดุล ระเบียบวิธีในหัวข้อนี้อาศัยกรอบงานเชิงอัลกอริทึมที่เรียกว่า*อัลกอริทึมบนพื้นฐาน join* (join-based algorithms) [8] *join* เป็นฟังก์ชันพื้นฐานที่นิยามสำหรับแต่ละรูปแบบการรักษาสมดุล (balancing scheme) อัลกอริทึมต้นไม้อื่น ๆ ทั้งหมดจัดการการปรับสมดุลและการหมุน (rotation) ผ่าน *join* และดังนั้นจึงสามารถใช้โค้ดเดียวกันได้ข้ามหลายรูปแบบการรักษาสมดุล

ฟังก์ชัน *join*(T_L, e, T_R) สำหรับรูปแบบการรักษาสมดุลหนึ่ง ๆ รับต้นไม้ทวิภาคที่สมดุลสองต้น T_L, T_R ซึ่งสมดุลตามรูปแบบนั้น และรายการ (entry) เดี่ยว e เป็นข้อมูลเข้า แล้วคืนต้นไม้ทวิภาคที่สมดุลและถูกต้องต้นใหม่ ซึ่งมีรายการเดียวกันและการท่องแบบตามลำดับ (in-order traversal) เดียวกันกับ *node*(T_L, e, T_R) แต่สอดคล้องกับเกณฑ์การรักษาสมดุล เราเรียกรายการตรงกลาง e ว่า*ตัวกลาง* (pivot) ของ *join*

7.1 ความรู้เบื้องต้น

ต้นไม้ทวิภาค คือ *nil*-node หรือโหนดที่ประกอบด้วยต้นไม้ทวิภาคทางซ้าย T_l รายการ e และต้นไม้ทวิภาคทางขวา T_r แทนด้วย *node*(T_l, e, T_r) รายการอาจเป็นเพียงคีย์ หรือคู่คีย์–ค่าก็ได้ *ขนาด* ของต้นไม้ทวิภาค หรือ $|T|$ คือ 0 สำหรับ *nil*-node และ $|T_l| + |T_r| + 1$ สำหรับ *node*(T_l, e, T_r) *น้ำหนัก* (weight) ของต้นไม้ทวิภาค หรือ $w(T)$ มีค่ามากกว่าขนาดหนึ่ง (คือจำนวนใบในต้นไม้) *ความสูง* ของต้นไม้ทวิภาค หรือ $h(T)$ คือ 0 สำหรับ *nil*-node และ $\max(h(T_l), h(T_r)) + 1$ สำหรับ *node*(T_l, e, T_r) โหนดแม่ (parent) โหนดลูก (child) บรรพบุรุษ (ancestor) และลูกหลาน (descendant) นิยามตามปกติ (บรรพบุรุษและลูกหลานรวมตัวโหนดเองด้วย) โหนดจะเรียกว่า*ใบ* (leaf) เมื่อลูกทั้งสองเป็น *nil*-node *สันซ้าย* (left spine) ของต้นไม้ทวิภาคคือวิถีของโหนดจากรากไปยังใบที่ไปทางต้นไม้ซ้ายเสมอ และ*สันขวา* (right spine) คือวิถีไปยังใบที่ไปทางต้นไม้ขวา *ค่าตามลำดับ* (in-order values) (เรียกอีกอย่างว่าลำดับสมมาตร (symmetric order)) ของต้นไม้ทวิภาคคือลำดับของค่าที่ได้จากการท่องต้นไม้แบบตามลำดับ เมื่อบริบทชัดเจน เราใช้โหนด u แทนต้นไม้ย่อย T_u ที่มีรากที่ u และในทางกลับกัน

รูปแบบการรักษาสมดุล สำหรับต้นไม้ทวิภาคคือค่าคงตัว (หรือเซตของค่าคงตัว) ที่เป็นจริงสำหรับทุกโหนดของต้นไม้ และมีจุดประสงค์เพื่อรักษาให้ต้นไม้เกือบสมดุล ในหัวข้อนี้เราพิจารณารูปแบบการรักษาสมดุลสี่แบบที่รับประกันว่าความสูงของทุกต้นไม้ขนาด n ถูกจำกัดด้วย $O(\log n)$

ต้นไม้ AVL (AVL Trees) [3]. ต้นไม้ AVL มีค่าคงตัวว่าสำหรับทุก *node*(T_l, e, T_r) ความสูงของ T_l และ T_r ต่างกันไม่เกินหนึ่ง สมบัตินี้บ่งบอกว่าต้นไม้ AVL ใด ๆ ขนาด n มีความสูงไม่เกิน $\log_\phi(n+1)$ เมื่อ $\phi = \frac{1+\sqrt{5}}{2}$ คืออัตราส่วนทองคำ

ต้นไม้แดง–ดำ (Red-black (RB) Trees) [6]. ต้นไม้ RB กำหนดสีให้ทุกโหนดและรักษาค่าคงตัวสองข้อ: (*กฎสีแดง*) ไม่มีโหนดสีแดงใดมีลูกเป็นสีแดง และ (*กฎสีดำ*) จำนวนโหนดสีดำบนทุกวิถีจากรากลงไปถึงใบเท่ากัน *nil*-node ทั้งหมดเป็นสีดำเสมอ ต่างจากการนำเสนอบางแบบ เราไม่กำหนดให้รากของต้นไม้ต้องเป็นสีดำ แม้ว่าสิ่งนี้จะไม่ส่งผลต่อความถูกต้องของอัลกอริทึมของเรา แต่การพิสูจน์ขอบเขตงานของเราต้องอนุญาตให้รากเป็นสีแดงได้ เรานิยาม*ความสูงดำ* (black height) ของโหนด T แทนด้วย $\hat{h}(T)$ ว่าเป็นจำนวนโหนดสีดำบนวิถีลงจากโหนดไปยังใบ (รวมตัวโหนดเอง) ต้นไม้ RB ใด ๆ ขนาด n มีความสูงไม่เกิน $2\log_2(n+1)$

ต้นไม้สมดุลน้ำหนัก (Weight-balanced (WB) Trees). ต้นไม้ WB นิยามด้วยพารามิเตอร์ α (เรียกอีกอย่างว่าต้นไม้ BB[α]) [19] รักษาค่าคงตัว $\alpha \le \frac{w(T_l)}{w(T)} \le 1-\alpha$ สำหรับทุก $T = $ *node*(T_l, e, T_r) เรากล่าวว่าต้นไม้สมดุลน้ำหนักสองต้น T_1 และ T_2 มีน้ำหนัก*ใกล้เคียงกัน* (like weights) หาก *node*(T_1, e, T_2) สมดุลน้ำหนัก ต้นไม้สมดุลน้ำหนัก α ใด ๆ ขนาด n มีความสูงไม่เกิน $\log_{\frac{1}{1-\alpha}} n$ สำหรับ $\frac{2}{11} < \alpha \le 1 - \frac{1}{\sqrt{2}}$ การแทรกและการลบสามารถทำบนต้นไม้สมดุลน้ำหนักได้โดยใช้เพียงการหมุนเดี่ยวและการหมุนคู่ [19][แก้ 37] เรากำหนดเงื่อนไขเดียวกันสำหรับการสร้าง *join* ของเรา และโดยเฉพาะใช้ $\alpha = 0.29$ ในการทดลอง เรายังแทน $\beta = \frac{1-\alpha}{\alpha}$ ซึ่งหมายความว่าต้นไม้ย่อยใดต้นหนึ่งไม่สามารถมีขนาดมากกว่า β เท่าของต้นไม้ย่อยอีกต้นได้

สัญกรณ์	คำอธิบาย
$\|T\|$	ขนาดของต้นไม้ T
$h(T)$	ความสูงของต้นไม้ T
$\hat{h}(T)$	ความสูงดำของต้นไม้ RB T
$w(T)$	น้ำหนักของต้นไม้ T (คือ $\|T\|+1$)
$p(T)$	โหนดแม่ของโหนด T
$k(T)$	คีย์ของโหนด T
$lc(T)$	ลูกทางซ้ายของโหนด T
$rc(T)$	ลูกทางขวาของโหนด T
$\texttt{expose}(T)$	$\langle lc(T), k(T), rc(T)\rangle$

ตารางที่ 2: **สรุปสัญกรณ์**

Treaps. [22] Treap กำหนดลำดับความสำคัญแบบสุ่มสม่ำเสมอให้ทุกโหนด และรักษาค่าคงตัวว่าลำดับความสำคัญที่แต่ละโหนดไม่น้อยกว่าลำดับความสำคัญของลูกทั้งสอง[แก้ 31] Treap ใด ๆ ขนาด n มีความสูง $O(\log n)$ ด้วยความน่าจะเป็นสูง (w.h.p)

สัญกรณ์ที่เราใช้สำหรับต้นไม้ทวิภาคสรุปไว้ในตารางที่ 2[แก้ 32]

7.2 อัลกอริทึม *join* สำหรับแต่ละรูปแบบการรักษาสมดุล

ในที่นี้เราอธิบายอัลกอริทึม *join* ของรูปแบบการรักษาสมดุลทั้งสี่ที่เรานิยามไว้ในหัวข้อ 7.1 รวมถึงนิยาม*อันดับ* (rank) สำหรับแต่ละแบบ จากนั้นเราจะพิสูจน์ว่ามัน*ทำ join ได้* (joinable) สำหรับ *join* ตัวกลางอาจเป็นเพียงรายการข้อมูล (ซึ่งอัลกอริทึมจะสร้างโหนดต้นไม้ใหม่ให้) หรือโหนดต้นไม้ที่จัดสรรไว้ล่วงหน้าในหน่วยความจำซึ่งบรรจุรายการข้อมูลที่สอดคล้องกัน (ซึ่งโหนดอาจถูกนำกลับมาใช้ใหม่ ทำให้ปรับปรุงในที่ (in-place) ได้)

ดังที่กล่าวไว้ในบทนำและตอนต้นของบทนี้ *join* ครอบคลุมสิ่งที่จำเป็นในการปรับสมดุลต้นไม้อย่างสมบูรณ์ และสามารถใช้เป็นฟังก์ชันเดียวที่รู้จักและรักษาค่าคงตัวของสมดุล สำหรับต้นไม้ AVL, RB และ WB เราแสดงว่า *join* ใช้งานที่เป็นสัดส่วนกับผลต่างของอันดับของต้นไม้ทั้งสอง สำหรับ treap งานขึ้นอยู่กับลำดับความสำคัญของ k อัลกอริทึม *join* ทั้งหมดเป็นแบบลำดับ ดังนั้นสแปนจึงเท่ากับงาน มีการแสดงไว้ใน [8][แก้ 33] ว่าอัลกอริทึม *join* สำหรับทุกรูปแบบการรักษาสมดุลที่เราพิจารณานำไปสู่งานที่เหมาะสมที่สุดสำหรับฟังก์ชันหลายอย่างบน map และ set

7.2.1 ต้นไม้ AVL

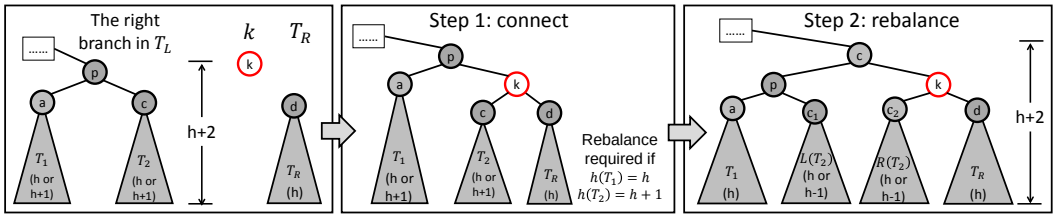
สำหรับต้นไม้ AVL เรานิยามอันดับเป็นความสูง คือ $r(T) = h(T)$ รหัสเทียมสำหรับ *join* ของ AVL แสดงในรูปที่ 21 และแสดงภาพในรูปที่ 22 ทุกโหนดเก็บความสูงของตนเอง ดังนั้น $h(\cdot)$ ใช้เวลาคงที่ หากต้นไม้ทั้งสอง T_l และ T_r มีความสูงต่างกันไม่เกินหนึ่ง *join* สามารถสร้าง $node(T_l, e, T_r)$ ใหม่ได้ทันที อย่างไรก็ตาม หากต่างกันมากกว่าหนึ่ง ต้องมีการปรับสมดุล สมมติว่า $h(T_l) > h(T_r) + 1$ (อีกกรณีสมมาตรกัน) แนวคิดคือไล่ตามสันขวาของ T_l จนพบโหนด c ซึ่ง $h(c) \leq h(T_r) + 1$ (บรรทัดที่ 3) ณ จุดนี้ $node(c, e, T_r)$ ใหม่จะถูกสร้างขึ้นแทนที่ c (บรรทัดที่ 4) เนื่องจาก $h(c) = h(T_r)$ หรือ $h(c) = h(T_r) + 1$ โหนดใหม่จึงสอดคล้องกับค่าคงตัว AVL และความสูงของมันมากกว่า c หนึ่ง ความสูงที่เพิ่มขึ้นอาจเพิ่มความสูงของบรรพบุรุษของมัน ซึ่งอาจทำให้ค่าคงตัว AVL ของโหนดเหล่านั้นไม่เป็นจริง สิ่งนี้แก้ได้ด้วยการหมุนคู่หากไม่ถูกต้องที่โหนดแม่ (บรรทัดที่ 6) หรือการหมุนซ้ายเดี่ยวหากไม่ถูกต้องที่สูงขึ้นไปในต้นไม้ (บรรทัดที่ 10) ซึ่งทั้งสองกรณีคืนความสูงสำหรับโหนดบรรพบุรุษที่สูงขึ้นไป ดังนั้นอัลกอริทึมจะต้องหมุนไม่เกินสองครั้ง ดังที่เราสรุปในบทตั้งต่อไปนี้

บทตั้ง 7.1. *อัลกอริทึม* join *ในรูปที่ 21 บนต้นไม้ AVL ต้องหมุนไม่เกินสองครั้ง*

```
1  joinRightAVL(T_l, k, T_r) {
2    (l, k', c) = expose(T_l);
3    if h(c) ≤ h(T_r) + 1 then {
4      T' = node(c, k, T_r);
5      if h(T') ≤ h(l) + 1 then return node(l, k', T');
6      else return rotateLeft(node(l, k', rotateRight(T')));
7    } else {
8      T' = joinRightAVL(c, k, T_r);
9      T'' = node(l, k', T');
10     if h(T') ≤ h(l) + 1 then return T''; else return rotateLeft(T''); }}
11 join(T_l, k, T_r) {
12   if h(T_l) > h(T_r) + 1 then return joinRightAVL(T_l, k, T_r);
13   else if h(T_r) > h(T_l) + 1 then return joinLeftAVL(T_l, k, T_r);
14   else return node(T_l, k, T_r); }
```

รูปที่ 21: **อัลกอริทึม *join* บนต้นไม้ AVL** – `joinLeftAVL` สมมาตรกับ `joinRightAVL`

บทตั้ง 7.2. *สำหรับต้นไม้ AVL สองต้น T_l และ T_r อัลกอริทึม* join *ของ AVL ทำงานได้ถูกต้อง ทำงานด้วยงาน $O(|h(T_l) - h(T_r)|)$ และคืนต้นไม้ที่สอดคล้องกับค่าคงตัว AVL ซึ่งมีความสูงไม่เกิน $1 + \max(h(T_l), h(T_r))$*



รูปที่ 22: **ตัวอย่างของ *join* บนต้นไม้ AVL** – ตัวอย่างของ *join* บนต้นไม้ AVL ($h(T_l) > h(T_r) + 1$) ขั้นแรกเราไล่ตามสันขวาของ T_l จนพบต้นไม้ย่อยที่มีความสูงไม่เกิน $h(T_r) + 1$ (คือ T_2 ที่มีรากที่ c) จากนั้นสร้าง $node(c, k, T_r)$ ใหม่แทนที่ c (ขั้นที่ 1) หาก $h(T_1) = h$ และ $h(T_2) = h + 1$ โหนด p จะไม่สอดคล้องกับค่าคงตัว AVL อีกต่อไป การหมุนคู่ (ขั้นที่ 2) จะคืนทั้งความสมดุลและความสูงเดิมของมัน

พิสูจน์. เนื่องจากอัลกอริทึมเยี่ยมเฉพาะโหนดบนวิถีจากรากไปยัง c และต้องหมุนไม่เกินสองครั้ง (บทตั้ง 7.1) อัลกอริทึมจึงใช้งานเป็นสัดส่วนกับความยาววิถี ความยาววิถีไม่เกินผลต่างของความสูงของต้นไม้ทั้งสอง เพราะความสูงของแต่ละโหนดที่ติดกันตามสันขวาของ T_l ต่างกันอย่างน้อยหนึ่ง เมื่อรวมกับกรณี $h(T_r) > h(T_l) + 1$ ซึ่งสมมาตรกัน จะได้ขอบเขตงานตามที่กล่าว ต้นไม้ผลลัพธ์สอดคล้องกับค่าคงตัว AVL เพราะใช้การหมุนเพื่อคืนค่าคงตัว ความสูงของโหนดใด ๆ เพิ่มได้ไม่เกินหนึ่ง ดังนั้นความสูงของต้นไม้ทั้งต้นจึงเพิ่มได้ไม่เกินหนึ่ง □

7.2.2 ต้นไม้แดง–ดำ

Tarjan อธิบายวิธีสร้างฟังก์ชัน *join* สำหรับต้นไม้แดง–ดำ [27] ในที่นี้เราอธิบายรูปแบบที่ไม่สมมติว่ารากเป็นสีดำ (เพื่อจำกัดการเพิ่มขึ้นของอันดับจาก *union*) รหัสเทียมแสดงในรูปที่ 23 เราเก็บความสูงดำ $\hat{h}(\cdot)$ ไว้ที่ทุกโหนด นอกจากนี้ เรานิยาม*โหนดเพิ่ม-2* (increase-2 node) ว่าเป็นโหนดสีดำที่ลูกทั้งสองเป็นสีดำด้วย ซึ่งหมายความว่าโหนดนี้เพิ่มอันดับของลูกขึ้น 2 ในอัลกอริทึม กรณีแรกคือเมื่อ $\hat{h}(T_r) = \hat{h}(T_l)$ จากนั้นหากโหนดข้อมูลเข้าเป็นโหนดเพิ่ม-2 เราใช้มันเป็นโหนดสีดำและต่อต้นไม้ข้อมูล

เข้าทั้งสองเข้าด้วยกันโดยตรง ซึ่งเพิ่มอันดับของข้อมูลเข้าไม่เกิน 2 มิฉะนั้น หากทั้ง $root(T_r)$ และ $root(T_l)$ เป็นสีดำ เราสร้าง $node(T_l, e, T_r)$ สีแดง เมื่อ $root(T_r)$ หรือ $root(T_l)$ เป็นสีแดง เราสร้าง $node(T_l, e, T_r)$ สีดำ

กรณีที่สองคือเมื่อ $\hat{h}(T_r) < \hat{h}(T_l) = \hat{h}$ (กรณีที่สามสมมาตรกัน) เช่นเดียวกับต้นไม้ AVL *join* ไล่ตามสันขวาของ T_l จนพบโหนดสีดำ c ซึ่ง $\hat{h}(c) = \hat{h}(T_r)$ จากนั้นสร้าง $node(c, k, T_r)$ สีแดงใหม่แทนที่ c เนื่องจากทั้ง c และ T_r มีความสูงเท่ากัน ค่าคงตัวเดียวที่อาจถูกละเมิดคือกฎสีแดงที่รากของ T_r โหนดใหม่ และโหนดแม่ของมัน ซึ่งอาจเป็นสีแดงทั้งหมด ในกรณีแย่ที่สุดเราอาจมีโหนดสีแดงสามโหนดเรียงกัน สิ่งนี้แก้ได้ด้วยการหมุนซ้ายเดี่ยว: หากโหนดสีดำ v มี $rc(v)$ และ $rc(rc(v))$ เป็นสีแดงทั้งคู่ เราเปลี่ยน $rc(rc(v))$ เป็นสีดำและหมุนซ้ายเดี่ยวที่ v ทำให้โหนดใหม่เป็นสีดำ แล้วหมุนซ้ายเดี่ยวที่ v การปรับปรุงนี้แสดงในรูปที่ 24 อย่างไรก็ตาม การหมุนอาจละเมิดกฎสีแดงระหว่างรากของต้นไม้ที่ถูกหมุนกับโหนดแม่ของมันอีกครั้ง ซึ่งต้องหมุนอีกครั้ง เห็นได้ชัดว่าปัญหาแดงสามตัวถูกแก้ไปแล้วหลังการหมุนครั้งแรก ดังนั้น ยกเว้นระดับล่างสุด ปัญหาแดงสามตัวจะไม่เกิดขึ้น ปัญหาแดงสองตัวอาจลุกลามขึ้นไปถึงรากของ T_l หากรากเดิมของ T_l เป็นสีแดง อัลกอริทึมอาจจบด้วยรากสีแดงที่มีลูกสีแดง ซึ่งในกรณีนี้รากจะถูกเปลี่ยนเป็นสีดำ ทำให้อันดับของ T_l เปลี่ยนจาก $2\hat{h} - 1$ เป็น $2\hat{h}$ หากรากเดิมของ T_l เป็นสีดำ อัลกอริทึมอาจจบด้วยรากสีแดงที่มีลูกสีดำสองตัว ทำให้อันดับของ T_l เปลี่ยนจาก $2\hat{h} - 2$ เป็น $2\hat{h} - 1$ ในทั้งสองกรณี อันดับของต้นไม้ผลลัพธ์มีค่าไม่เกิน $1 + r(T_l)$

```
1  joinRightRB(T_l, k, T_r) {
2    if (r(T_l) = ⌊r(T_r)/2⌋ × 2) then return node(T_l, ⟨k, red⟩, T_r); else {
3      (L', ⟨k', c'⟩, R')=expose(T_l);
4      T' = node(L', ⟨k', c'⟩, joinRightRB(R', k, T_r));
5      if (c'=black) and (color(rc(T')) = color(rc(rc(T')))=red) then {
6        set rc(rc(T')) as black;
7        return rotateLeft(T');
8      } else return T'; }}
9  joinRB(T_l, k, T_r) {
10   if T_l has a larger black height then {
11     T' =joinRightRB(T_l, k, T_r);
12     if (color(T')=red) and (color(rc(T'))=red) then return node(lc(T'), ⟨k(T'), black⟩, rc(T'));
13     else return T';
14   } else if T_r has a larger black height then {
15     T' =joinLeftRB(T_l, k, T_r);
16     if (color(T')=red) and (color(lc(T'))=red) then return node(lc(T'), ⟨k(T'), black⟩, rc(T'));
17     else return T';
18   } else {
19     if (k is a increase-2 node) then
20       return node(T_l, ⟨k, black⟩, T_r);
21     else if (color(T_l)=black) and (color(T_r)=black)
22       return node(T_l, ⟨k, red⟩, T_r);
23     else return node(T_l, ⟨k, black⟩, T_r); }
24 }
```

รูปที่ 23: **อัลกอริทึม *join* บนต้นไม้แดง–ดำ** – อัลกอริทึม *join* บนต้นไม้แดง–ดำ `joinLeftRB` สมมาตรกับ `joinRightRB`

เราสังเกตว่าอันดับของผลลัพธ์จะเพิ่มขึ้นจากอันดับที่มากกว่าของต้นไม้ข้อมูลเข้าได้ 2 ก็ต่อเมื่อตัวกลางเป็นโหนดเพิ่ม-2 และต้นไม้ข้อมูลเข้าทั้งสองสมดุลกันและมีรากสีดำทั้งคู่ โดยทั่วไปเราไม่จำเป็นต้องจัดการกับโหนดเพิ่ม-2 เป็นการเฉพาะเพื่อให้ได้อัลกอริทึม *join* ที่ถูกต้อง เรานิยามโหนดเพิ่ม-2 เพื่อจุดประสงค์ในการจำกัดต้นทุนของอัลกอริทึมบนพื้นฐาน *join* บางตัว

บทตั้ง 7.3. *สำหรับต้นไม้ RB สองต้น T_l และ T_r อัลกอริทึม* join *ของ RB ทำงานได้ถูกต้อง ทำงานด้วยงาน $O(|\hat{h}(T_l) - \hat{h}(T_r)|)$ และคืนต้นไม้ที่สอดคล้องกับค่าคงตัวแดง–ดำ และมีความสูงดำไม่เกิน $1 + \max(\hat{h}(T_l), \hat{h}(T_r))$*

พิสูจน์. กรณีฐานที่ $h(T_l) = h(T_r)$ ตรงไปตรงมา ด้วยความสมมาตร ในที่นี้เราพิสูจน์เฉพาะกรณี $h(T_l) > h(T_r)$ เราพิสูจน์ข้อความโดยอุปนัย

ก่อนอื่นเราแสดงความถูกต้อง ดังแสดงในรูปที่ 24 หลังจากต่อ T_r เข้ากับ T_l หาก p เป็นสีดำ การปรับสมดุลก็เสร็จแล้ว ความสูงของแต่ละโหนดไม่เปลี่ยนแปลง ดังนั้นต้นไม้ RB ยังคงถูกต้อง มิฉะนั้น p เป็นสีแดง โหนดแม่ของ p คือ g จะต้องเป็นสีดำ โดยการหมุนซ้ายที่ p และ g เราจะได้ต้นไม้ RB ที่สมดุลซึ่งมีรากที่ p ยกเว้นว่าราก p เป็นสีแดง หาก p เป็นรากของต้นไม้ทั้งต้น เราเปลี่ยนสีของ p เป็นสีดำ และความสูงของต้นไม้ทั้งต้นเพิ่มขึ้น 1 ต้นไม้ RB ยังคงถูกต้อง มิฉะนั้น หากโหนดแม่ปัจจุบันของ p (เดิมคือโหนดแม่ของ g) เป็นสีดำ การปรับสมดุลก็เสร็จที่นี่ มิฉะนั้นต้องปรับสมดุลในทำนองเดียวกันที่ p และโหนดแม่ปัจจุบันของมัน ดังนั้นในที่สุดเราจะพบว่าโหนดปัจจุบันถูกต้อง (โหนดสีแดงปัจจุบันมีโหนดแม่สีดำ) หรือไปถึงราก แล้วเปลี่ยนสีของรากเป็นสีดำ ดังนั้นเมื่อเราหยุด เราจะได้ต้นไม้ RB ที่ถูกต้องเสมอ

เนื่องจากอัลกอริทึมเยี่ยมเฉพาะโหนดบนวิถีจากรากไปยัง c และต้องหมุนไม่เกินหนึ่งครั้งต่อโหนดบนวิถี งานโดยรวมของอัลกอริทึมจึงเป็นสัดส่วนกับความลึกของ c ใน T_l[แก้ 34] ซึ่งในทางกลับกันไม่เกินสองเท่าของผลต่างของความสูงดำของต้นไม้ทั้งสอง เพราะความสูงดำลดลงอย่างน้อยทุก ๆ สองโหนดตามวิถี ซึ่งให้ขอบเขตงานตามที่กล่าว

สำหรับอันดับ สังเกตว่าตลอดอัลกอริทึม ก่อนไปถึงราก กฎสีดำไม่เคยถูกละเมิด (หรือถูกแก้ทันที) และการละเมิดเดียวที่เกิดขึ้นคือกฎสีแดง หากต้นไม้ข้อมูลเข้าทั้งสองสมดุลกันตั้งแต่ต้น อันดับจะเพิ่มขึ้นไม่เกิน 2 กรณีเดียวที่อันดับเพิ่มขึ้น 2 คือเมื่อ k มาจากโหนดเพิ่ม-2 และทั้ง $\mathit{root}(T_r)$ และ $\mathit{root}(T_l)$ เป็นสีดำ

หากต้นไม้ข้อมูลเข้าทั้งสองไม่สมดุลกัน ความสูงดำของรากจะไม่เปลี่ยนแปลงก่อนที่อัลกอริทึมจะไปถึงราก (ขั้นที่ 3 ในรูปที่ 24) จากนั้นมีสามกรณี:

1. การหมุนไม่ลุกลามไปถึงราก และดังนั้นอันดับของต้นไม้ยังคงเป็น $\max(\hat{h}(T_l), \hat{h}(T_r))$
2. (ขั้นที่ 3 กรณีที่ 1) สีของรากเดิมเป็นสีแดง และดังนั้นปัญหาแดงสองตัวเกิดขึ้นที่รากและลูกทางขวาของมัน ในกรณีนี้รากถูกระบายเป็นสีดำ ความสูงดำของต้นไม้เพิ่มขึ้น 1 แต่เนื่องจากรากเดิมเป็นสีแดง อันดับจึงเพิ่มขึ้นเพียง 1
3. (ขั้นที่ 3 กรณีที่ 2[แก้ 35]) สีของรากเดิมเป็นสีดำ แต่ปัญหาแดงสองตัวเกิดขึ้นที่ลูกและหลานของราก ในกรณีนี้มีการหมุนอีกครั้งดังแสดงในรูปที่ 24 ความสูงดำคงเดิม แต่รากเปลี่ยนจากสีดำเป็นสีแดง ทำให้อันดับเพิ่มขึ้น 1

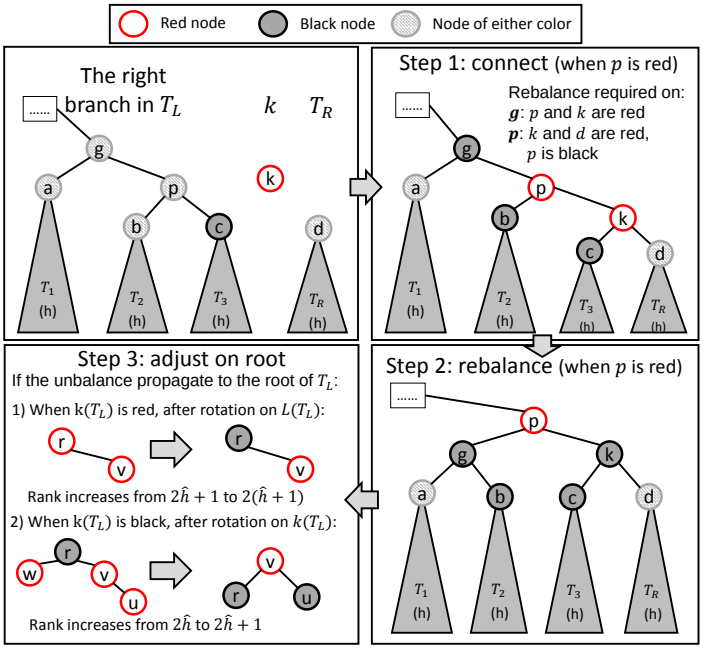
□

7.2.3 ต้นไม้สมดุลน้ำหนัก

สำหรับต้นไม้ WB $r(T) = \log_2(w(T)) - 1$ เราเก็บน้ำหนักของแต่ละต้นไม้ย่อยไว้ที่ทุกโหนด อัลกอริทึมสำหรับ join ต้นไม้สมดุลน้ำหนักสองต้นคล้ายกับของต้นไม้ AVL และต้นไม้ RB รหัสเทียมแสดงในรูปที่ 25 ฟังก์ชัน `like` ในโค้ดคืนค่าจริงหากขนาดของต้นไม้ข้อมูลเข้าทั้งสองสมดุลกันตามตัวประกอบ α และคืนค่าเท็จในกรณีอื่น หาก T_l และ T_r มีน้ำหนักใกล้เคียงกัน อัลกอริทึมจะคืน $\mathit{node}(T_l, e, T_r)$ ใหม่ สมมติว่า $|T_r| \leq |T_l|$ อัลกอริทึมไล่ตามกิ่งขวาของ T_l จนถึงโหนด c ที่มีน้ำหนักใกล้เคียงกับ T_r จากนั้นสร้าง $\mathit{node}(c, k, T_r)$[แก้ 36] ใหม่แทนที่ c โหนดใหม่จะมีน้ำหนักมากกว่า c และดังนั้นอาจทำให้น้ำหนักของบรรพบุรุษของ c ไม่สมดุล สิ่งนี้แก้ได้ด้วยการหมุนเดี่ยวหรือหมุนคู่ (ดังแสดงในรูปที่ 26) ที่แต่ละโหนด โดยสมมติว่า α อยู่ภายในขอบเขตที่ให้ไว้ในหัวข้อ 7.1

บทตั้ง 7.4. *สำหรับต้นไม้สมดุลน้ำหนัก α สองต้น T_l และ T_r และ $\alpha \leq 1 - \frac{1}{\sqrt{2}} \approx 0.29$ อัลกอริทึม* join *แบบสมดุลน้ำหนักทำงานได้ถูกต้อง ทำงานด้วยงาน $O(|\log(w(T_l)/w(T_r))|)$ และคืนต้นไม้ที่สอดคล้องกับค่าคงตัวสมดุลน้ำหนัก α*

การพิสูจน์บทตั้งนี้หาได้ใน [8][แก้ 37] สังเกตว่าขอบเขตบนนี้เหมือนกับข้อจำกัดของ α ที่ให้ต้นไม้สมดุลน้ำหนักที่ถูกต้องเมื่อแทรกโหนดเดี่ยว จากนั้นเราสามารถอนุมานได้ว่าเมื่อกระบวนการปรับสมดุลไปถึงราก ต้นไม้สมดุลน้ำหนักใหม่จะถูกต้อง การพิสูจน์

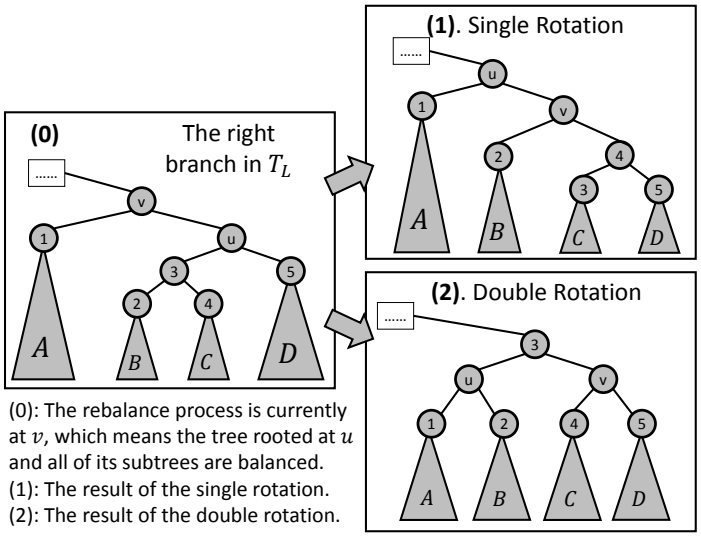


รูปที่ 24: **ตัวอย่างของ *join* บนต้นไม้แดง–ดำ** – ตัวอย่างของ *join* บนต้นไม้แดง–ดำ ($\hat{h} = \hat{h}(T_l) > \hat{h}(T_r)$) เราไล่ตามสันขวาของ T_l จนพบโหนดสีดำที่มีความสูงดำเท่ากับ T_r (คือ c) จากนั้นสร้าง $node(c, k, T_r)$ สีแดงใหม่แทนที่ c (ขั้นที่ 1) ค่าคงตัวเดียวที่อาจถูกละเมิดคือเมื่อโหนดแม่เดิมของ c คือ p หรือราก d ของ T_r เป็นสีแดง หากเป็นเช่นนั้น จะมีการหมุนซ้ายที่โหนดสีดำบางโหนด ขั้นที่ 2 แสดงการปรับสมดุลเมื่อ p เป็นสีแดง ความสูงดำของต้นไม้ย่อยที่ถูกหมุน (ตอนนี้มีรากที่ p) เท่ากับก่อนหน้า ($h+1$) แต่โหนดแม่ของ p อาจเป็นสีแดง ซึ่งต้องหมุนอีกครั้ง หากการละเมิดกฎสีแดงลุกลามไปถึงราก รากจะถูกระบายเป็นสีแดง หรือถูกหมุนซ้าย (ขั้นที่ 3)

```
1  joinRightWB(T_l, k, T_r) {
2    (l, k', c)=expose(T_l);
3    if (balance(|T_l|, |T_r|) then return node(T_l, k, T_r)); else {
4      T' = joinRightWB(c, k, T_r);
5      (l_1, k_1, r_1) = expose(T');
6      if like(|l|, |T'|) then return node(l, k', T');
7      else if (like(|l|, |l_1|)) and (like(|l| + |l_1|, r_1)) then return rotateLeft(node(l, k', T'));
8      else return rotateLeft(node(l, k',rotateRight(T'))); }}

9  joinWB(T_l, k, T_r) {
10   if heavy(T_l, T_r) then return joinRightWB(T_l, k, T_r);
11   else if heavy(T_r, T_l) then return joinLeftWB(T_l, k, T_r);
12   else return node(T_l, k, T_r); }
```

รูปที่ 25: **อัลกอริทึม *join* บนต้นไม้สมดุลน้ำหนัก** – `joinLeftWB` สมมาตรกับ `joinRightWB`



รูปที่ 26: **ภาพแสดงการหมุนเดี่ยวและการหมุนคู่ที่อาจจำเป็นในการปรับสมดุลต้นไม้สมดุลน้ำหนัก** – ในรูป ต้นไม้ย่อยที่มีรากที่ u หนักขึ้นเนื่องจากการ join เข้าใน T_l และโหนดแม่ของมัน v ตอนนี้ละเมิดค่าคงตัวของสมดุล

มีแนวคิดคล้ายกับการพิสูจน์ที่กล่าวไว้ใน [19][แก้ 37] ซึ่งพิสูจน์ว่าเมื่อ $\frac{2}{11} \leq \alpha \leq 1 - \frac{1}{\sqrt{2}}$ การหมุนจะปรับสมดุลต้นไม้หลังการแทรกเพียงครั้งเดียว ที่จริงแล้ว ในอัลกอริทึม *join* ต้นไม้ย่อยที่ “ถูกแทรก” จะต้องอยู่ตามสันซ้ายหรือสันขวา ซึ่งทำให้การวิเคราะห์ง่ายขึ้นจริง ๆ

```
1  joinTreap(T_l, k, T_r) {
2    if prior(k, k_1) and prior(k, k_2) then return node(T_l, k, T_r) else {
3      (l_1, k_1, r_1)=expose(T_l);
4      (l_2, k_2, r_2)=expose(T_r);
5      if prior(k_1, k_2) then return node(l_1, k_1, joinTreap(r_1, k, T_r));
6      else return node(joinTreap(T_l, k, l_2), k_2, r_2); }}
```

รูปที่ 27: **อัลกอริทึม *join* บน treap** – $\texttt{prior}(k_1, k_2)$ ตัดสินว่าโหนด k_1 มีลำดับความสำคัญสูงกว่า k_2 หรือไม่

7.2.4 Treaps

อัลกอริทึม *join* ของ treap (ดังในรูปที่ 27) เลือกคีย์ที่มีลำดับความสำคัญสูงสุดในบรรดา k, $k(T_l)$ และ $k(T_r)$ เป็นรากก่อน หาก k เป็นราก เราสามารถคืน $node(T_l, k, T_r)$ ได้ มิฉะนั้น โดยไม่เสียนัยทั่วไป (WLOG) สมมติว่า $k(T_l)$ มีลำดับความสำคัญสูงกว่า ในกรณีนี้ $k(T_l)$ จะเป็นรากของผลลัพธ์ $lc(T_l)$ จะเป็นต้นไม้ซ้าย และ $rc(T_l)$, k และ T_r จะประกอบกันเป็นต้นไม้ขวา ดังนั้น *join* จึงเรียกตัวเองแบบเวียนเกิดบน $rc(T_l)$, k และ T_r และใช้ผลลัพธ์เป็นลูกทางขวาของ $k(T_l)$ เมื่อ $k(T_r)$ มีลำดับความสำคัญสูงกว่า กรณีจะสมมาตรกัน ต้นทุนของ *join* จึงเท่ากับความลึกของคีย์ k ในต้นไม้ผลลัพธ์ (แต่ละการเรียกเวียนเกิดดันมันลงไปหนึ่งระดับ) ใน treap รูปร่างของต้นไม้ผลลัพธ์ และดังนั้นความลึกของ k ขึ้นอยู่กับคีย์และลำดับความสำคัญเท่านั้น ไม่ขึ้นกับประวัติ โดยเฉพาะ หากคีย์มีลำดับความสำคัญสูงสุดเป็นลำดับที่ t^{th} ในบรรดาคีย์ทั้งหมด ความลึกคาดหมายของมันใน treap คือ $O(\log t)$ (และ w.h.p. ด้วย) ตัวอย่างเช่น หากมันมีลำดับความสำคัญสูงสุด มันจะยังคงอยู่ที่ราก

บทตั้ง 7.5. *สำหรับ treap สองต้น T_l และ T_r หากลำดับความสำคัญของ k สูงเป็นลำดับที่ t ในบรรดาคีย์ทั้งหมดใน $T_l \cup \{k\} \cup T_r$ อัลกอริทึม* join *ของ treap ทำงานได้ถูกต้อง ทำงานด้วยงาน $O(\log t + 1)$ ในค่าคาดหมายและ w.h.p. และคืนต้นไม้ที่*

สอดคล้องกับค่าคงตัวของ treap

ฟังก์ชัน	***งาน (Work)***	***สแปน (Span)***
insert, delete, update, find, first, last, range, split, join2, previous, next, rank, select, up_to, down_to	$O(\log n)$	$O(\log n)$
union, intersection, difference	$O\left(m \log\left(\frac{n}{m}+1\right)\right)$	$O(\log n \log m)$
map, reduce, map_reduce, to_array	$O(n)$	$O(\log n)$
build, filter	$O(n)$	$O(\log^2 n)$

ตารางที่ 3: **อัลกอริทึมหลักบนพื้นฐาน *join* และต้นทุนเชิงเส้นกำกับ** – ต้นทุนให้ไว้ภายใต้สมมติฐานว่าฟังก์ชันพารามิเตอร์ทั้งหมดใช้เวลาคงที่ในการคืนค่า สำหรับฟังก์ชันที่มีต้นไม้ข้อมูลเข้าสองต้น (*union*, *intersection* และ *difference*) n คือขนาดของข้อมูลเข้าที่ใหญ่กว่า และ m คือขนาดของข้อมูลเข้าที่เล็กกว่า

7.3 อัลกอริทึมที่ใช้ *join*

Split

```
1  split(T, k) {
2    if T = ∅ then
3      return (∅,false,∅);
4    (L, m, R) = expose(T);
5    if k = m then return (L,true,R);
6    if k < m then {
7      (T_L, b, T_R) = split(L, k);
8      return (T_L,b,join(T_R, m, R)); }
9    (T_L, b, T_R) = split(R, k);
10   return (join(L, m, T_L), b, T_R); } }
```

join2

```
1  split_last(T) { // split_first is symmetric
2    (L, k, R) = expose(T);
3    if R = ∅ then return(L, k);
4    (T', k') = split_last(R);
5    return (join(L, k, T'),k'); }
6  join2(T_l,T_r) {
7    if T_l = ∅ then return T_r;
8    (T', k) = split_last(T_l);
9    return join(T', k, T_r);
10 }
```

รูปที่ 28: **อัลกอริทึม *split* และ *join2*** – ทั้งสองไม่ขึ้นกับรูปแบบการรักษาสมดุล

ฟังก์ชัน *join* ในฐานะรูทีนย่อย ได้ถูกใช้และศึกษาโดยนักวิจัยและโปรแกรมเมอร์จำนวนมากเพื่อสร้างการดำเนินการบนเซตที่ทั่วไปกว่า ในหัวข้อนี้ เราอธิบายอัลกอริทึมสำหรับฟังก์ชันต่าง ๆ ที่ใช้เพียง *join* อัลกอริทึมเหล่านี้ใช้ได้ทั่วไปข้ามรูปแบบการรักษาสมดุล รหัสเทียมสำหรับอัลกอริทึมในหัวข้อนี้แสดงในรูปที่ 30 นอกจาก *join* แล้ว การเข้าถึงต้นไม้เพียงอย่างเดียวที่เราใช้คือผ่าน *expose* ซึ่งอ่านเฉพาะราก การดำเนินการบนเซตหลัก ได้แก่ *union*, *intersection* และ *difference* มีงานที่เหมาะสมที่สุด (หรือที่เรียกว่ามีประสิทธิภาพ) รหัสเทียมสำหรับอัลกอริทึมทั้งหมดที่นำเสนอในหัวข้อนี้แสดงในรูปที่ 31

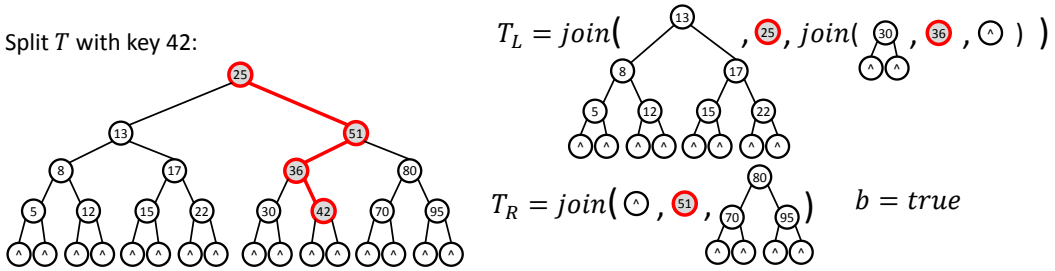
7.3.1 ฟังก์ชันช่วยสองตัว: *split* และ *join2*

เราเริ่มด้วยการนำเสนอฟังก์ชันช่วย (helper function) สองตัวคือ *split* และ *join2* สำหรับ BST T และคีย์ k *split*(T, k) คืนสามสิ่ง (T_l, b, T_r) โดยที่ T_l (T_r) เป็นต้นไม้ที่มีคีย์ทั้งหมดใน T ที่น้อยกว่า (มากกว่า) k และ b เป็นแฟล็กที่บ่งบอกว่า $k \in T$ หรือไม่ *join2*(T_l, T_r) คืนต้นไม้ทวิภาคที่มีค่าตามลำดับเป็นการต่อกันของค่าตามลำดับของต้นไม้ทวิภาค T_l และ T_r (เหมือน *join* แต่ไม่มีคีย์ตรงกลาง) สำหรับ BST คีย์ทั้งหมดใน T_l ต้องน้อยกว่าคีย์ใน T_r

แม้ว่าทั้งสองจะเป็นแบบลำดับ ฟังก์ชันทั้งสองนี้ ร่วมกับฟังก์ชัน *join* มีความจำเป็นในการช่วยให้อัลกอริทึมอื่นได้ความขนานที่ดี โดยสัญชาตญาณ เมื่อประมวลผลต้นไม้แบบขนาน เราเรียกเวียนเกิดบนส่วนประกอบย่อยสองส่วนของต้นไม้แบบขนานโดย

แยกต้นไม้ด้วยคีย์บางตัว ในหลายกรณี คีย์ที่ใช้แยกก็คือราก ซึ่งหมายถึงการใช้ต้นไม้ย่อยสองต้นของโครงสร้างต้นไม้ทวิภาคตามธรรมชาติโดยตรง หลังจากการเวียนเกิดคืนค่า เรารวมผลลัพธ์ของส่วนซ้ายและขวา โดยมีหรือไม่มีคีย์ตรงกลาง ด้วย *join* หรือ *join2* เนื่องจากความสมดุลของต้นไม้ กรอบงานนี้มักให้ความขนานสูงพร้อมสแปนที่ตื้น (เช่น พหุลอการิทึม)

Split. ดังที่กล่าวข้างต้น *split*(T, k) แยกต้นไม้ T ด้วยคีย์ k เป็น T_l และ T_r พร้อมบิต b ที่บ่งบอกว่า $k \in T$ หรือไม่ โดยสัญชาตญาณ อัลกอริทึม *split* ค้นหา k ใน T ก่อน โดยแยกต้นไม้ตามวิถีออกเป็นสามส่วน: คีย์ทางซ้ายของวิถี k เอง (ถ้ามี) และคีย์ทางขวา จากนั้นโดยการใช้ *join* อัลกอริทึมผสานต้นไม้ย่อยทั้งหมดทางด้านซ้าย (โดยใช้คีย์บนวิถีเป็นโหนดกลาง) จากล่างขึ้นบนเพื่อสร้าง T_l และผสานส่วนทางขวาเพื่อสร้าง T_r เมื่อเขียนโค้ดแบบเวียนเกิด อัลกอริทึมนี้ตัดสินก่อนว่า k อยู่ในต้นไม้ย่อยซ้าย (ขวา) หรือเป็นรากพอดี หากเป็นราก อัลกอริทึมจะคืนต้นไม้ย่อยซ้ายและขวาเป็นต้นไม้ผลลัพธ์สองต้นและ *true* เป็นบิต b ได้ทันที มิฉะนั้น โดยไม่เสียนัยทั่วไป สมมติว่า k อยู่ในต้นไม้ย่อยซ้าย อัลกอริทึม *split* ต้นไม้ย่อยซ้ายต่อเป็น T_L และ T_R พร้อมบิตผลลัพธ์ b' จากนั้นบิตผลลัพธ์ $b = b'$, T_l ในผลลัพธ์สุดท้ายจะเป็น T_L และ T_r หมายถึงการ *join* T_R กับต้นไม้ย่อยขวาเดิมด้วยรากเดิม รูปที่ 29 แสดงตัวอย่าง



รูปที่ 29: **ตัวอย่างของ *split* ใน BST ด้วยคีย์ 42** – ขั้นแรกเราค้นหา 42 ในต้นไม้และแยกต้นไม้ตามวิถีการค้นหา จากนั้นใช้ *join* เพื่อรวมต้นไม้ทางซ้ายและทางขวาตามลำดับ จากล่างขึ้นบน

ต้นทุนของอัลกอริทึมเป็นสัดส่วนกับอันดับของต้นไม้ ดังที่เราสรุปและพิสูจน์ในทฤษฎีบทต่อไปนี้

ทฤษฎีบท 7.1. *งานของ* split(T, k) *คือ* $O(h(T))$ *สำหรับต้นไม้ AVL, RB, WB และ treap*

ร่างการพิสูจน์. เราพิจารณาเฉพาะงานของการ join ต้นไม้ย่อยทั้งหมดทางด้านซ้าย อีกด้านสมมาตรกัน สมมติว่าเรามีต้นไม้ย่อย l ต้นทางด้านซ้าย แทนจากล่างขึ้นบนเป็น $T_1, T_2, \ldots T_l$ เรา join T_1 และ T_2 ต่อเนื่องกันได้ T_2' จากนั้น join T_2' กับ T_3 ได้ T_3' และต่อไปเรื่อย ๆ จนกว่าต้นไม้ทั้งหมดจะถูกผสาน งานโดยรวมของ *split* คือผลรวมของต้นทุนของฟังก์ชัน *join* $l - 1$ ครั้ง

ตอนนี้เราใช้ต้นไม้ AVL เป็นตัวอย่างเพื่อแสดงการพิสูจน์ จำไว้ว่าแต่ละ *join* ใช้เวลา $O(|h(T_{i+1}) - h(T_i')|)$ และเพิ่มความสูงของ T_{i+1} ไม่เกิน 1 นอกจากนี้ $h(T_i')$ ได้จากการ join T_i และ T_{i-1}' เมื่อพิจารณาว่า T_i เป็นต้นไม้ย่อยในพี่น้อง (sibling) ของ T_{i+1} ดังนั้น $h(T_i')$ จึงไม่เกิน $h(T_{i+1}) + 2$ ความซับซ้อนโดยรวมคือ $\sum_{i=1}^{l} |h(T_{i+1}) - h(T_i')| \leq \sum_{i=1}^{l} h(T_{i+1}) - h(T_i') + 2 = O(h(T))$

สำหรับต้นไม้ RB และ WB การพิสูจน์คล้ายกับการพิสูจน์สำหรับต้นไม้ AVL ข้างต้น เพียงเปลี่ยนต้นทุนของ *join* ให้อิงกับผลต่างของความสูงดำหรือลอการิทึมของน้ำหนัก แทนความสูง

สำหรับ treap แต่ละ *join* ใช้คีย์ที่มีลำดับความสำคัญสูงสุด เนื่องจากคีย์อยู่ในระดับที่สูงกว่าเสมอ ดังนั้นโดยบทตั้ง 7.5 ความซับซ้อนของแต่ละ *join* คือ $O(1)$ และงานของ split มีค่าไม่เกิน $O(h(T))$ □

Join2. ดังที่กล่าวข้างต้น ฟังก์ชัน *join2* นิยามคล้ายกับ *join* แต่ไม่มีรายการตรงกลาง อัลกอริทึม *join2* เลือกต้นไม้ข้อมูลเข้าต้นใดต้นหนึ่งก่อน แล้วดึงสมาชิกตัวสุดท้าย (หากเป็น T_l) หรือตัวแรก (หากเป็น T_r) k ออกมา ทั้งสองกรณีมีต้นทุนเชิงเส้นกำกับเท่ากัน กระบวนการดึงออกคล้ายกับอัลกอริทึม *split* จากนั้นอัลกอริทึมใช้ k เป็นตัวกลางเพื่อ *join* ต้นไม้ทั้งสอง ในโค้ดที่แสดงในรูป

Union

```
union(T1,T2) {
  if T1 = ∅ then return T2;
  if T2 = ∅ then return T1;
  (L2,k2,R2) = expose(T2);
  (L1,b,R1) = split(T1,k2);
  Tl = union(L1,L2) ||
       Tr = union(R1,R2);
  return join(Tl,k2,Tr);
}
```

Intersection

```
intersect(T1,T2) {
  if T1 = ∅ then return ∅;
  if T2 = ∅ then return ∅;
  (L2,k2,R2) = expose(T2);
  (L1,b,R1) = split(T1,k2);
  Tl = intersect(L1,L2) ||
    Tr = intersect(R1,R2);
  if b then return join(Tl,k2,Tr);
  else return join2(Tl,Tr); }
```

Difference

```
difference(T1,T2) {
  if T1 = ∅ then ∅;
  if T2 = ∅ then T1;
  (L2,k2,R2) = expose(T2);
  (L1,b,R1) = split(T1,k2);
  Tl = difference(L1,L2) ||
    Tr = difference(R1,R2);
  return join2(Tl,Tr);
}
```

รูปที่ 30: **อัลกอริทึมบนพื้นฐาน *join* สำหรับการดำเนินการระหว่างเซตกับเซต** – ทั้งหมดไม่ขึ้นกับรูปแบบการรักษาสมดุล ไวยากรณ์ $S_1 \| S_2$ หมายความว่าคำสั่งสองคำสั่ง S_1 และ S_2 สามารถรันแบบขนานได้บนพื้นฐานของความขนานแบบ fork-join ใดก็ได้

ที่ 28 อัลกอริทึม *split_last* หาสมาชิกตัวสุดท้าย k ก่อน (โดยไล่ตามสันขวา) ใน T_l และระหว่างทางกลับไปยังราก จะ *join* ต้นไม้ย่อยตามวิถี เราแทนผลลัพธ์ของการนำ k ออกจาก T_l[แก้ 38] ด้วย T' จากนั้น $join(T', k, T_r)$ คือผลลัพธ์ของ *join2* ต่างจาก *join* งานของ *join2* เป็นสัดส่วนกับอันดับของต้นไม้ทั้งสอง เพราะทั้ง *split* และ *join* ใช้งานไม่เกินลอการิทึม

ทฤษฎีบท 7.2. *งานของ* $T = \text{join2}(T_l, T_r)$ *คือ* $O(r(T_l) + r(T_r))$ *สำหรับต้นไม้ที่ทำ join ได้ทั้งหมด*

ขอบเขตต้นทุนเป็นจริงเพราะทั้ง *split_last* และ *join* ใช้งานไม่เกินอันดับของต้นไม้ที่ใหญ่กว่าในเชิงเส้นกำกับ

7.4 ฟังก์ชันระหว่างเซตกับเซตที่ใช้ *join*

ในหัวข้อนี้ เราจะนำเสนออัลกอริทึมบนพื้นฐาน *join* สำหรับฟังก์ชันระหว่างเซตกับเซต (set-set functions) ได้แก่ *union*, *intersection* และ *difference* การดำเนินการระหว่างเซตกับเซตอื่น ๆ อีกมาก เช่น ผลต่างสมมาตร (symmetric difference) สามารถสร้างได้จากการรวมกันของ *union*, *intersection* และ *difference* โดยไม่มีงานเชิงเส้นกำกับเพิ่มเติม เราจะเริ่มด้วยการนำเสนอภูมิหลังบางส่วนของอัลกอริทึมเหล่านี้ แล้วอธิบายรายละเอียดเกี่ยวกับอัลกอริทึมบนพื้นฐาน *join* สุดท้าย เราแสดงการพิสูจน์ขอบเขตต้นทุนของมัน

ภูมิหลัง (Background). ฟังก์ชันระหว่างเซตกับเซตแบบขนานมีประโยชน์อย่างยิ่งเมื่อใช้เครื่องแบบขนาน เพราะสามารถรองรับการปรับปรุงเป็นกลุ่ม (bulk update) แบบขนาน ดังที่กล่าวไว้ แม้ว่าการรองรับอัลกอริทึมที่มีประสิทธิภาพสำหรับการดำเนินการพื้นฐานบนต้นไม้ เช่น การแทรกและการลบ จะค่อนข้างตรงไปตรงมา แต่การสร้างการดำเนินการเป็นกลุ่มที่มีประสิทธิภาพนั้นท้าทายกว่า โดยเฉพาะเมื่อพิจารณาความขนานและรูปแบบการรักษาสมดุลที่ต่างกัน ตัวอย่างเช่น การรวมเซตที่มีลำดับสองเซตขนาด n และ $m \leq n$ ในรูปแบบอาร์เรย์สองอาร์เรย์จะใช้งาน $O(m+n)$ โดยใช้อัลกอริทึมการผสานมาตรฐานใน merge sort สิ่งนี้ทำให้แม้แต่การแทรกสมาชิกเดียวลงในเซตขนาด n ก็มีต้นทุนเชิงเส้น เพราะแม้ข้อมูลส่วนใหญ่ในข้อมูลเข้าจะยังคงต่อเนื่องกัน อัลกอริทึมก็ยังต้องสแกนและคัดลอกไปยังอาร์เรย์ผลลัพธ์ การสร้างแบบง่ายอีกแบบคือเก็บทั้งสองเซตเป็นต้นไม้ที่สมดุล และแทรกสมาชิกในต้นไม้ที่เล็กกว่าลงในต้นที่ใหญ่กว่า ซึ่งใช้งาน $O(m \log n)$ วิธีนี้แก้ปัญหาการสแกนและคัดลอกที่ซ้ำซ้อน เพราะต้นไม้ย่อยจำนวนมากในต้นไม้ที่ใหญ่กว่ายังคงไม่ถูกแตะต้อง อย่างไรก็ตาม วิธีนี้ใช้เวลา $O(n \log n)$ ในการรวมเซตที่มีลำดับสองเซตที่มีขนาดเท่ากัน ในขณะที่ทำให้เป็น $O(n)$ ได้ง่ายด้วยอาร์เรย์ ปัญหาอยู่ที่อัลกอริทึมไม่ได้ใช้ประโยชน์จากลำดับในต้นไม้ที่เล็กกว่า

ขอบเขตล่างสำหรับอัลกอริทึมแบบเปรียบเทียบสำหรับ *union*, *intersection* และ *difference* สำหรับข้อมูลเข้าขนาด n และ $m \leq n$ และคืนโครงสร้างที่มีลำดับ[2] คือ $\log_2 \binom{m+n}{n} = \Theta\left(m \log\left(\frac{n}{m} + 1\right)\right)$ ($\binom{m+n}{n}$ คือจำนวนวิธีที่เป็นไปได้ที่คีย์ n

[2]โดย “โครงสร้างที่มีลำดับ” เราหมายถึงโครงสร้างข้อมูลใด ๆ ที่สามารถส่งออกสมาชิกตามลำดับที่เรียงแล้วโดยไม่ต้องเปรียบเทียบเพิ่มเติม เช่น อาร์เรย์ที่เรียงแล้ว หรือต้นไม้ค้นหาทวิภาค

ตัวจะสลับแทรกกับคีย์ m ตัว) ขอบเขตนี้น่าสนใจเพราะแสดงว่าการสร้างการแทรกด้วย union หรือการลบด้วย difference มีประสิทธิภาพเชิงเส้นกำกับ (เวลา $O(\log n)$) เช่นเดียวกับการหา union ของสองเซตที่มีขนาดเท่ากัน (เวลา $O(n)$)

Brown และ Tarjan เป็นคนแรกที่ได้ขอบเขตเหล่านี้ในเชิงเส้นกำกับ โดยใช้อัลกอริทึมแบบลำดับบนพื้นฐานต้นไม้แดง–ดำ [11] ต่อมา Adams อธิบายอัลกอริทึมที่สง่างามมากสำหรับ union, intersection และ difference รวมถึงฟังก์ชันอื่น ๆ บนพื้นฐาน *join* [1, 2] อัลกอริทึมของ Adams ถูกเสนอในการแข่งขันระดับนานาชาติของชุมชน Standard ML ซึ่งเกี่ยวกับการสร้าง "เซตของจำนวนเต็ม" รางวัลมีสองประเภท: อัลกอริทึมที่เร็วที่สุด และโปรแกรมที่สง่างามที่สุดแต่ยังคงมีประสิทธิภาพ Adams ได้รับรางวัลความสง่างาม ในขณะที่อัลกอริทึมของเขาเร็วเกือบเท่าโปรแกรมที่เร็วที่สุดสำหรับเซตขนาดใหญ่มาก และเร็วกว่าสำหรับเซตขนาดเล็ก เนื่องจากความสง่างามของอัลกอริทึม มีไลบรารีอย่างน้อยสามไลบรารีที่ใช้อัลกอริทึมของ Adams ในการสร้างเซตและตารางที่มีลำดับ (Haskell [16] และ MIT/GNU Scheme และ SML) อันที่จริง อัลกอริทึมบนพื้นฐาน *join* ที่จะนำเสนอในภายหลังในหัวข้อนี้อาศัยอัลกอริทึมของ Adams ต่อมา Blelloch และ Reid-Miller แสดงว่าอัลกอริทึมที่คล้ายกันสำหรับ treap [10] มีงานที่เหมาะสมที่สุด (คือ $\Theta\left(m \log\left(\frac{n}{m}+1\right)\right)$) และเป็นแบบขนานด้วย ต่อมา Blelloch และคณะ [8] ขยายอัลกอริทึมของ Adams ไปยังรูปแบบการรักษาสมดุลหลายแบบและพิสูจน์ขอบเขตต้นทุน

อัลกอริทึม (Algorithms). *union*(T_1, T_2) รับ BST สองต้นและคืน BST ที่มียูเนียนของคีย์ทั้งหมด อัลกอริทึมใช้กลยุทธ์แบ่งแยกและเอาชนะแบบคลาสสิก ซึ่งเป็นแบบขนาน ในแต่ละระดับของการเวียนเกิด T_1 ถูกแยกด้วย $k(T_2)$ ทำให้ T_1 แตกเป็นสามส่วน: ส่วนหนึ่งมีคีย์ทั้งหมดที่เล็กกว่า $k(T_2)$ (แทนด้วย L_1) ส่วนตรงกลางซึ่งมีเพียงคีย์เดียวที่เท่ากับ $k(T_2)$ (เมื่อ $k(T_2) \in T_1$) หรือว่าง (เมื่อ $k(T_2) \notin T_1$) และส่วนที่สามมีคีย์ทั้งหมดที่ใหญ่กว่า $k(T_2)$ (แทนด้วย R_1)[แก้ 39] จากนั้นมีการเรียกเวียนเกิด *union* สองครั้งแบบขนาน ครั้งหนึ่งหา union ของ $lc(T_2)$ กับ L_1 คืน T_l และอีกครั้งหา union ของ $rc(T_2)$ กับ R_1 คืน T_r สุดท้ายอัลกอริทึมคืน *join*$(T_l, k(T_2), T_r)$ ซึ่งถูกต้องเพราะ $k(T_2)$ มากกว่าคีย์ทั้งหมดใน T_l และน้อยกว่าคีย์ทั้งหมดใน T_r

ฟังก์ชัน *intersection*(T_1, T_2) และ *difference*(T_1, T_2) หาส่วนร่วม (intersection) และผลต่าง (difference) ของคีย์ในสองเซตตามลำดับ อัลกอริทึมคล้ายกับ *union* ตรงที่ใช้ต้นไม้หนึ่งแยกอีกต้นหนึ่ง อย่างไรก็ตาม วิธีการ join และกรณีฐานต่างกัน สำหรับ *intersection* จะใช้ *join2* แทน *join* หากรากของต้นแรก*ไม่*พบในต้นที่สอง ดังนั้นกรณีฐานของอัลกอริทึม *intersection* คือคืนเซตว่างเมื่อเซตใดเซตหนึ่งว่าง สำหรับ *difference* จะใช้ *join2* เสมอ เพราะ $k(T_2)$ ควรถูกตัดออกจากต้นไม้ผลลัพธ์ กรณีฐานก็ต่างกันในแบบที่เห็นได้ชัด

ต้นทุนของอัลกอริทึมที่อธิบายข้างต้นสามารถสรุปได้ในทฤษฎีบทต่อไปนี้

ทฤษฎีบท 7.3. *สำหรับต้นไม้ AVL, RB, WB และ treap งานและสแปนของอัลกอริทึม (ดังแสดงในรูปที่ 30) ของ* union, intersection *หรือ* difference *บน BST ที่สมดุลสองต้นขนาด m และ n ($n \geq m$) คือ $O\left(m \log\left(\frac{n}{m}+1\right)\right)$ (ในค่าคาดหมายสำหรับ treap) และ $O(\log n \log m)$ ตามลำดับ (w.h.p. สำหรับ treap)*

ขอบเขตงานของอัลกอริทึมเหล่านี้เหมาะสมที่สุดในแบบจำลองแบบเปรียบเทียบ โดยเฉพาะเมื่อพิจารณาการสลับแทรกที่เป็นไปได้ทั้งหมด จำนวนการเปรียบเทียบน้อยที่สุดที่ต้องใช้เพื่อแยกแยะพวกมันคือ $\log\binom{m+n}{n} = \Theta\left(m \log\left(\frac{n}{m}+1\right)\right)$ [15] การพิสูจน์ทั่วไปของทฤษฎีบท 7.3 ที่ใช้ได้กับรูปแบบการรักษาสมดุลทั้งสี่หาได้ใน [8] สแปนของอัลกอริทึมเหล่านี้สามารถลดลงเป็น $O(\log m)$ สำหรับต้นไม้สมดุลน้ำหนัก แม้บนแบบจำลอง fork ทวิภาค [9] โดยใช้กลยุทธ์แบ่งแยกและเอาชนะที่ซับซ้อนกว่า

7.5 อัลกอริทึมต้นไม้อื่น ๆ ที่ใช้ *join*

การแทรกและการลบ (Insert and Delete). แทนที่จะใช้การสร้าง *insert* และ *delete* แบบคลาสสิก ซึ่งเฉพาะเจาะจงกับรูปแบบการรักษาสมดุล เรานิยามรุ่นที่อาศัย *join* ล้วน ๆ และดังนั้นจึงไม่ขึ้นกับรูปแบบการรักษาสมดุล

เรานำเสนอรหัสเทียมในรูปที่ 31 เพื่อแทรกรายการ e ลงในต้นไม้ T กรณีฐานคือเมื่อ t ว่าง และอัลกอริทึมสร้างโหนดใหม่สำหรับ e มิฉะนั้น อัลกอริทึมนี้เปรียบเทียบ k กับคีย์ที่รากและแทรก e ลงในต้นไม้ย่อยซ้ายหรือขวาแบบเวียนเกิด หลังจากนั้นต้นไม้ย่อยทั้งสองจะถูก *join* อีกครั้งโดยใช้โหนดราก เนื่องจากความถูกต้องของอัลกอริทึม *join* แม้จะมีความไม่สมดุล *join* ก็จะแก้ปัญหาได้

อัลกอริทึม *delete* คล้ายกับ *insert* ยกเว้นเมื่อพบคีย์ที่จะลบที่ราก ซึ่ง *delete* จะใช้ *join2* เพื่อเชื่อมต้นไม้ย่อยทั้งสองแทน ทั้งอัลกอริทึม *insert* และ *delete* ทำงานด้วยงาน $O(\log n)$ (และสแปนเท่ากันเพราะเป็นแบบลำดับ)

บางคนอาจคาดว่าการทำให้การแทรกหรือการลบเป็นนามธรรมโดยใช้ *join* แทนการเฉพาะเจาะจงสำหรับเกณฑ์สมดุลหนึ่ง ๆ จะมีโอเวอร์เฮดมาก ที่จริงการทดลองแสดงว่าไม่เป็นเช่นนั้น แม้จะมีเมทาดาทาเพิ่มเติมบางอย่าง (เช่น ตัวนับการอ้างอิง (reference counter)) อัลกอริทึมการแทรกบนพื้นฐาน join ช้ากว่าไลบรารี C++ STL ที่ได้รับการปรับแต่งอย่างสูงเพียง 17% ในแบบลำดับ [26]

ทฤษฎีบท 7.4. *อัลกอริทึมการแทรกบนพื้นฐาน join ใช้เวลาไม่เกิน $O(\log |T|)$ สำหรับต้นไม้ AVL, RB, WB หรือ treap*

ร่างการพิสูจน์. อัลกอริทึมการแทรกไล่ตามวิถีในต้นไม้ก่อนเพื่อหาตำแหน่งที่ถูกต้องสำหรับ k แล้วทำอัลกอริทึม *join* $O(\log n)$ ครั้ง แต่ละ *join* เชื่อม T_1 และ $T_2 \cup \{k\}$ ซึ่ง T_1 และ T_2 เดิมสมดุลกันอยู่แล้ว สำหรับรูปแบบการรักษาสมดุลใด ๆ ที่กล่าวถึง ต้นทุนของ *join* เป็นค่าคงที่ การพิสูจน์ที่เข้มงวดกว่าสามารถแสดงได้โดยอุปนัย □

ทฤษฎีบท 7.5. *อัลกอริทึมการลบบนพื้นฐาน join ใช้เวลาไม่เกิน $O(\log |T|)$ สำหรับต้นไม้ AVL, RB, WB หรือ treap*

ร่างการพิสูจน์. การพิสูจน์คล้ายกับการพิสูจน์ทฤษฎีบท 7.4 ข้อยกเว้นเดียวคือสามารถทำอัลกอริทึม *join2* ได้ไม่เกินหนึ่งครั้ง ซึ่งเพิ่มต้นทุนเพียง $O(\log n)$ □

การสร้างต้นไม้ (Build). ต้นไม้ทวิภาคที่สมดุลสามารถสร้างได้จากอาร์เรย์ที่เรียงแล้วของคู่คีย์–ค่า โดยใช้การแบ่งแยกและเอาชนะที่สมดุลบนอาร์เรย์ข้อมูลเข้าและรวมด้วย *join* ในการสร้างต้นไม้ทวิภาคที่สมดุลจากอาร์เรย์ใด ๆ เราเรียงลำดับอาร์เรย์ตามคีย์ก่อน แล้วนำตัวซ้ำออก รายการทั้งหมดที่มีคีย์เดียวกันจะอยู่ติดกันหลังการเรียงลำดับ ดังนั้นอัลกอริทึมจึงใช้การเรียงลำดับแบบขนานก่อน แล้วตามด้วยการบีบอัด (packing) แบบขนาน จากนั้นอัลกอริทึมดึงมัธยฐานในอาร์เรย์ที่นำตัวซ้ำออกแล้ว และสร้างต้นไม้ย่อยซ้าย/ขวาจากส่วนซ้าย/ขวาของอาร์เรย์แบบเวียนเกิดตามลำดับ สุดท้าย อัลกอริทึมใช้ *join* เพื่อเชื่อมมัธยฐานกับต้นไม้ย่อยทั้งสอง งานจึงเป็น $O(W_{\text{sort}(n)} + W_{\text{remove}(n)} + n)$ และสแปนเป็น $O(S_{\text{sort}(n)} + S_{\text{remove}(n)} + \log n)$ สำหรับอัลกอริทึมการเรียงลำดับและการนำตัวซ้ำออกที่มีประสิทธิภาพเชิงงานและมีสแปน $O(\log n)$ สิ่งนี้ให้ขอบเขตในตารางที่ 3

การปรับปรุงเป็นกลุ่ม (Bulk Updates). เราใช้ *multi_insert* และ *multi_delete* เพื่อบันทึกชุดของการดำเนินการเขียน ฟังก์ชัน *multi_insert*(T, A, m) รับข้อมูลเข้าเป็นรากของต้นไม้ t และพอยน์เตอร์หัวของอาร์เรย์ A พร้อมความยาว m

เรานำเสนอรหัสเทียมของ *multi_insert* ในรูปที่ 31 อัลกอริทึมนี้เรียงลำดับ A ตามคีย์ก่อน แล้วนำตัวซ้ำออกในลักษณะคล้ายกับใน *build* จากนั้นเราใช้อัลกอริทึมแบ่งแยกและเอาชนะ *multi_insert_s* เพื่อแทรกอาร์เรย์ที่เรียงแล้วลงในต้นไม้ กรณีฐานคือเมื่ออาร์เรย์ A หรือ T ว่าง มิฉะนั้น อัลกอริทึมใช้การค้นหาแบบทวิภาคเพื่อหาตำแหน่งของคีย์ของ t ในอาร์เรย์ ได้ดัชนี b ที่สอดคล้องกันใน A d เป็นบิตที่บ่งบอกว่า k ปรากฏใน A หรือไม่ จากนั้นอัลกอริทึม multi-insert ส่วนซ้ายของ A (ถึง $A[b]$) ลงในต้นไม้ย่อยซ้าย และส่วนขวาของ A ลงในต้นไม้ย่อยขวาแบบเวียนเกิด การเรียกเวียนเกิดทั้งสองสามารถรันแบบขนานได้ สุดท้ายอัลกอริทึมต่อผลลัพธ์ทั้งสองเข้าด้วยกันด้วยรากของ T อัลกอริทึมแบ่งแยกและเอาชนะที่คล้ายกันสามารถใช้สำหรับ *multi_delete* โดยใช้ *join2* แทน *join* เมื่อจำเป็น

การแยกการเรียงลำดับออกจากการแทรกมีประโยชน์หลายประการ ประการแรก การเรียงลำดับแบบขนานได้รับการศึกษาอย่างดีและมีอัลกอริทึมการเรียงลำดับที่ได้รับการปรับแต่งอย่างสูงให้ใช้ ซึ่งทำให้ปัญหาง่ายขึ้น ประการที่สอง หลังการเรียงลำดับ รายการทั้งหมดใน A ที่จะถูกผสานกับต้นไม้ย่อยหนึ่ง ๆ ใน T จะอยู่ติดกัน ซึ่งทำให้ใช้แนวทางแบ่งแยกและเอาชนะที่ให้ความขนานที่ดีและยังให้ความเป็นท้องถิ่น (locality) ที่ดีกว่าด้วย

งานและสแปนรวมของการแทรกหรือการลบอาร์เรย์ความยาว m ลงในต้นไม้ขนาด $n \geq m$ คือ $O\left(m \log\left(\frac{n}{m} + 1\right)\right)$ และ $O(\log m \log n)$ ตามลำดับ [8] การวิเคราะห์คล้ายกับอัลกอริทึม union

Build

```
1  build_sorted(S, i, j) {
2    if i = j then return ∅;
3    if i + 1 = j then
4      return singleton(S[i]);
5    m = (i + j)/2;
6    L = build'(S, i, m) ||
7      R = build'(S, m + 1, j);
8    return join(L, S[m], R); }
9  build(S, m) {
10   (S2, m2) = sort_rm_dup(S, m);
11   build_sorted(S2,0,m2);}
```

Filter

```
1  filter(T, f) {
2    if T = ∅ then return ∅;
3    (L, e, R) = expose(T);
4    L' = filter(L, f) ||
5        R' = filter(R, f);
6    if f(e) then
7      return join(L', e, R');
8    else join2(L', R'); }
```

Map and Reduce

```
1  map_reduce(T, g', f', I') {
2    if T = ∅ then return I';
3    ⟨L, k, v, R⟩ = expose(T);
4    L' = MapReduce(L, g', f', I') ||
5      R' = MapReduce(R, g', f', I');
6    return f'(L', f'(g'(k, v), R')); }
```

Range

```
1  range(T, l, r) {
2    (T1, T2) = split(T, l);
3    (T3, T4) = split(T2, r);
4    return T3; }
```

Foreach Index

```
1  foreach_index(T, φ, s) {
2    if (t = ∅) return;
3    (L, e, R) = expose(T);
4    left = size(L);
5    L = foreach_index(L, φ, s); ||
6      R = foreach_index(R, φ, s+1+left);
7    φ(e, left);}
```

Insertion

```
1  insert(T, e) {
2    if T = ∅ then return singleton(e);
3    ⟨L, e', R⟩ = expose(T);
4    if k(e) = k(e') then return T;
5    if k(e) < k(e') then
6      return join(insert(L, e), e', R);
7    return join(L, e', insert(R, e)); } }
```

Deletion

```
1  delete(T, k) {
2    if T = ∅ then return ∅;
3    ⟨L, e', R⟩ = expose(T);
4    if k < k(e') then return join(delete(L, k), e', R);
5    if k(e') < k then return join(L, e', delete(R, k));
6    return join2(L, R); }
```

Multi-insertion

```
1  multi_insert_s(T, A, m) {
2    if (T = ∅) return build(A, m);
3    if (m = 0) return t;
4    ⟨L, e, R⟩ = expose(T);
5    b = binary_search(A, m, k(e));
6    d = (b < m) and (k(A[b]) > k(e));
7    L = multi_insert_s(r→lc, A, b) ||
8      R = multi_insert_s(r→rc, A+b-d, m-b-d);
9    return concat(L, e, R); }
10 multi_insert(t, A, m) {
11   (A2, m2) = sort_rm_dup(A, m);
12   return multi_insert_sorted(t, A2, m2);}
```

รูปที่ 31: **รหัสเทียมของฟังก์ชันบนพื้นฐาน *join* บางตัว** – ทั้งหมดไม่ขึ้นกับรูปแบบการรักษาสมดุล ไวยากรณ์ $S_1 || S_2$ หมายความว่าคำสั่งสองคำสั่ง S_1 และ S_2 สามารถรันแบบขนานได้บนพื้นฐานของความขนานแบบ fork-join ใดก็ได้ *(โค้ดในรูปมีจุดผิด ที่ถูกคือ: **`foreach_index`** บรรทัด 7 ต้องเป็น $\phi(e, s + left)$; ใน **`multi_insert_s`** บรรทัด 6 ต้องเป็น $d = (b < m)$ and $(k(A[b]) = k(e))$, บรรทัด 7–8 ต้องใช้ $lc(T)$, $rc(T)$ และ $A + b + d$ แทน **`r->lc`**, **`r->rc`** และ $A + b - d$; และ **`multi_insert`** ต้องเรียก **`multi_insert_s`**)*[แก้ 40]

ช่วงคีย์ (Range). *range* ดึงเซตย่อยของทูเพิลในช่วงคีย์หนึ่ง ๆ ออกจากต้นไม้ และส่งออกเป็นต้นไม้ใหม่ ต้นทุนของฟังก์ชัน range คือ $O(\log n)$ อัลกอริทึม range แบบบริสุทธิ์ (pure) คัดลอกโหนดบนสองวิถี วิถีละหนึ่งปลายของช่วง และใช้โหนดเหล่านั้นเป็นตัวกลางเพื่อ join ต้นไม้ย่อยกลับเข้าด้วยกัน เมื่อช่วงที่ดึงออกมีขนาดใหญ่ อัลกอริทึม range แบบบริสุทธิ์นี้มีประสิทธิภาพมากกว่า (เวลาลอการิทึม) การเยี่ยมทั้งช่วงแล้วคัดลอกอย่างมาก

การกรอง (Filter). ฟังก์ชัน *filter*(t, ϕ) คืนต้นไม้ที่มีทูเพิลทั้งหมดใน T ที่สอดคล้องกับเพรดิเคต ϕ อัลกอริทึมนี้กรองต้นไม้ย่อยทั้งสองแบบเวียนเกิดแบบขนาน แล้วตัดสินว่ารากสอดคล้องกับ ϕ หรือไม่ ถ้าใช่ อัลกอริทึมใช้รากเป็นตัวกลางเพื่อ *join* ผลลัพธ์เวียนเกิดทั้งสอง มิฉะนั้นจะเรียก *join2* งานของ *filter* คือ $O(n)$ และความลึกคือ $O(\log^2 n)$ เมื่อ n คือขนาดของต้นไม้

Map และ Reduce. ฟังก์ชัน *map_reduce*$(T, f_m, \langle f_r, I\rangle)$ บนต้นไม้ t (ที่มีชนิดข้อมูล E สำหรับทูเพิล) รับอาร์กิวเมนต์สามตัวและคืนค่าชนิด V' $f_m : E \mapsto V'$ คือฟังก์ชัน *map* ที่แปลงแต่ละทูเพิลที่เก็บไว้เป็นค่าชนิด V' $\langle f_r, I\rangle$ คือมอนอยด์ (monoid) ซึ่ง $f_r : V' \times V' \mapsto V'$ เป็นฟังก์ชัน *reduce* ที่มีสมบัติการเปลี่ยนหมู่บน V' และ $I \in V'$ คือเอกลักษณ์ของ f_r อัลกอริทึมจะเรียกฟังก์ชันบนต้นไม้ย่อยทั้งสองของมันแบบเวียนเกิดแบบขนาน แล้ว reduce ผลลัพธ์ด้วย f_r ในภายหลัง

Claude อธิบายเพิ่ม: สรุปบทที่ 7: แนวคิด “join อย่างเดียวพอ”

- **แยกความรับผิดชอบ:** มีเพียง *join* ที่รู้กฎการรักษาสมดุล (AVL / แดง–ดำ / สมดุลน้ำหนัก / treap) ฟังก์ชันอื่นทั้งหมด (*split*, *union*, *insert*, *filter*, ...) เขียนครั้งเดียวแล้วใช้ได้กับทุกแบบ
- **รูปแบบโค้ดที่เห็นซ้ำ:** `expose` ราก $\rightarrow$ เรียกเวียนเกิดซ้าย $\|$ ขวา $\rightarrow$ รวมด้วย *join* (ถ้าเก็บราก) หรือ *join2* (ถ้าทิ้งราก) นี่คือแบ่งแยกและเอาชนะแบบเดียวกับ mergesort ในบทที่ 5 แต่ทำบนต้นไม้
- **ตัวอย่างความหมายของ $O(m \log(\frac{n}{m} + 1))$:** ถ้า $m = 1$ จะได้ $O(\log n)$ เท่ากับการแทรกตัวเดียว ถ้า $m = n$ จะได้ $O(n)$ เท่ากับการผสานอาร์เรย์ union จึงดีที่สุดในทั้งสองสุดขั้วโดยอัตโนมัติ
- **สแปน $O(\log n \log m)$** มาจากความลึกของการเวียนเกิด $O(\log m)$ ระดับ คูณต้นทุนของ *split/join* ระดับละ $O(\log n)$

เชื่อมโยง: *split* เทียบได้กับ KTH ในการผสาน (4.4) คือหาจุดแบ่งเพื่อให้สองส่วนทำงานอิสระต่อกัน แนวคิดนี้ใช้จริงในไลบรารี PAM [26]

8 แบบจำลองอื่น ๆ และการจำลอง (Other Models and Simulations)

ในหัวข้อนี้เราพิจารณาแบบจำลองอื่น ๆ บางแบบ (ปัจจุบันมีเพียง PRAM) และอภิปรายผลการจำลอง (simulation) ระหว่างแบบจำลอง เราสนใจเป็นพิเศษว่าจะจำลอง MP-RAM บนเครื่องที่มีตัวประมวลผลจำนวนคงที่ได้อย่างไร โดยเฉพาะเราพิจารณาปัญหาการจัดตาราง (scheduling problem) ซึ่งเป็นปัญหาการจัดตารางโพรเซสลงบนตัวประมวลผลอย่างมีประสิทธิภาพ

8.1 PRAM

แบบจำลอง Parallel Random Access Machine (PRAM)[แก้ 41] เป็นหนึ่งในแบบจำลองแรก ๆ ที่ใช้วิเคราะห์ต้นทุนของอัลกอริทึมแบบขนาน อัลกอริทึมจำนวนมากถูกวิเคราะห์ในแบบจำลองนี้ในทศวรรษ 80 และต้นทศวรรษ 90 PRAM ประกอบด้วยตัวประมวลผล p ตัวที่ใช้หน่วยความจำขนาดไม่จำกัดร่วมกัน แต่ละตัวมีชุดรีจิสเตอร์ของตัวเองและตัวนับโปรแกรมของตัวเอง แต่ทั้งหมดทำงานพร้อมเพรียงกัน (synchronously) (หนึ่งคำสั่งต่อรอบ) ในอัลกอริทึมทั่วไป ตัวประมวลผลทั้งหมดรันลำดับคำสั่งเดียวกัน ยกเว้นบางตัวที่อาจไม่ทำงาน (inactive) ตัวประมวลผลแต่ละตัวสามารถดึงตัวระบุ (identifier) ของตน ซึ่งเป็นจำนวนเต็มใน $[1, \ldots, p]$ ได้ PRAM แตกต่างจาก MP-RAM[แก้ 41] ในสองประการสำคัญ ประการแรก ระหว่างการคำนวณ มันมีตัวประมวลผลจำนวนคงที่เสมอ แทนที่จะอนุญาตให้สร้างโพรเซสแบบพลวัต ประการที่สอง PRAM เป็นแบบพร้อมเพรียงอย่างสมบูรณ์ ตัวประมวลผลทั้งหมดทำงานเป็นจังหวะเดียวกัน (lock-step)

ต้นทุนวัดในรูปของจำนวนคำสั่ง เวลา และจำนวนตัวประมวลผล เวลาของอัลกอริทึมมักเป็นฟังก์ชันของจำนวนตัวประมวลผล ตัวอย่างเช่น การหาผลรวมของค่า n ค่าในต้นไม้สามารถทำได้ในเวลา $O(n/p + \log p)$ แนวคิดคือแบ่งข้อมูลเข้าออกเป็นบล็อกขนาด n/p ให้ตัวประมวลผล i หาผลรวมของสมาชิกในบล็อกที่ i แล้วหาผลรวมของผลลัพธ์ในต้นไม้

เนื่องจากตัวประมวลผลทั้งหมดทำงานพร้อมเพรียงกัน ประเภทของสภาวะแข่งขัน (race condition) จึงค่อนข้างต่างจากใน MP-RAM หากมีการอ่านและการเขียนในรอบเดียวกันที่ตำแหน่งเดียวกัน การอ่านจะเกิดก่อนการเขียน PRAM มีรูปแบบย่อยหลายแบบขึ้นอยู่กับว่าเกิดอะไรขึ้นในกรณีที่มีการเขียนหลายครั้งไปยังตำแหน่งเดียวกันในรอบเดียวกัน รุ่นเขียนแบบเอกสิทธิ์ (exclusive-write: EW) ไม่อนุญาตให้เขียนพร้อมกันไปยังตำแหน่งเดียวกัน รุ่นเขียนพร้อมกันแบบใดก็ได้ (Arbitrary Concurrent Write: ACW) สมมติว่าการเขียนใดการเขียนหนึ่งจะชนะ รุ่นเขียนพร้อมกันแบบลำดับความสำคัญ (Priority Concurrent Write: PCW) สมมติว่าตัวประมวลผลที่มีหมายเลขสูงที่สุดจะชนะ มี PRAM รุ่นไม่พร้อมเพรียง (asynchronous) ด้วย แม้ว่าเราจะไม่อภิปรายในที่นี้

8.2 การจำลอง (Simulations)

จะเขียนในภายหลัง

8.3 ปัญหาการจัดตาราง (The Scheduling Problem)

เราสนใจการจัดตารางการสร้างงานแบบพลวัตที่บ่งบอกโดย MP-RAM ลงบนตัวประมวลผลจำนวนคงที่ และการแปลงขอบเขตของงานและความลึกเป็นขอบเขตเวลาสำหรับตัวประมวลผลเหล่านั้น ปัญหาการจัดตารางนี้สามารถทำให้เป็นนามธรรมได้ว่าเป็นการท่อง DAG โดยเฉพาะ ปัญหาการจัดตารางสำหรับตัวประมวลผล p ตัวคือ กำหนด DAG ที่มีรากเดียว ให้เยี่ยมจุดยอดทั้งหมดเป็นขั้น ๆ โดยแต่ละขั้นเยี่ยมจุดยอดไม่เกิน p จุด และไม่มีจุดยอดใดถูกเยี่ยมในขั้นใด เว้นแต่จุดยอดก่อนหน้า (predecessor) ทั้งหมดใน DAG ถูกเยี่ยมแล้วในขั้นก่อนหน้า สิ่งนี้จำลองประเภทของการคำนวณที่เราสนใจ เพราะแต่ละคำสั่งสามารถพิจารณาเป็นจุดยอดใน DAG ไม่มีคำสั่งใดทำงานได้จนกว่าคำสั่งก่อนหน้าจะถูกรัน และเราสมมติว่าแต่ละคำสั่งใช้เวลาคงที่

เป้าหมายของเราคือจำกัดจำนวนขั้นในรูปฟังก์ชันของจำนวนจุดยอด w ใน DAG และความลึก d ของมัน นอกจากนี้เราต้องการให้แต่ละขั้นทำงานเร็ว ในที่นี้เราจะสมมติแบบจำลอง PRAM แบบพร้อมเพรียงเป็นเป้าหมาย แต่แนวคิดส่วนใหญ่ใช้ได้กับแบบจำลองที่ไม่พร้อมเพรียงมากกว่าด้วย

ปรากฏว่าโดยทั่วไป การหาตารางที่มีจำนวนขั้นน้อยที่สุดเป็นปัญหา NP-hard [28] แต่การหาการประมาณที่สมเหตุสมผลไม่ยากนัก ข้อสังเกตแรกของเราคือขอบเขตล่างอย่างง่าย เนื่องจากมีจุดยอด w จุดและแต่ละขั้นเยี่ยมได้เพียง p จุด ตารางใด ๆ จะต้องใช้

อย่างน้อย w/p ขั้น นอกจากนี้ เนื่องจากเราต้องทำจุดยอดก่อนหน้าให้เสร็จก่อนตัวจุดยอดเอง ตารางจะต้องใช้อย่างน้อย d ขั้นด้วย เมื่อรวมกันจะได้:

ข้อสังเกต 8.1. ตารางสำหรับตัวประมวลผล p ตัวใด ๆ ของ DAG ที่มีความลึก d และขนาด w ต้องใช้อย่างน้อย $\max(w/p, d)$ ขั้น

ต่อไปเราดูว่าเราเข้าใกล้ขอบเขตนี้ได้แค่ไหน

8.4 การจัดตารางแบบละโมบ (Greedy Scheduling)

ตัวจัดตาราง*แบบละโมบ* (greedy scheduler) คือตัวจัดตารางที่ตัวประมวลผลไม่เคยว่างงานเมื่อมีงานให้ทำ กล่าวให้แม่นยำขึ้น ตารางแบบ p-ละโมบ คือตารางที่หากมีจุดยอดที่พร้อม (ready) r จุดในขั้นหนึ่ง ขั้นนั้นต้องเยี่ยม $\min(r, p)$ จุดในนั้น

ทฤษฎีบท 8.1. *ตารางแบบ p-ละโมบใด ๆ บน DAG ขนาด w และความลึก d จะใช้ไม่เกิน $w/p + d$ ขั้น*

พิสูจน์. สมมติว่าขั้นหนึ่ง*ยุ่ง* (busy) หากมันเยี่ยมจุดยอด p จุด และ*ไม่สมบูรณ์* (incomplete) ในกรณีอื่น มีขั้นที่ยุ่งไม่เกิน $\lfloor w/p \rfloor$ ขั้น เพราะขั้นที่ยุ่งจำนวนเท่านั้นจะเยี่ยมจุดยอดทั้งหมดยกเว้น $r < p$ จุด ต่อไปเราจำกัดจำนวนขั้นที่ไม่สมบูรณ์ พิจารณาขั้นที่ไม่สมบูรณ์หนึ่งขั้น และให้ j เป็นระดับแรกที่มีจุดยอดที่ยังไม่ถูกเยี่ยมก่อนทำขั้นนั้น จุดยอดทั้งหมดในระดับ j พร้อม เพราะระดับก่อนหน้าถูกเยี่ยมหมดแล้ว นอกจากนี้ เนื่องจากขั้นนี้ไม่สมบูรณ์ จำนวนจุดยอดที่พร้อมจึงน้อยกว่า p และตารางแบบละโมบเยี่ยมจุดยอดที่พร้อมทั้งหมด[แก้ 42] ดังนั้นขั้นนี้จะเยี่ยมจุดยอดที่เหลือทั้งหมดในระดับ j (และอาจมีจุดอื่นด้วย) เนื่องจากมีเพียง d ระดับ จึงมีขั้นที่ไม่สมบูรณ์ได้ไม่เกิน d ขั้น การรวมขอบเขตบนของขั้นที่ยุ่งและขั้นที่ไม่สมบูรณ์พิสูจน์ทฤษฎีบท □

เราควรสังเกตว่าตารางแบบละโมบเช่นนี้มีจำนวนขั้นที่อยู่ภายในตัวประกอบสองของขอบเขตล่างเสมอ ดังนั้นมันจึงเป็นการประมาณสองเท่า (two-approximation) ของค่าเหมาะที่สุด หากพจน์ใดพจน์หนึ่งครอบงำอีกพจน์ การประมาณจะดียิ่งขึ้น แม้ว่าการจัดตารางแบบละโมบจะรับประกันขอบเขตที่ดี แต่มันไม่ได้บอกเราว่าจะส่งจุดยอดที่พร้อมไปยังตัวประมวลผลได้อย่างไร โดยเฉพาะ ไม่ชัดเจนว่าเราจะกำหนดงานที่พร้อมให้ตัวประมวลผลในเวลาคงที่ได้

8.5 ตัวจัดตารางแบบขโมยงาน (Work Stealing Schedulers)

ต่อไปเราพิจารณาอัลกอริทึมการจัดตาราง *การขโมยงาน* (work stealing) ซึ่งรวมต้นทุนทั้งหมดไว้ อัลกอริทึมนี้ไม่ได้ละโมบอย่างเคร่งครัด แต่รับประกันขอบเขตที่ใกล้เคียงกับขอบเขตแบบละโมบ และทำให้เราทำแต่ละขั้นได้ในเวลาคงที่ ตัวจัดตารางที่เราอภิปรายจำกัดอยู่ที่การ fork และ join แบบทวิภาค เราสมมติว่าการเยี่ยมจุดยอดคืนผลลัพธ์ที่เป็นไปได้หนึ่งในสามแบบ: `fork`(v_1, v_2) หากจุดยอดเป็น fork, `next`(v) หากมีลูกที่พร้อมหนึ่งตัว หรือ `empty` หากไม่มีลูกที่พร้อม สังเกตว่าหากลูกของจุดยอดเป็นจุด join การเยี่ยมอาจคืน `next`(v) หากโหนดแม่อีกตัวของ v เสร็จแล้ว หรือ `empty` หากยังไม่เสร็จ เนื่องจากโหนดแม่ทั้งสองของจุด join อาจเสร็จพร้อมกัน เราจึงใช้ test-and-set (หรือการเขียนพร้อมกันตามด้วยการอ่าน) เพื่อจัดลำดับพวกมัน

อัลกอริทึม (หรือตัวจัดตาราง) แบบขโมยงานเก็บจุดยอดที่พร้อมไว้ในเซตของคิวงาน คิวละหนึ่งตัวประมวลผล ตัวประมวลผลแต่ละตัวจะ push และ pop เฉพาะที่ด้านล่าง (bottom) ของคิวของตัวเอง และ pop จากด้านบน (top) เมื่อขโมยจากคิวใด ๆ ตัวจัดตารางเริ่มด้วยรากของ DAG ในคิวหนึ่งคิว และคิวที่เหลือว่าง รหัสเทียมของอัลกอริทึมแสดงในรูปที่ 32 แต่ละขั้นของตัวจัดตารางประกอบด้วยระยะขโมย (steal phase) ตามด้วยระยะเยี่ยม (visit phase) ในระยะขโมย ตัวประมวลผลแต่ละตัวที่มีคิวว่างจะเลือกตัวประมวลผลเป้าหมายแบบสุ่ม และพยายาม “ขโมย” จุดยอดบนสุดจากคิวของมัน ความพยายามอาจล้มเหลวหากคิวเป้าหมายว่าง หรือหากมีตัวประมวลผลอื่นพยายามขโมยจากเป้าหมายเดียวกันในรอบเดียวกันและชนะ ความล้มเหลวอาจเกิดขึ้นได้แม้คิวมีจุดยอดหลายจุด เพราะทุกตัวพยายามขโมยจุดบนสุด หากการขโมยสำเร็จ ตัวประมวลผลจะเพิ่มจุดยอดที่ขโมยมาลงในคิวของตน ในระยะเยี่ยม ตัวประมวลผลแต่ละตัวที่มีคิวไม่ว่างจะนำจุดยอดออกจากด้านล่างของคิว เยี่ยมมัน แล้ว push จุดยอดใหม่ 0, 1 หรือ 2 จุดกลับลงไปที่ด้านล่าง

```
1  workStealingScheduler(v) =
2    pushBot(Q[0], v);
3    while not all queues are empty
4      parfor i in [0 : p]
5        if empty(Q[i]) then                    % steal phase
6          j = rand([0 : p]);
7          steal[j] = i;
8          if (steal[j] = i) and not(empty(Q[j])) then
9            pushBot(Q[i],popTop(Q[j]))
10       if (not(empty(Q[i]))) then             % visit phase
11         u = popBot(Q[i]);
12         case (visit(u)) of
13           fork(v1, v2) ⇒ pushBot(Q[i], v2); pushBot(Q[i], v1);
14           next(v) ⇒ pushBot(Q[i], v);
```

รูปที่ 32: ตัวจัดตารางแบบขโมยงาน ตัวประมวลผลต้องประสานจังหวะกันระหว่างบรรทัดที่ 7 กับบรรทัดถัดไป และระหว่างสองระยะ

อัลกอริทึมขโมยงานไม่ได้ละโมบอย่างสมบูรณ์ เพราะจุดยอดที่พร้อมบางจุดอาจไม่ถูกเยี่ยม แม้ว่าตัวประมวลผลบางตัวอาจขโมยล้มเหลว ในการวิเคราะห์การขโมยงาน เราจะใช้นิยามต่อไปนี้ เรากล่าวว่าจุดยอดที่ด้านบนของทุกคิวที่ไม่ว่างเป็น*จุดเอก* (prime) ในตัวจัดตารางแบบขโมยงาน แต่ละโหนด join ถูกเปิดใช้งาน (enabled) โดยโหนดแม่ตัวใดตัวหนึ่ง (คือถูกใส่ลงในคิวของมัน) หากตลอดทั้ง DAG เรารวมเฉพาะเส้นเชื่อมไปยังโหนดแม่ตัวนั้น และไม่รวมอีกตัว สิ่งที่เหลือคือต้นไม้ ในต้นไม้นั้นมีวิถีเดียวจากรากของ DAG ไปยังจุดปลาย (sink) ซึ่งเราเรียกว่า*วิถีวิกฤต* (critical path) วิถีใดเป็นวิถีวิกฤตอาจขึ้นอยู่กับการเลือกแบบสุ่มในตัวจัดตาราง เรานิยาม*วิถีวิกฤตขยาย* (expanded critical path: ECP) ว่าเป็นวิถีวิกฤตรวมกับลูกทางขวาทั้งหมดของจุดยอดบนวิถี

ทฤษฎีบท 8.2. *ระหว่างสองรอบใด ๆ ของอัลกอริทึมขโมยงานบน DAG G มีจุดเอกอย่างน้อยหนึ่งจุดที่อยู่ใน ECP*

พิสูจน์. (โครงร่าง) จะต้องมีจุดยอดที่พร้อมพอดีหนึ่งจุด v บนวิถีวิกฤต และจุดยอดนั้นต้องอยู่ในคิวใดคิวหนึ่ง เราอ้างว่าจุดยอดทั้งหมดที่อยู่เหนือ v ในคิวนั้นเป็นลูกทางขวาของวิถีวิกฤต และดังนั้นอยู่บนวิถีวิกฤตขยาย ดังนั้นสมาชิกบนสุดของคิวนั้นจึงอยู่บน ECP และเป็นจุดเอก สมบัติลูกทางขวาเป็นผลจากข้อเท็จจริงที่ว่าเมื่อ push ลงด้านล่างของคิวตอน fork เรา push ลูกทางขวาก่อนแล้วจึงลูกทางซ้าย จากนั้นเราจะ pop ลูกทางซ้ายและลูกทางขวาจะยังคงอยู่ การ push จุดเดี่ยวลงด้านล่างก็รักษาสมบัตินี้ไว้ เช่นเดียวกับการ pop จุดยอดจากด้านล่างหรือการขโมยจากด้านบน ดังนั้นสมบัตินี้ถูกรักษาไว้ภายใต้การดำเนินการทั้งหมดบนคิว □

ตอนนี้เราสามารถพิสูจน์ขอบเขตของการขโมยงานได้

ทฤษฎีบท 8.3. *ตารางแบบขโมยงานที่มีตัวประมวลผล p ตัวบน DAG ทวิภาคขนาด w และความลึก d จะใช้ไม่เกิน $w/p + O(d + \log(1/\epsilon))$ ขั้นด้วยความน่าจะเป็น $1 - \epsilon$*

พิสูจน์. เช่นเดียวกับการพิสูจน์การจัดตารางแบบละโมบ เรานับตัวประมวลผลที่ว่างเข้ากับความลึก และตัวที่ยุ่งเข้ากับงาน สำหรับแต่ละขั้น i เราพิจารณาจำนวนตัวประมวลผล q_i ที่มีคิวว่าง (เป็นตัวแปรสุ่มเพราะขึ้นกับการเลือกแบบสุ่มของเรา) ตัวประมวลผลแต่ละตัวที่มีคิวว่างจะพยายามขโมย จากนั้นเราแสดงว่าจำนวนความพยายามขโมย $S = \sum_{i=0}^{\infty} q_i$ ถูกจำกัดด้วย $O(pd + p\ln(1/\epsilon))$ ด้วยความน่าจะเป็น $1-\epsilon$ ดังนั้นงานรวมถึงขั้นที่อาจว่างคือ $w+O(pd+p\ln(1/\epsilon))$ การหารด้วย p จะได้ขอบเขต

สัญชาตญาณของการจำกัดจำนวนความพยายามขโมยคือ แต่ละความพยายามมีโอกาสบางส่วนที่จะขโมยโหนดเอกบน ECP ดังนั้นหลังจากพยายามขโมยมากพอ เราจะทำวิถีวิกฤตเสร็จด้วยความน่าจะเป็นสูง

พิจารณาขั้น i ที่มีคิวว่าง q_i คิว และพิจารณาจุดเอก v ในขั้นนั้น คิวว่างแต่ละคิวจะขโมย v ด้วยความน่าจะเป็น $1/p$ ดังนั้น

ความน่าจะเป็นโดยรวมที่จุดเอก (รวมถึงจุดบนวิถีวิกฤต) จะถูกขโมยในขั้น i คือ:

$$\rho_i = 1 - \left(1 - \frac{1}{p}\right)^{q_i} > \frac{q_i}{p}\left(1 - \frac{1}{e}\right) > \frac{q_i}{2p},$$

กล่าวคือ ยิ่งมีคิวว่างมาก เราก็ยิ่งมีโอกาสขโมยและเยี่ยมจุดยอดบน ECP

ให้ X_i เป็นตัวแปรสุ่มชี้บอกว่าโหนดเอกบน ECP ถูกขโมยในขั้น i และให้ $X = \sum_{i=0}^{\infty} X_i$ ค่าคาดหมาย $E[X_i] = \rho_i$ และค่าคาดหมาย

$$\mu = E[X] = \sum_{i=0}^{\infty} \rho_i > \sum_{i=0}^{\infty} \frac{q_i}{2p} = \frac{S}{2p}.$$

หาก X ถึง $2d$ ตารางจะต้องเสร็จแล้ว เพราะมีจุดยอดบน ECP ไม่เกิน $2d$ จุด ดังนั้นเราสนใจที่จะทำให้แน่ใจว่าความน่าจะเป็น $P[X < 2d]$ มีค่าน้อย เราใช้ขอบเขตเชอร์นอฟ:

$$P[X < (1-\delta)\mu] < e^{-\frac{\delta^2\mu}{2}}.$$

กำหนด $(1-\delta)\mu = 2d$ จะได้ $\delta = (1 - 2d/\mu)$ จากนั้นเราได้ $\delta^2 = (1 - 4d/\mu + (2d/\mu)^2) > (1 - 4d/\mu)$ และดังนั้น $\delta^2\mu > \mu - 4d$ ซึ่งให้:

$$P[X < 2d] < e^{-\frac{\mu-4d}{2}}.$$

สิ่งนี้จำกัดความน่าจะเป็นที่วิถีวิกฤตขยาย (ECP) ยังไม่เสร็จ แต่เราไม่รู้ว่าวิถีใดเป็นวิถีวิกฤต มีวิถีวิกฤตที่เป็นไปได้ไม่เกิน 2^d วิถี เพราะ DAG มีการ fork แบบทวิภาค เราสามารถใช้ขอบเขตยูเนียนบนทุกวิถี ซึ่งให้ความน่าจะเป็นที่วิถีวิกฤตที่เป็นไปได้ใด ๆ ยังไม่เสร็จถูกจำกัดบนด้วย:

$$P[X < 2d] \cdot 2^d < e^{-\frac{\mu-4d}{2}} \cdot 2^d = e^{-\frac{\mu}{2} + d(2+\ln 2)}.$$

กำหนดให้ค่านี้เท่ากับ ϵ และเนื่องจาก $\mu > \frac{S}{2p}$ จะได้ผลเฉลย:

$$S < 4p(d(2+\ln 2) + \ln(1/\epsilon)) \in O(pd + p\ln(1/\epsilon)).$$

ดังนั้นความน่าจะเป็นที่ S มีค่าไม่เกิน $O(pd + p\ln(1/\epsilon))$ จึงมีค่าอย่างน้อย $(1-\epsilon)$ ซึ่งให้ขอบเขตของจำนวนความพยายามขโมย □

เนื่องจากแต่ละขั้นของอัลกอริทึมขโมยงานใช้เวลาคงที่บน ACW PRAM จึงนำไปสู่บทแทรกต่อไปนี้

บทแทรก 8.1. *สำหรับ DAG ทวิภาคขนาด w และความลึก d บน ACW PRAM ที่มีตัวประมวลผล p ตัว ตัวจัดตารางแบบขโมยงานจะใช้เวลา*

$$O(w/p + d + \log(1/\epsilon))$$

ด้วยความน่าจะเป็น $1-\epsilon$

Claude อธิบายเพิ่ม: สรุปบทที่ 8: ปิดวงจากทฤษฎีสู่เครื่องจริง

- **ขอบเขตล่าง** $\max(w/p, d)$ (ข้อสังเกต 8.1) กับ**ขอบเขตบนแบบละโมบ** $w/p + d$ (ทฤษฎีบท 8.1) คือสองฝั่งของสมการ (1) ในบทที่ 2 จึงพิสูจน์ได้ว่าการวิเคราะห์งาน–สแปนทำนายเวลาจริงได้ภายในตัวประกอบสอง
- **Work stealing** ทำให้ได้ผลใกล้เคียงกับแบบละโมบโดยไม่ต้องมีคิวกลาง ตัวประมวลผลที่ว่างจะไปขโมยงานจาก “ด้านบน” ของคิวคนอื่นซึ่งมักเป็นงานชิ้นใหญ่ ส่วนเจ้าของคิวทำงานที่ “ด้านล่าง” ซึ่งเป็นงานล่าสุดที่ยังอยู่ในแคช
- **ในทางปฏิบัติ:** ระบบที่ใช้ work stealing ได้แก่ Cilk, Intel TBB, Java ForkJoinPool และ ParlayLib คุณจึงเขียนโค้ดด้วย fork/join หรือ parfor แล้ววิเคราะห์ด้วยงาน–สแปนตามบทที่ 2 ได้เลย

ภาคผนวก: รายการแก้คำผิดจากต้นฉบับ

ภาคผนวกนี้ไม่ใช่ส่วนของต้นฉบับ แต่ Claude รวบรวมไว้เอง จุดที่แก้ทั้งหมดมีเครื่องหมาย [แก้ n] กำกับในเนื้อหา หมายเลขหน้าที่อ้างถึงเป็นหน้าของต้นฉบับ (`parallel.pdf`) จุดที่ผิดภายในรูปซึ่งตัดมาจากต้นฉบับแก้ในภาพโดยตรงไม่ได้ ฉบับที่ถูกจึงเขียนไว้ในคำบรรยายใต้รูปแทน

[แก้ 1] สแปนของ SUM หัวข้อ 2 หน้า 7

ต้นฉบับ: $D(n) = \max(D(n/2), D(n/2) + O(1) = D(n/2) + O(1)$

แก้เป็น: $D(n) = \max(D(n/2), D(n/2)) + O(1) = D(n/2) + O(1)$

เหตุผล: วงเล็บของ max ไม่ปิด ตามนิยามสแปนของ $\|$ คือค่าสูงสุดของสแปนของสองคำสั่ง แล้วจึงบวกต้นทุน $O(1)$ ของการ fork/join

[แก้ 2] คิวของตัวจัดตาราง หัวข้อ 2 หน้า 8

ต้นฉบับ: “last-in last out”

แก้เป็น: “last-in first-out” (เข้าหลังออกก่อน)

เหตุผล: “last-in last out” ไม่ใช่วินัยคิวที่มีอยู่จริง (ถ้าเข้าหลังและออกหลังก็คือ first-in first-out ซึ่งกล่าวไปแล้ว) ประโยคนี้ยกตัวอย่างวินัยคิวสองแบบที่ตรงข้ามกัน คือ FIFO และ LIFO

[แก้ 3] ขอบเขตยูเนียน หัวข้อ 3 หน้า 10

ต้นฉบับ: “at least $1 + (1/n)^{c-1}$”

แก้เป็น: “at least $1 - (1/n)^{c-1}$”

เหตุผล: ความน่าจะเป็นมีค่าเกิน 1 ไม่ได้ ความน่าจะเป็นที่ส่วนประกอบหนึ่งล้มเหลวคือ $\leq (1/n)^c$ เมื่อใช้ขอบเขตยูเนียนกับ n ส่วน ความน่าจะเป็นที่มี*บางส่วน*ล้มเหลวคือ $\leq n(1/n)^c = (1/n)^{c-1}$ จึงได้ว่าทุกส่วนสำเร็จด้วยความน่าจะเป็น*อย่างน้อย* $1 - (1/n)^{c-1}$

[แก้ 4] ทฤษฎีบท 3.1 และบทแทรก 3.1 หน้า 10–11

ต้นฉบับ: “all possible events $X_j = \{0, 1\}$”

แก้เป็น: “$X_j \in \{0, 1\}$”

เหตุผล: X_j เป็นตัวแปรสุ่มชี้บอกซึ่งมีค่าเป็น 0 หรือ 1 ไม่ใช่เซต ความหมายที่ต้องการคือ “ภายใต้เงื่อนไขของทุกค่าที่เป็นไปได้ของ X_j” จึงต้องใช้ $\in$

[แก้ 5] บทพิสูจน์บทแทรก 3.1 หน้า 11

ต้นฉบับ: $g(n) \in O(c(f(n) + \log n)$

แก้เป็น: $g(n) \in O(c(f(n) + \log n))$

เหตุผล: วงเล็บไม่ครบคู่ (เปิดสามปิดสอง)

[แก้ 6] นิยามของ scan หัวข้อ 4.1 หน้า 12

ต้นฉบับ: $r_i = f(r_{i-1}, A_i)$ และ “the prefix $A[0, i]$”

แก้เป็น: $r_i = f(r_{i-1}, A_{i-1})$ และ $A[0 : i]$

เหตุผล: อาร์เรย์เริ่มที่ศูนย์ $(A_0, \ldots, A_{|A|-1})$ ถ้าใช้ A_i จะได้ $r_1 = f(\perp, A_1)$ ซึ่งข้าม A_0 และ $r_{|A|}$ จะอ้างถึง $A_{|A|}$ ที่ไม่มีอยู่ ประโยคถัดมาเองก็บอกว่าค่าที่ดัชนี i คือ “ผลรวมของสมาชิกทั้งหมด*ก่อน*ตัวที่ i” ซึ่งคือ $A_0 \oplus \cdots \oplus A_{i-1}$ ส่วน $A[0, i]$ ต้องเขียนเป็น $A[0 : i]$ ตามสัญกรณ์ slice ของเอกสาร

[แก้ 7] **คำอธิบาย** scanDown **หัวข้อ 4.1 หน้า 13**

ต้นฉบับ: "passes $f(s, L[m])$ to its right child. Recall that $L[m]$ is the sum of the left child"

แก้เป็น: $f(s, L[m-1])$ และ $L[m-1]$

เหตุผล: ย่อหน้าก่อนหน้าบอกว่า scanUp เขียนผลรวมของลูกซ้ายลงใน $L[m-1]$ และโค้ดในรูปที่ 2 ก็ใช้ $f(s, L[m-1])$

[แก้ 8] **โค้ด** scanDown **ในรูปที่ 2 หน้า 14**

ต้นฉบับ: scanDown$(R[0:m], L[0:m-1], s)$; $n \leftarrow |A|$; $R[0] = s$

แก้เป็น: scanDown$(R[0:m], L[0:m-1], f, s)$; $n \leftarrow |R|$; $R[0] \leftarrow s$

เหตุผล: scanDown รับสี่อาร์กิวเมนต์ (R, L, f, s) แต่การเรียกครั้งแรกส่งไปแค่สาม (ขาด f) ส่วน A ไม่ใช่พารามิเตอร์ของ scanDown ขนาดที่ต้องใช้คือ $|R|$ และการเขียนค่าลง $R[0]$ เป็นการกำหนดค่า ซึ่งเอกสารนี้ใช้ $\leftarrow$

[แก้ 9] **โค้ด** findBlock **ในรูปที่ 5 หน้า 16**

ต้นฉบับ: **return** $(A[r \times s, (r+1) \times s],\ i \times s)$

แก้เป็น: **return** $(A[r \times s : (r+1) \times s],\ r \times s)$

เหตุผล: i เป็นตัวแปรของลูป **parfor** ซึ่งใช้นอกลูปไม่ได้ บล็อกที่พบคือบล็อกที่ r ออฟเซ็ตจึงต้องเป็น $r \times s$ และ slice ต้องใช้ ":" ไม่ใช่ ","

[แก้ 10] **การค้นหาแบบ k ทาง หัวข้อ 4.3 หน้า 16**

ต้นฉบับ: $W(n) = S(n) = O(\log n)$

แก้เป็น: $W(n) = D(n) = O(\log n)$

เหตุผล: เอกสารนี้ใช้ D แทนสแปนตลอดทั้งเล่ม ไม่มีการนิยาม $S(n)$

[แก้ 11] **การผสาน หัวข้อ 4.4 หน้า 17–18**

ต้นฉบับ: $W(m) = 2W(m/2) + O(n^{1/2})$ และ "solving to $W(n) = O(n)$ and $D(n) = O(\log m)$" (สองแห่ง)

แก้เป็น: $O(m^{1/2})$ และ $W(m) = O(m)$, $D(m) = O(\log m)$ (และ $D(m) = O(\log \log m)$)

เหตุผล: ในหัวข้อนี้ขนาดข้อมูลคือ $m = |A| + |B|$ ความสัมพันธ์เวียนเกิดเขียนในรูป m ทั้งหมด การสลับไปใช้ n ทำให้ปนกันในสมการเดียว

[แก้ 12] **โค้ดในรูปที่ 9 และ 10 หน้า 19**

ต้นฉบับ: $p \leftarrow$ selectPivotA) ; **else if** $(k > |A| - |G|)$; $A[\text{rand}(n)]$

แก้เป็น: $p \leftarrow$ selectPivot(A) ; **else if** $(k \geq |A| - |G|)$; $A[\text{rand}(|A|)]$

เหตุผล: วงเล็บเปิดหายไป เงื่อนไขของ G: เมื่อ k เริ่มที่ศูนย์ (สอดคล้องกับเงื่อนไข $k < |L|$ ของ L) สมาชิกที่เท่ากับ p ครองอันดับ $|L|$ ถึง $|A| - |G| - 1$ อันดับ $k = |A| - |G|$ จึงเป็นสมาชิกตัวเล็กที่สุดของ G แต่ $k > |A| - |G|$ จะคืน p ผิดในกรณีนี้ ส่วน n ไม่ได้นิยามไว้ใน selectPivotR ขนาดที่ต้องใช้คือ $|A|$

[แก้ 13] **สแปนของ** kthSmallest **แบบสุ่ม หัวข้อ 4.5 หน้า 19**

ต้นฉบับ: $O(\log |A|^2)$

แก้เป็น: $O(\log^2 |A|)$

เหตุผล: มีการเรียกเวียนเกิด $O(\log |A|)$ ครั้ง และแต่ละครั้งเรียก filter ซึ่งมีสแปน $O(\log |A|)$ จึงได้ $O(\log |A| \cdot \log |A|) = O(\log^2 |A|)$ ส่วน $\log |A|^2 = 2 \log |A|$ ซึ่งผิด

[แก้ 14] **ความสัมพันธ์เวียนเกิดของ kthSmallest แบบเชิงกำหนด หัวข้อ 4.5 หน้า 19**

ต้นฉบับ: $D(m) = D(7/10n) + \ldots$ และ $D(m) = D(3/4n) + \ldots$

แก้เป็น: $D(n) = \ldots$ ทั้งสองแห่ง

เหตุผล: ด้านขวาและผลเฉลยใช้ตัวแปร n ด้านซ้ายจึงต้องเป็น $D(n)$ เช่นเดียวกับ $W(n)$ ในบรรทัดก่อนหน้า

[แก้ 15] **สแปนของ mergesort หัวข้อ 5.1 หน้า 20**

ต้นฉบับ: $D(n) = D(n/2) + D_{merge}O(n)$

แก้เป็น: $D(n) = D(n/2) + D_{merge}(n)$

เหตุผล: D_{merge} เป็นฟังก์ชันของขนาด เขียนคู่กับ $W_{merge}(n)$ ในบรรทัดบน "$D_{merge}O(n)$" ไม่มีความหมาย และถ้าตีความเป็น $O(n)$ ก็จะได้สแปน $O(n)$ ซึ่งขัดกับผลเฉลย $O(\log n \log\log n)$ ที่ให้ไว้

[แก้ 16] **จำนวนการเปรียบเทียบของ quicksort หัวข้อ 5.2 หน้า 21–22**

ต้นฉบับ: $B = \sum_{i=1}^{n}\sum_{j=1}^{n} A_{ij}$ และ $E[B] = \sum_i \left(\sum_{k=1}^{i-1}\frac{1}{k} + \sum_{k=1}^{n-i}\frac{1}{k}\right) < \sum_i (H_{i-1} + H_{n-i}) < \sum_i 2\ln n$

แก้เป็น: $B = \sum_i \sum_{j\neq i} A_{ij}$ และ $E[B] = \sum_i \left(\sum_{k=1}^{i-1}\frac{1}{k+1} + \sum_{k=1}^{n-i}\frac{1}{k+1}\right) = \sum_i \big((H_i - 1) + (H_{n-i+1} - 1)\big) \leq \sum_i 2\ln n = 2n\ln n$

เหตุผล: มีสามจุด (1) สมาชิกไม่ถูกเปรียบเทียบกับตัวเอง ต้องตัด $j = i$ ออก (ถ้านับ $A_{ii} = 1$ จะเกินมา n) (2) $E[A_{ij}] = 1/(1 + |j - i|)$ เมื่อ $|j - i| = k$ จึงได้ $1/(k + 1)$ ไม่ใช่ $1/k$ บรรทัดนี้ใช้เครื่องหมาย $=$ ไม่ได้ (3) ด้วย $1/k$ ผลรวม $\sum_{k=1}^{i-1} 1/k$ เท่ากับ H_{i-1} พอดี บรรทัดถัดไปจึงต้องเป็น $=$ ไม่ใช่ $<$ ฉบับที่แก้คำนวณค่าจริง $(H_i - 1) + (H_{n-i+1} - 1)$ แล้วใช้ $H_m - 1 \leq \ln m$ ผลสรุป $E[B] \leq 2n\ln n$ ยังเหมือนเดิม

[แก้ 17] **ความลึกของ quicksort หัวข้อ 5.2 หน้า 22**

ต้นฉบับ: $D_i = \sum_{j=0}^{n} A_{ji}$ และ $E[D_i] = \sum_{j=0}^{n} \frac{1}{1+|i-j|} < 2\ln n$

แก้เป็น: $D_i = \sum_{j=1, j\neq i}^{n} A_{ji}$ และ $E[D_i] = \sum_{j\neq i} \frac{1}{1+|i-j|} \leq 2\ln n$

เหตุผล: อันดับเริ่มที่ 1 (ผลรวมของ B ใช้ $i, j = 1..n$) $j = 0$ จึงไม่มีอยู่ และตัวสมาชิกเองไม่นับเป็นตัวแบ่ง "เหนือ" ตัวมันเอง ผลรวมคือ $(H_i - 1) + (H_{n-i+1} - 1) \leq \ln i + \ln(n - i + 1) \leq 2\ln n$

[แก้ 18] **counting sort หัวข้อ 5.4 หน้า 23**

ต้นฉบับ: "the $O(n \log n)$ lower bound that comparison sorts have"

แก้เป็น: $\Omega(n \log n)$

เหตุผล: ขอบเขตล่างเขียนด้วย Ω ส่วน O หมายถึงขอบเขตบน

[แก้ 19] **radix sort หัวข้อ 5.4 หน้า 24**

ต้นฉบับ: "For a radix of size m, round i ... round key $(k/n^i) \bmod n$"

แก้เป็น: "For a radix of size n"

เหตุผล: คีย์ประจำรอบคือ $(k/n^i) \bmod n$ ซึ่งเป็นฐาน n และประโยคถัดไปบอกว่า "round key is bounded by n" ฐานจึงมีขนาด n ไม่ใช่ m

[แก้ 20] **counting sort แบบขนาน หัวข้อ 5.4 หน้า 24**

ต้นฉบับ: "breaking the input into blocks of size $b = n/m$"

แก้เป็น: "breaking the input into $b = n/m$ blocks of size m"

เหตุผล: ในรูปที่ 13 บรรทัด 19–21 $b \leftarrow |K|/m$ คือ*จำนวน*บล็อก และแต่ละบล็อกคือ $K[mi : m(i + 1)]$ ซึ่งมีขนาด m ย่อหน้าถัดไปก็วิเคราะห์ว่า "when called on m keys" และ "across the $|K|/m$ parallel calls"

[แก้ 21] **integer sort หัวข้อ 5.4 หน้า 24**

ต้นฉบับ: $[0 : n \log^k n)]$

แก้เป็น: $[0 : n \log^k n]$

เหตุผล: มีวงเล็บปิด ")" เกินมาหนึ่งตัว

[แก้ 22] **โค้ด EDGEMAP ในรูปที่ 14 หน้า 26**

ต้นฉบับ: $N[i] \leftarrow \{v \in N^+(G, v) \mid f_u(u, v)\}$

แก้เป็น: $N[i] \leftarrow \{v \in N^+(G, u) \mid f_u(u, v)\}$ เมื่อ $u = U[i]$

เหตุผล: ต้องหาจุดข้างเคียงขาออกของ u (จุดยอดใน frontier) ไม่ใช่ของ v ซึ่งเป็นตัวแปรที่กำลังนิยาม และในโค้ด u ไม่ได้ถูกนิยามจาก $U[i]$ ตามที่ข้อความบรรยาย "For each u we filter its out-neighbors"

[แก้ 23] **คำอธิบาย BFS หัวข้อ 6.2 หน้า 26**

ต้นฉบับ: "a parents array to all -1"

แก้เป็น: ให้แต่ละจุดยอดเป็นจุดยอดแม่ของตัวเอง ($P_v \leftarrow v$)

เหตุผล: ข้อความขัดกับโค้ดในรูปที่ 15 ซึ่งกำหนด $P_v \leftarrow v$ (โค้ดคืนค่า P เป็นผลลัพธ์ จึงยึดตามโค้ด) ทั้งสองแบบใช้ได้ แต่ต้องตรงกัน

[แก้ 24] **LDD แบบลำดับ หัวข้อ 6.3 หน้า 28**

ต้นฉบับ: "a $(\beta, O(\log n/\beta)$-decomposition"

แก้เป็น: $(\beta, O(\log n/\beta))$

เหตุผล: วงเล็บไม่ครบคู่

[แก้ 25] **การแจกแจงเลขชี้กำลัง หัวข้อ 6.3 หน้า 28**

ต้นฉบับ: $\mathbf{Pr}[X = x] = \beta e^{-\beta x}$

แก้เป็น: ฟังก์ชันความหนาแน่น $f_X(x) = \beta e^{-\beta x}$

เหตุผล: สำหรับตัวแปรสุ่มต่อเนื่อง $\mathbf{Pr}[X = x] = 0$ ทุก x ค่า $\beta e^{-\beta x}$ คือความหนาแน่น (และอาจเกิน 1 ได้เมื่อ $\beta > 1$ ซึ่งเป็นความน่าจะเป็นไม่ได้) ประโยคถัดไปที่ว่า "by integration" ก็ยืนยันว่านี่คือความหนาแน่น

[แก้ 26] **จุดเริ่มของ LDD แบบขนาน หัวข้อ 6.3 หน้า 28**

ต้นฉบับ: "The initial breadth-first search starts at the vertex with the largest start time, $\delta_{\max}$"

แก้เป็น: เริ่มที่จุดยอดที่มี δ_v มากที่สุด ($= \delta_{\max}$) ซึ่งมีเวลาเริ่มต้น*น้อย*ที่สุด $T_v = 0$

เหตุผล: เวลาเริ่มต้นคือ $T_v = \delta_{\max} - \delta_v$ จุดยอดที่สุ่มได้ $\delta_v = \delta_{\max}$ จึงมี $T_v = 0$ ซึ่งเริ่มก่อนใคร ส่วนจุดยอดที่มีเวลาเริ่มต้นมากที่สุดจะเริ่มหลังสุด

[แก้ 27] **spanner หัวข้อ 6.5 หน้า 32**

ต้นฉบับ: $\left(1 - e^{-2 \lg n/2k}\right)^{l-1} = \left(1 - n^{-1/k}\right)^{l-1}$

แก้เป็น: $\left(1 - e^{-2 \ln n/2k}\right)^{l-1}$

เหตุผล: $\beta = \ln n/2k$ (ใช้ ln ตามที่นิยามไว้ในย่อหน้าก่อน) และ $e^{-\ln n/k} = n^{-1/k}$ เป็นจริงเฉพาะเมื่อใช้ลอการิทึมธรรมชาติ ถ้าใช้ lg จะได้ $n^{-1/(k \ln 2)}$ ซึ่งไม่เท่ากับด้านขวา

[แก้ 28] **โค้ด LUBYMIS ในรูปที่ 19 หน้า 33 และ 35**

ต้นฉบับ: $I \leftarrow \{v \in V \mid \rho(v) < \min_{u \in N(G,v)} \rho(u)\}$ และ "taking the minimum of the neighbors to determine I"

แก้เป็น: $I \leftarrow \{v \in V \mid \rho(v) > \max_{u \in N(G,v)} \rho(u)\}$ และ "ค่าสูงสุดของจุดข้างเคียง"

เหตุผล: คำอธิบายบอกว่า I คือ "local maximum" ("all of their neighbors have lower priorities") และบทพิสูจน์ทฤษฎีบทใช้เหตุการณ์ที่ u มีลำดับความสำคัญ*สูงสุด* โค้ดเลือกค่า*ต่ำสุด*เฉพาะที่ ซึ่งยังให้ MIS ได้ (สมมาตรกัน) แต่ขัดกับข้อความและบทพิสูจน์ จึงแก้โค้ดให้ตรงกับข้อความ

[แก้ 29] อ้างอิงรูป หัวข้อ 6.6 หน้า 33

ต้นฉบับ: "Pseudocode is given in Figure 20"

แก้เป็น: รูปที่ 19

เหตุผล: รูปที่ 20 คือกราฟตัวอย่างสำหรับบทพิสูจน์ รหัสเทียมของ Luby อยู่ในรูปที่ 19

[แก้ 30] การวิเคราะห์ต้นทุนของ Luby หัวข้อ 6.6 หน้า 35

ต้นฉบับ: "Let F_i be a random variable for the fraction *removed* on step i ... $M_i = m\Pi_0^{i-1} F_i$"

แก้เป็น: F_i คือสัดส่วนที่*เหลืออยู่* และ $M_i = m\prod_{j=0}^{i-1} F_j$ (และ $\bar{E}[F_j]$ ในผลคูณ, $m^{1+k\lg\beta}$)

เหตุผล: จำนวนที่เหลือคือ m คูณสัดส่วนที่*เหลือ* ไม่ใช่สัดส่วนที่ถูกนำออก (ถ้านำออกครึ่งหนึ่ง ทั้งสองค่าเท่ากัน $= 1/2$ จึงอาจไม่เห็นความผิด) ข้อสมมติ $E[F_i] \le \beta = 1/2$ ก็สอดคล้องกับ "เหลือไม่เกินครึ่ง" ตามทฤษฎีบทที่ว่านำออก*อย่างน้อย*ครึ่งหนึ่ง ตัวแปรในผลคูณต้องเป็นดัชนีของผลคูณ j ไม่ใช่ i และ $\log\beta$ ต้องเป็น $\lg\beta$ ให้ตรงกับ $m^{k\lg\beta}$ ในบรรทัดเดียวกัน

[แก้ 31] ค่าคงตัวของ treap หัวข้อ 7.1 หน้า 37

ต้นฉบับ: "the priority at each node is no greater than the priority of its two children"

แก้เป็น: "no less than" (ไม่น้อยกว่า)

เหตุผล: อัลกอริทึม join ของ treap (หัวข้อ 7.2.4) เลือกคีย์ที่มีลำดับความสำคัญ*สูงสุด*เป็นราก และบอกว่าคีย์ที่มีลำดับความสำคัญสูงสุด "remains at the root" ดังนั้นแต่ละโหนดต้องมีลำดับความสำคัญไม่น้อยกว่าลูก

[แก้ 32] อ้างอิงตาราง หัวข้อ 7.1 หน้า 37

ต้นฉบับ: "summarized in Figure 2"

แก้เป็น: ตารางที่ 2 (Table 2)

เหตุผล: รูปที่ 2 คือฟังก์ชัน scan สรุปสัญกรณ์อยู่ใน Table 2

[แก้ 33] หัวข้อ 7.2 หน้า 38

ต้นฉบับ: "We show in this thesis that ..."

แก้เป็น: "มีการแสดงไว้ใน [8] ว่า ..."

เหตุผล: เอกสารนี้ไม่ใช่วิทยานิพนธ์ และไม่ได้พิสูจน์ข้อความนี้เอง (ข้อความหลงมาจากวิทยานิพนธ์ต้นทาง) บทพิสูจน์อยู่ใน [8] ซึ่งหัวข้อ 7.4 อ้างถึงเรื่องเดียวกัน

[แก้ 34] บทพิสูจน์ join ของ RB หัวข้อ 7.2.2 หน้า 41

ต้นฉบับ: "proportional to the depth of c in T_r"

แก้เป็น: "in T_l"

เหตุผล: c เป็นโหนดบนสันขวาของ T_l ("follows the right spine of T_l until it finds a black node c")

[แก้ 35] บทพิสูจน์ join ของ RB หัวข้อ 7.2.2 หน้า 41

ต้นฉบับ: กรณีที่ 3 เขียนว่า "(Step 3 Case 1)"

แก้เป็น: "(Step 3 Case 2)"

เหตุผล: กรณีที่ 2 และ 3 ของรายการเป็นคนละกรณีกัน (รากเดิมสีแดง vs. สีดำ) ตรงกับ "1)" และ "2)" ของ Step 3 ในรูปที่ 24 แต่ต้นฉบับเขียนว่า Case 1 ซ้ำกัน

[แก้ 36] join ของ WB หัวข้อ 7.2.3 หน้า 42

ต้นฉบับ: "creates a new $node(c, r, T_r)$"

แก้เป็น: $node(c, k, T_r)$

เหตุผล: อาร์กิวเมนต์ตรงกลางต้องเป็นตัวกลาง (pivot) ของ join ซึ่งในโค้ดรูปที่ 25 คือ k ไม่มีตัวแปร r ในบริบทนี้

[แก้ 37] การอ้างอิง หัวข้อ 7.1 หน้า 37 และหัวข้อ 7.2.3 หน้า 43

ต้นฉบับ: “The proof of this lemma can be found in ??” และ “[19, ?]” (สองแห่ง)

แก้เป็น: “... found in [8]” และ [19]

เหตุผล: ต้นฉบับมีการอ้างอิงที่ LaTeX หาไม่พบ (แสดงเป็น ?? และ ?) บทพิสูจน์ join ของต้นไม้สมดุลน้ำหนักอยู่ใน [8] (Blelloch, Ferizovic, Sun, *Just join for parallel ordered sets*) ซึ่งเป็นงานที่เสนอ join ทั้งสี่แบบในหัวข้อนี้ ส่วนรายการที่สองใน [19, ?] ระบุไม่ได้ว่าคืออะไร จึงตัดออกเหลือ [19]

[แก้ 38] join2 หัวข้อ 7.3.1 หน้า 47

ต้นฉบับ: “the result of dropping k in T_L”

แก้เป็น: “in T_l”

เหตุผล: SPLIT_LAST ทำงานบนต้นไม้ซ้ายของ join2 ซึ่งเขียนเป็น T_l ในประโยคก่อนหน้า (T_L ใช้ในบริบทของ split คนละความหมาย)

[แก้ 39] อัลกอริทึม union หัวข้อ 7.4 หน้า 48

ต้นฉบับ: “... (denoted as R_1). ger) than $k(T_1)$. Then two recursive calls ...”

แก้เป็น: ตัด “ger) than $k(T_1)$.” ออก

เหตุผล: เป็นเศษข้อความที่หลงเหลือจากการแก้ไข (ส่วนท้ายของคำว่า “larger) than $k(T_1)$”) ไม่มีความหมายในประโยค

[แก้ 40] โค้ดในรูปที่ 31 หน้า 50

ต้นฉบับ: `foreach_index`: $\phi(e, left)$ `multi_insert_s`: $d = (b < m)$ and $(k(A[b]) > k(e))$; `r->lc`, `r->rc`; $A + b - d$ `multi_insert`: เรียก `multi_insert_sorted`

แก้เป็น: $\phi(e, s + left)$ $d = (b < m)$ and $(k(A[b]) = k(e))$; $lc(T)$, $rc(T)$; $A + b + d$ เรียก `multi_insert_s`

เหตุผล: `foreach_index`: รากอยู่ที่ดัชนี $s + left$ (ทางขวาก็ส่ง $s + 1 + left$ ตามนี้) `multi_insert_s`: ข้อความบอกว่า “d is a bit denoting if k appears in A” จึงต้องเทียบความเท่ากัน เมื่อคีย์ของรากอยู่ใน A ($d = 1$) ต้องข้าม $A[b]$ ส่วนขวาจึงเริ่มที่ $A + b + d$ ขนาด $m - b - d$ ตัวแปร `r` ไม่มีในฟังก์ชันนี้ (ต้นไม้คือ T) และฟังก์ชันชื่อ `multi_insert_s` ไม่ใช่ `multi_insert_sorted`

[แก้ 41] PRAM หัวข้อ 8.1 หน้า 52

ต้นฉบับ: “The Parallel Random Access RAM (PRAM)” และ “differs from the MP-PRAM”

แก้เป็น: “Parallel Random Access Machine” และ “MP-RAM”

เหตุผล: RAM ย่อมาจาก Random Access Machine อยู่แล้ว (หัวข้อ 2) และแบบจำลองของเอกสารชื่อ MP-RAM ไม่มี MP-PRAM

[แก้ 42] บทพิสูจน์การจัดตารางแบบละโมบ หัวข้อ 8.4 หน้า 54

ต้นฉบับ: “Also $j < p$ since this step is incomplete”

แก้เป็น: เนื่องจากขั้นนี้ไม่สมบูรณ์ จำนวนจุดยอดที่พร้อมจึงน้อยกว่า p และตารางแบบละโมบเยี่ยมจุดยอดที่พร้อมทั้งหมด

เหตุผล: j คือหมายเลข*ระดับ* การเทียบ j กับจำนวนตัวประมวลผล p ไม่มีความหมาย สิ่งที่บทพิสูจน์ต้องการคือ ขั้นที่ไม่สมบูรณ์เยี่ยมน้อยกว่า p จุด ซึ่งสำหรับตารางแบบ p-ละโมบ หมายความว่ามีจุดยอดที่พร้อม $r < p$ จุดและทั้งหมดถูกเยี่ยม จุดยอดที่เหลือในระดับ j ซึ่งพร้อมทั้งหมดจึงถูกเยี่ยมครบในขั้นนี้

[แก้ 43] **เอกสารอ้างอิง [1] หน้า 57**

ต้นฉบับ: “Implementing sets effciently in a functional language”

แก้เป็น: “efficiently”

เหตุผล: สะกดผิด ชื่อบทความจริงคือ *Implementing Sets Efficiently in a Functional Language*

คำผิดภาษาอังกฤษที่หายไปเองในการแปล (ไม่กระทบความหมาย)

“is intended an introduction” (หน้า 3), “its work is work asymptotically” (หน้า 5), “generic sence” (หน้า 8), “definiton”, “w.h.p.„” (หน้า 9), “overal”, “The goals is” (หน้า 9–10), “every subset than includes” (หน้า 10), “which is is tighter” (หน้า 12), “from from the leaves”, “position of of the midpoint”, “the identify I” (หน้า 13), “maintains it that”, “taked” (หน้า 14), “runs in in”, “sublock” (หน้า 17), “produces”, “showin” (หน้า 17), “an sequences” (หน้า 18), “We first Let”, “the even” (หน้า 21), “the the span” (หน้า 22), “ouput”, “These corresponds” (หน้า 23), “To uses”, “this comes to adds to” (หน้า 24), “the the total” (หน้า 28), “the edge is bridges”, “andfor for”, “Weshowed” (หน้า 30–31), “There a conjectured”, “graph of with”, “on it cycles”, “many sequential and parallel that” (หน้า 31), “each cluster have”, “bounded by is” (หน้า 32), “expect the bottommost level” (except), “two input tree are” (หน้า 40–41), “weighted-balanced” (หน้า 43), “spliting”, “for help” (หน้า 45), “an single element”, “still need” (หน้า 48), “cost time”, “first follow” (หน้า 51), “differes” (หน้า 52), “it does not it does not” (หน้า 54), “corrolary” (หน้า 56)

ข้อสังเกตเพิ่มเติม (ไม่ได้แก้ในเนื้อหา)

- **บทพิสูจน์บทแทรก 3.1 บรรทัดแรก** ใช้ $\frac{f(n)}{2f(n)+\ln n}$ ซึ่งไม่ใช่ค่า $\frac{ef(n)}{g(n)} = \frac{f(n)}{2c(f(n)+\lg n)}$ โดยตรง แต่เป็นขอบเขตบนที่ถูกต้อง (เพราะ $c \geq 1$ และ $\lg n > \ln n$) จึงไม่ใช่คำผิด
- **PCW ในหัวข้อ 8.1** ให้ตัวประมวลผลที่มีหมายเลขสูงที่สุดชนะ ขณะที่ตำราส่วนใหญ่ (และเอกสาร PRAM Algorithms ของ Chatterjee–Prins) ใช้หมายเลขต่ำที่สุด ทั้งสองแบบเป็นแบบจำลองที่สมมาตรกัน จึงไม่ถือว่าผิด
- **สแปนของ build (หัวข้อ 7.5)** ข้อความให้ $O(S_{\text{sort}}(n) + S_{\text{remove}}(n) + \log n)$ ซึ่งเมื่อการเรียงลำดับมีสแปน $O(\log n)$ จะได้ $O(\log n)$ แต่ตารางที่ 3 ระบุ $O(\log^2 n)$ ตารางน่าจะสมมติการเรียงลำดับที่มีสแปน $O(\log^2 n)$ (เช่นบนแบบจำลอง fork ทวิภาค) ไม่แน่ชัดว่าฝั่งใดคือเจตนาของผู้เขียน จึงคงไว้ทั้งคู่

เอกสารอ้างอิง

[1] S. Adams. Implementing sets efficiently[แก้ 43] in a functional language. Technical Report CSTR 92-10, University of Southampton, 1992.

[2] S. Adams. Efficient sets—a balancing act. *Journal of functional programming*, 3(04), 1993.

[3] G. Adelson-Velsky and E. M. Landis. An algorithm for the organization of information. *USSR Academy of Sciences*, 145:263–266, 1962. In Russian, English translation by Myron J. Ricci in Soviet Doklady, 3:1259-1263, 1962.

[4] R. Ahmed, G. Bodwin, F. D. Sahneh, K. Hamm, M. J. Latifi Jebelli, S. Kobourov, and R. Spence. Graph spanners: A tutorial review. *Computer Science Review*, 37:100253, 2020.

[5] B. Awerbuch. Complexity of network synchronization. *Journal of the ACM (JACM)*, 32(4):804–823, 1985.

[6] R. Bayer. Symmetric binary b-trees: Data structure and maintenance algorithms. *Acta Informatica*, 1:290–306, 1972.

[7] S. Beamer, K. Asanović, and D. Patterson. Direction-optimizing breadth-first search. In *SC*, 2012.

[8] G. E. Blelloch, D. Ferizovic, and Y. Sun. Just join for parallel ordered sets. In *Proc. 28th ACM Symposium on Parallelism in Algorithms and Architectures*, pages 253–264. ACM, 2016.

[9] G. E. Blelloch, J. T. Fineman, Y. Gu, and Y. Sun. Optimal parallel algorithms in the binary-forking model. *CoRR*, abs/1903.04650, 2019.

[10] G. E. Blelloch and M. Reid-Miller. Fast set operations using treaps. In *Proc. ACM Symp. on Parallel Algorithms and Architectures (SPAA)*, pages 16–26, 1998.

[11] M. R. Brown and R. E. Tarjan. A fast merging algorithm. *Journal of the ACM (JACM)*, 26(2):211–226, 1979.

[12] R. Cole and U. Vishkin. Faster optimal parallel prefix sums and list ranking. *Information and Computation*, 81(3):334–352, 1989.

[13] T. H. Cormen, C. E. Leiserson, R. L. Rivest, and C. Stein. *Introduction to Algorithms (3. ed.)*. MIT Press, 2009.

[14] Y. Gu, J. Shun, Y. Sun, and G. E. Blelloch. A top-down parallel semisort. In *SPAA*, 2015.

[15] F. K. Hwang and S. Lin. A simple algorithm for merging two disjoint linearly ordered sets. *SIAM J. on Computing*, 1(1):31–39, 1972.

[16] S. Marlow et al. Haskell 2010 language report. Available online http://www.haskell.org/ (May 2011), 2010.

[17] G. L. Miller, R. Peng, A. Vladu, and S. C. Xu. Improved parallel algorithms for spanners and hopsets. In *SPAA*, 2015.

[18] G. L. Miller, R. Peng, and S. C. Xu. Parallel graph decompositions using random shifts. In *SPAA*, pages 196–203, 2013.

[19] J. Nievergelt and E. M. Reingold. Binary search trees of bounded balance. *SIAM J. Comput.*, 2(1):33–43, 1973.

[20] D. Peleg and A. A. Schäffer. Graph spanners. *J. Graph Theory*, 13(1):99–116, 1989.

[21] S. Rajasekaran and J. H. Reif. Optimal and sublogarithmic time randomized parallel sorting algorithms. *SIAM Journal on Computing*, 1989.

[22] R. Seidel and C. R. Aragon. Randomized search trees. *Algorithmica*, 16:464–497, 1996.

[23] Y. Shiloach and U. Vishkin. Finding the maximum, merging, and sorting in a parallel computation model. 2(1), 1981.

[24] J. Shun and G. E. Blelloch. Ligra: A lightweight graph processing framework for shared memory. In *PPoPP*, 2013.

[25] J. Shun, L. Dhulipala, and G. Blelloch. A simple and practical linear-work parallel algorithm for connectivity. In *SPAA*, pages 143–153, 2014.

[26] Y. Sun, D. Ferizovic, and G. E. Blelloch. PAM: parallel augmented maps. In *ACM Symposium on Principles and Practice of Parallel Programming (PPoPP)*, 2018.

[27] R. E. Tarjan. *Data structures and network algorithms*, volume 44 of CBMS-NSF regional conference series in applied mathematics. SIAM, 1983.

[28] J. D. Ullman. Np-complete scheduling problems. *J. Comput. Syst. Sci.*, 10(3):384–393, 1975.